Aula 06 - Prof. Vinícius Borges e Raphael Lacerda

SEFAZ-AL (Auditor Fiscal da Administração Tributária Estadual - AFTE) Desenvolvimento de Sistemas - 2026 (Pós-Edital)

Autor:
Vinicius Borges

27 de Agosto de 2026

02648216103 - HENRIQUE AMORIM BARRETO

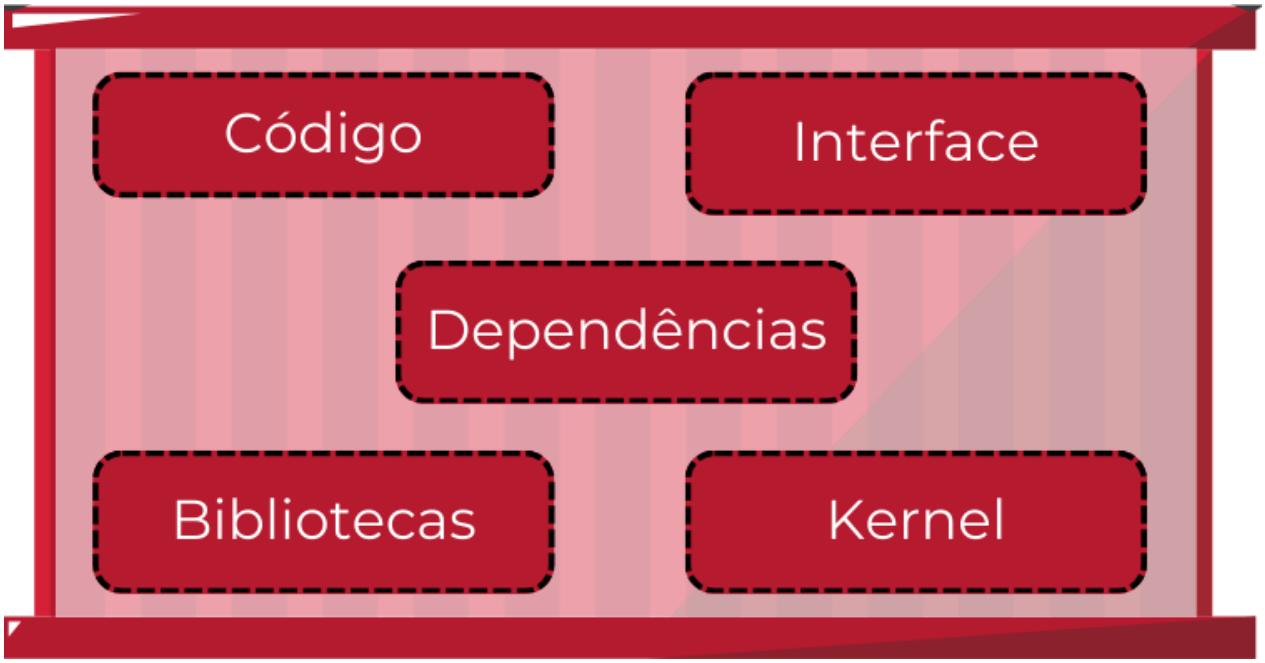
Índice

1) Contêineres - Teoria 3
2) Docker - Teoria 10
3) Docker - Questões Comentadas 50
4) Docker - Lista de Questões 65
5) Kubernetes - Teoria 74
6) Kubernetes - Questões Comentadas 112
7) Kubernetes - Lista de Questões 124
8) Podman - Teoria 132
9) Podman - Questões comentadas 150
10) Podman - Lista de questões 158

CONTÊINERES

Conceitos Gerais

Um contêiner é, basicamente, uma **imagem que encapsula todo o código, as dependências, bibliotecas e requisitos** de uma aplicação, para que possamos rodar e replicar nossa aplicação rapidamente e com segurança.



Com essa abordagem, temos extrema facilidade de movimentar a aplicação entre ambientes internos (desenvolvimento, teste, produção etc.), entre servidores externos, escalabilidade, adaptabilidade, entre outros. Para essa compatibilidade, os contêineres oferecem uma forma de virtualização de sistema operacional, que conta com seu Kernel embutido, carregando propriedades como os *namespaces* e *cgroups* em Linux, ou os *silos* em Windows.

Inicialmente, os contêineres surgiram como uma ferramenta no Linux, os Linux Containers (LNX), reunindo o conjunto de arquivos necessários para rodar determinado processo, de forma isolada. Com isso, conseguimos rodar esses processos de forma autônoma e completa, isolados um do outro – quase como uma virtualização.

Acontece que o LNX é planejado para rodar apenas em sistemas operacionais Linux e, com o tempo, a demanda por soluções mais completas exigiu uma mudança de abordagem. Nesse contexto que surge o **Docker**, o principal criador de contêineres atuais. O seu processo consiste em duas etapas – a criação da **imagem**, que é o arquivo-base para a criação de contêineres, e os contêineres em si, que são as imagens sendo rodadas em tempo de execução. É como se as

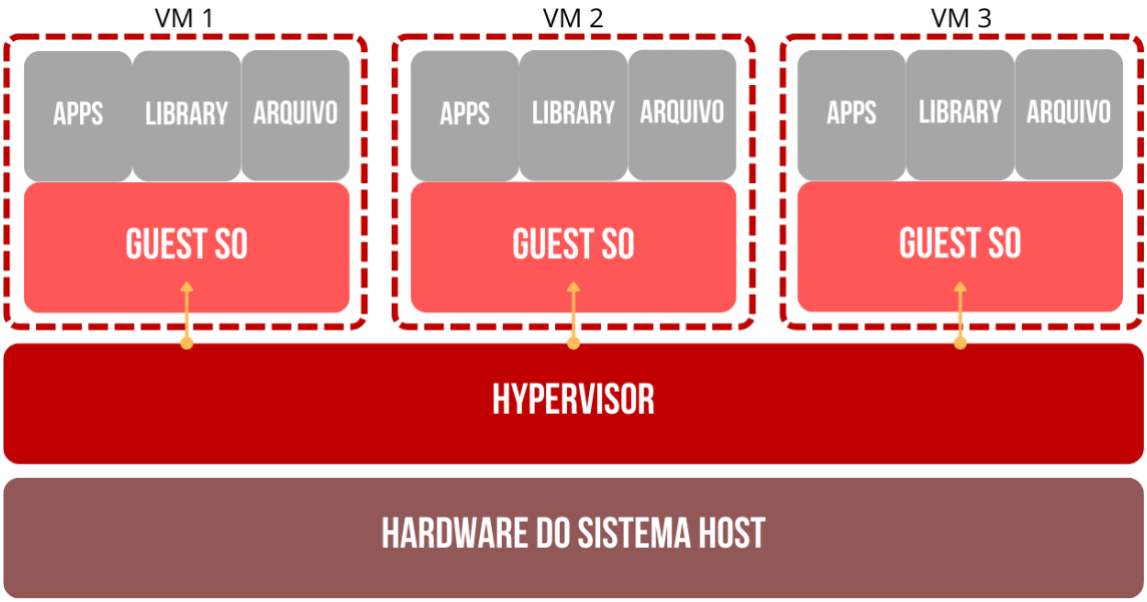
imagens fossem o arquivo PDF dessa aula, alocado na sua memória, e os contêineres a aula aberta, permitindo que você possa ler.

02648216103 - HENRIQUE AMORIM BARRETO

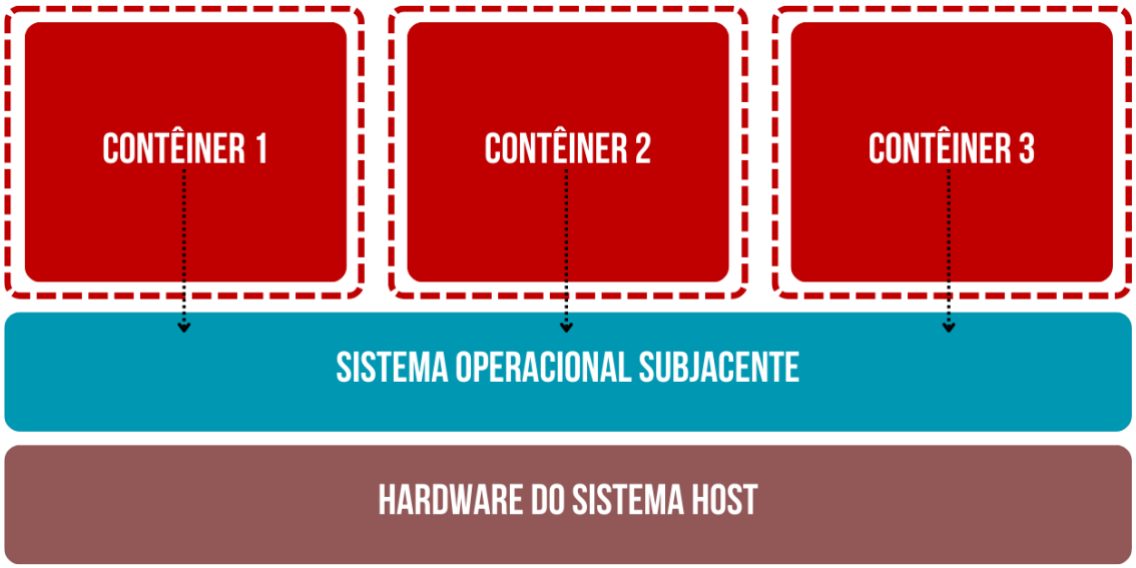
Contêiner *vs* Virtualização

"Mas Felipe, isso não é a mesma coisa que a virtualização?"

Não, meu caro aluno! Bom, eles são bem próximos – diria até que são irmãos, mas existem algumas diferenças basilares entre contêineres e virtualização que os tornam bem diferentes. Quando pensamos em virtualização, estamos pensando nas máquinas virtuais, a criação de todo um ambiente único, que envolve literalmente um computador virtual. Sistema operacional, bibliotecas, disco rígido virtual, entre outros. Tudo isso orquestrado por um *hypervisor*, independentemente do tipo.



Já os contêineres virtualizam o sistema operacional subjacente (do *host*), passando, ao aplicativo, a percepção que ele conta com esse sistema operacional – incluindo memória, CPU, arquivos e conexões de rede, tudo para ele. Porém, como temos uma abstração entre o sistema subjacente e o contêiner, podemos replicar essa imagem em qualquer sistema operacional.



Aqui, não temos o uso de *hypervisors*, já que os contêineres compartilham o sistema operacional com o *host*, e, graças a isso, podemos deixar as bibliotecas necessárias alocadas dentro do sistema operacional. Dessa forma, conseguimos imagens de sistema extremamente leves e eficientes.
Um defeito dos contêineres é que, apesar de serem altamente portáveis – isso é, transportados e movidos entre sistemas -, por serem desenvolvidos com a virtualização do sistema operacional do *host*, apresentam baixa flexibilidade. Um contêiner desenvolvido para um sistema operacional só irá rodar nele. Então, um contêiner Linux só roda em Linux, um Windows só roda em Windows, e assim por diante.

Vamos fazer uma breve comparação?

Comparação	Contêiner	Virtualização
Isolamento	Usa *namespaces* ou *silos*	Usa hipervisores
Overhead	Menor	Maior
Velocidade de Inicialização	Maior	Menor
Tamanho	Pequeno	Grande
Portabilidade	Alta	Média
Flexibilidade	Menor	Maior
Implementação	Simples	Complexa
Persistente	Não	Sim

(VUNESP/ALESP/2022) No contexto de virtualização por contêineres, por exemplo, por meio do software Docker, uma imagem de uma aplicação "conteinerizada" é:

a) um repositório centralizado na Internet, aberto ou não, onde se pode hospedar arquivos relativos aplicação.
b) um arquivo de log, contendo todos os eventos relevantes sobre a execução da aplicação, desde o instante inicial.
c) uma unidade de empacotamento, como um arquivo em formato específico, que contém os arquivos necessários para a execução da aplicação e suas dependências.
d) uma cópia de segurança (backup) de todo o código-fonte da aplicação.
e) uma instância em execução da aplicação.

Comentários:

Uma *imagem* é a composição básica de um contêiner, a "versão gênesis". Ela é o arquivo criado quando empacotamos todos os componentes e dependências necessárias, e é a partir dessa imagem que são criados os contêineres. *(Gabarito: Letra C)*

02648216103 - HENRIQUE AMORIM BARRETO

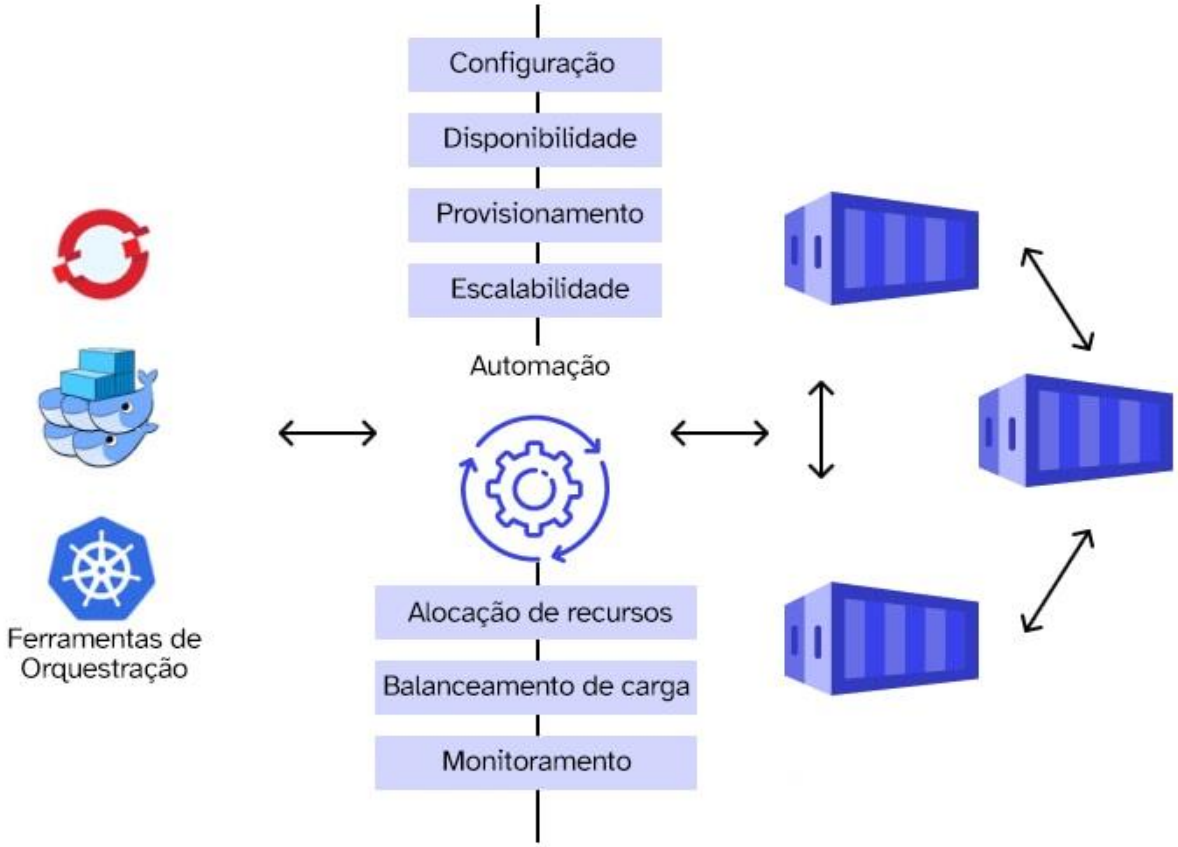
Orquestração de Contêineres

Dificilmente rodaremos uma aplicação em apenas um contêiner – e, por isso, precisamos de ferramentas que vão pegar as imagens, criar múltiplos contêineres e gerenciá-los. Esse processo, realizado de forma automatizada (como é o *standard* do DevOps) é denominado **orquestração de contêineres**, e é desempenhado com maestria por ferramentas como o Docker Swarm ou o Kubernetes.

As ferramentas de orquestração de contêineres visam **simplificar o gerenciamento da infraestrutura de contêineres**, automatizando seu ciclo de vida completo, desde o provisionamento e o agendamento até a implantação e a exclusão. As organizações podem se beneficiar da conteinerização em grande escala sem incorrer em despesas adicionais de manutenção.

Para implementarmos a orquestração precisamos, primariamente, de uma solução que irá gerar as imagens e criar os contêineres a partir dela – normalmente, é usado o Docker para essa solução. Além disso, precisamos implantar a ferramenta de orquestração nos nós que servirão para rodar a aplicação – designando um nó principal para fazer a coordenação e o gerenciamento, usualmente através de uma interface gráfica, denominado "nó mestre".

O gerenciador verifica a necessidade de novos contêineres, cria-os a partir das imagens do contêiner (com suas dependências e bibliotecas), e verifica o local em que deve ser alocado esse contêiner gerado – o nó específico (chamado de nó trabalhador, ou *worker node) ou* o segmento da rede. Esse processo é denominado de **agendamento**, ou, do inglês, *scheduling*.


Configuração
Disponibilidade
Provisionamento
Escalabilidade
Automação
Alocação de recursos
Balanceamento de carga
Monitoramento
Ferramentas de Orquestração

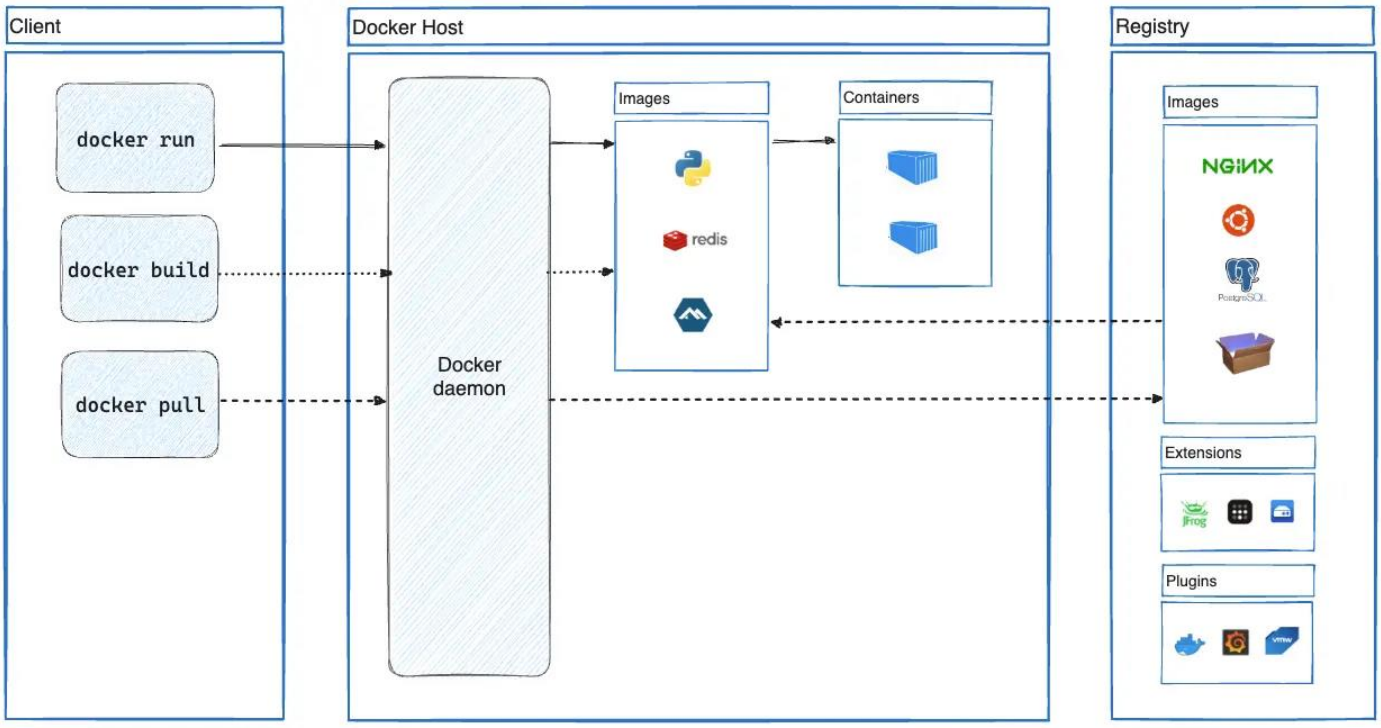
DOCKER

Conceitos Gerais

Docker é uma plataforma aberta (open source), que objetiva desenvolver, entregar e rodar aplicações, através de contêineres. Ele permite você separar as aplicações da infraestrutura subjacente, de forma que as entregas de aplicativos se tornem mais rápidas e eficientes.

O Docker usa uma arquitetura *cliente-servidor*. O cliente Docker se comunica com o daemon Docker, que faz toda o trabalho pesado de construir, rodar e distribuir os contêineres. O cliente e o *daemon* podem rodar num mesmo sistema, ou podemos ter conexões externas.

A comunicação entre cliente e *daemon* é feita através de chamadas HTTP numa API REST, rodando sobre *sockets* UNIX ou numa interface de rede. Podemos também usar o Docker Compose, um cliente que permite trabalhar com um conjunto amplo de contêineres ao mesmo tempo.



Sockets Unix, também conhecidos como "Unix domain sockets", é um mecanismo de comunicação interprocessos (IPC) que permite a troca de dados entre processos em um sistema operacional Unix-like, como Linux ou macOS, usando o sistema de arquivos local. Eles são similares aos sockets de rede, mas operam apenas na máquina local e não usam a pilha de protocolos de rede.

Dessa forma, o *Docker daemon* (*dockerd*) escuta as solicitações HTTP através da API, e gerência os objetos do Docker, como imagens, contêineres, redes, entre outros. Podemos, também, ter comunicação entre Docker *daemons*.

> *Um daemon é um tipo de processo de computador que opera em segundo plano, geralmente sem intervenção direta do usuário, realizando diversas tarefas essenciais para o funcionamento do sistema operacional ou de aplicativos específicos. Esses processos são iniciados durante o boot do sistema e permanecem em execução continuamente, aguardando por eventos específicos para responder, como solicitações de serviços de rede, monitoramento de dispositivos ou execução de tarefas programadas.*

Já o cliente Docker é a forma primária com que usuários interagem com o Docker. Quando digitamos os comandos, como *docker run*, o cliente manda esses comandos para o *daemon*, através da API, que os executa. Um cliente Docker pode se comunicar com diversos *daemons* diferentes.

O Docker fornece uma ampla base de objetos:



Objeto	Descrição
Imagens	Uma imagem Docker é um artefato de distribuição que contém todos os elementos necessários para executar uma aplicação, incluindo o sistema operacional, bibliotecas, código-fonte e dependências. As imagens são construídas a partir de um arquivo chamado Dockerfile, que descreve passo a passo como a imagem deve ser montada. As imagens Docker são usadas como base para criar e executar contêineres
Contêiner	Um contêiner é uma instância em tempo de execução de uma imagem Docker. Ele encapsula uma aplicação juntamente com todas as suas

	dependências, incluindo bibliotecas, ferramentas e binários necessários para executar a aplicação de forma isolada do ambiente de hospedagem. Os contêineres são leves, portáteis e executam em qualquer ambiente que suporte Docker.
Redes	As redes Docker fornecem conectividade entre contêineres em um ambiente Docker, permitindo que eles se comuniquem entre si e com outros recursos de rede, como sistemas externos e serviços. As redes Docker podem ser configuradas para oferecer diferentes níveis de isolamento e segurança, permitindo que os contêineres se comuniquem através de diferentes protocolos de rede, como TCP, UDP e HTTP.
Volumes	Os volumes são mecanismos de armazenamento persistente que permitem que os dados sejam compartilhados entre contêineres e persistam além do ciclo de vida de um contêiner. Eles são usados para armazenar dados sensíveis, como arquivos de configuração, bancos de dados e arquivos de log, garantindo que eles não sejam perdidos quando um contêiner é removido ou reiniciado.
Plugins	Os plugins Docker são extensões opcionais que adicionam funcionalidades adicionais ao Docker, como armazenamento de terceiros, redes personalizadas, autenticação e autorização avançadas, e integração com sistemas de monitoramento e orquestração. Os plugins permitem que os usuários personalizem e estendam as capacidades do Docker de acordo com suas necessidades específicas.
Docker Compose	Uma ferramenta para definir e gerenciar aplicativos multi-contêiner em um único arquivo YAML, simplificando o processo de configuração e orquestração de ambientes de desenvolvimento e produção.
Docker Swarm	Uma ferramenta de orquestração embutida no Docker Engine que permite implantar, escalar e gerenciar aplicativos distribuídos em um cluster de hosts Docker.
Docker Hub	Um registro de imagens Docker públicas e privadas que permite que os desenvolvedores compartilhem, distribuam e colaborem em imagens Docker pré-construídas.
Dockerfile	É um documento de texto que contém todos os comandos que um usuário pode chamar na linha de comando para montar uma imagem.

Para operacionalizarmos o Docker, podemos instalar o Docker Desktop, uma aplicação para Windows, Mac e Linux, que cria um ambiente para construir e compartilhar aplicações em contêineres e microsserviços. O Docker Desktop incluiu o Docker *daemon*, o Docker Client, Docker Compose, Kubernetes (assunto da nossa próxima aula), entre outros.



(FCC/TRT 22/2022) O Docker tornou muito mais fácil para os desenvolvedores entender e usar a tecnologia de contêineres. Para isso, oferece diversos recursos ou ferramentas, dentre as quais encontram-se:

I. Ferramenta de clustering e scheduling para contêineres do Docker, que permite que os administradores e desenvolvedores de TI possam estabelecer e gerenciar um cluster de nós do Docker como um único sistema virtual.
II. Reúne instruções necessárias para construir uma imagem de contêiner.
III. Aplicativo para plataforma Mac ou Windows que permite criar e compartilhar microsserviços e containerized applications. Inclui diversas ferramentas como o cliente Docker, o Docker Compose, o Docker Content Trust, o Kubernetes e o Credential Helper.

Os itens I, II e III correspondem, correta e respectivamente, a

a) Docherd – Docker Compose – Docker Hub.
b) Docker Engine – Docker Compose – Docker Desktop.
c) Docker Build – Docherd – Docker Swam.
d) Docker Engine – Dockerfile – Docker Hub.
e) Docker Swarm – Dockerfile – Docker Desktop.

Comentários:

Vamos analisar cada um dos itens.

I. A ferramenta responsável por fazer o clustering e scheduling, atividades que fazem parte da orquestração de contêineres, é o Docker Swarm.
II. As instruções são reunidas no Dockerfile.
III. O aplicativo que congrega todos esses objetos do docker é o Docker Desktop.

Portanto, correta a letra E. *(Gabarito: Letra E)*

Dockerfile

O *Dockerfile* é um arquivo de texto simples, que carrega uma série de instruções para que o Docker Engine automatize o processo de criação de imagens Docker (contêineres). Nela definimos tudo – mantenedor, a imagem em si, entre outros.

Esse arquivo contém diversas instruções ao Docker. Vamos vê-las:

- FROM: Define a imagem base a ser usada para construir a nova imagem. Todas as instruções subsequentes no Dockerfile são baseadas nesta imagem base. É o primeiro comando obrigatório do dockerfile.
- ARG: Define variáveis de argumento que podem ser passadas para o Docker durante o build. Essas variáveis podem ser usadas no Dockerfile para definir opções de configuração ou personalizar o processo de construção. É um comando opcional que pode aparecer antes do FROM ou depois do FROM.
- RUN: Executa um comando durante o processo de construção da imagem. Este comando pode ser usado para instalar dependências, configurar o ambiente de execução, ou executar outras tarefas de construção necessárias para a imagem.
- ADD: Este comando copia novos arquivos, diretórios ou arquivos remotos do host e os adiciona ao sistema de arquivos do contêiner em uma determinada localização. Ele também pode ser usado para extrair arquivos de um URL remoto e adicioná-los ao contêiner.
- COPY: Este comando copia arquivos ou diretórios do host para o sistema de arquivos do contêiner em uma determinada localização. Ele é semelhante ao comando ADD, mas não suporta a extração de arquivos remotos ou a adição de URLs.
- ENTRYPOINT: Define o comando ou script a ser executado quando o contêiner for iniciado. O comando ou script especificado é sempre executado e pode ser complementado com argumentos passados ao iniciar o contêiner.
- CMD: Define o comando padrão a ser executado quando o contêiner for iniciado. Este comando pode ser substituído por um comando específico do usuário ao iniciar o contêiner.
- LABEL: Define metadados para a imagem Docker, como informações de versão, descrição do aplicativo, mantenedor, etc. Esses metadados podem ser usados para documentar e identificar a imagem.
- EXPOSE: Informa ao Docker que o contêiner escuta em determinadas portas em tempo de execução. Isso não abre automaticamente as portas do contêiner para o host, mas é útil para documentar as portas que um aplicativo dentro do contêiner está usando.
- ENV: Define variáveis de ambiente no contêiner. Essas variáveis de ambiente podem ser usadas dentro do contêiner para configurar o ambiente de execução de aplicativos ou scripts.
- ADD: Este comando copia novos arquivos, diretórios ou arquivos remotos do host e os adiciona ao sistema de arquivos do contêiner em uma determinada localização. Ele também pode ser usado para extrair arquivos de um URL remoto e adicioná-los ao contêiner.

02648216103 - HENRIQUE AMORIM BARRETO

- HEALTHCHECK: Define um comando para verificar a saúde do contêiner e determinar se ele está em um estado saudável. Isso é útil para sistemas de orquestração de contêineres que podem querer monitorar e reagir a falhas de contêineres.
- MAINTAINER: Define o mantenedor (ou autor) da imagem Docker. Esta instrução é considerada obsoleta e foi substituída pela instrução LABEL.
- VOLUME: Cria um ponto de montagem no contêiner e marca o diretório como um volume que pode ser compartilhado com outros contêineres ou o host.
- USER: Define o usuário (por nome ou ID) que será usado para executar os comandos do Dockerfile durante o processo de construção da imagem e ao iniciar o contêiner.
- WORKDIR: Define o diretório de trabalho padrão para os comandos RUN, CMD, ENTRYPOINT, COPY e ADD subsequentes no Dockerfile. Isso define o diretório de trabalho para a execução desses comandos.
- STOPSIGNAL: Define o sinal que será enviado ao contêiner para indicar que ele deve ser interrompido. Por padrão, o sinal SIGTERM é usado.
- ONBUILD: Define instruções a serem executadas quando esta imagem é usada como base para outra imagem. Isso permite que você defina ações personalizadas que serão executadas durante o build de imagens derivadas.

O Dockerfile completo fica com essa carinha aqui:

DOCKERFILE

```
# Selecionamos uma imagem padrão como base
FROM node:18-alpine
# Escolhemos o /app como diretório central
WORKDIR /app
# Copiamos uma série de aplicativos do pacote JSON
COPY package*.json ./
# Copiamos diretórios locais para a imagem, dentro do /app
COPY ./src ./src
COPY ./public ./public
# Passamos uma série de instruções para quando criarmos a imagem
RUN npm install \    && npm install -g serve \    && npm run build \
    && rm -fr node_modules
# Expomos a porta 3000
EXPOSE 3000
# Começamos o app com o comando serve
CMD [ "serve", "-s", "build" ]
```

Se tivermos uma pasta de um aplicativo sendo desenvolvido, podemos usar o comando `docker init`, que criará todas as dependências necessárias para rodarmos o Docker e criarmos nossa imagem. Ele passará um questionário, perguntando a linguagem do código, entre outros, para criar as dependências e arquivos necessários.

(FGV/SEFAZ AM/2022) A plataforma Docker pode criar imagens automaticamente, executando as instruções de um arquivo Dockerfile. A primeira instrução presente em um Dockerfile é denominada

a) ADD.
b) RUN.
c) FROM.
d) EXPOSE.
e) WORKDIR.

Comentários:

O FROM é a primeira instrução mandatória do dockerfile, em que definimos uma imagem padrão para base da criação.

Cabe ressaltar um comando opcional que **pode aparecer antes do FROM**, que é o ARG. Como o ARG não estava nas alternativas da questão, nos coube optar pela alternativa com o primeiro comando obrigatório.

Portanto, correta a letra C. Aliás, na lista que lhe passei, os comandos já estão em ordem. *(Gabarito: Letra C)*

02648216103 - HENRIQUE AMORIM BARRETO

Criando Imagens

Temos nosso Dockerfile, que irá orientar a criação da nossa imagem, agora, através de uma aplicação de linha de comando (CLI), dentro da pasta onde temos os arquivos que queremos criar a imagem, usamos o seguinte comando:

```
docker build [OPTIONS] PATH | URL | -
```

Os principais parâmetros envolvidos na criação são:

- **OPTIONS:** Esses são os diferentes parâmetros que você pode passar para personalizar o processo de construção da imagem. Alguns dos parâmetros comuns incluem:
  - `-t, --tag:` Este parâmetro é usado para especificar um nome e uma tag para a imagem que está sendo construída. Por exemplo, -t myimage:latest.
  - `-f, --file:` Este parâmetro é usado para especificar o nome do Dockerfile a ser usado. Por padrão, o Docker procura por um arquivo chamado Dockerfile no diretório especificado, mas você pode usar este parâmetro para especificar um arquivo diferente. Por exemplo, -f Dockerfile.prod.
  - `--build-arg:` Este parâmetro é usado para definir argumentos de compilação que podem ser usados durante o processo de construção. Por exemplo, `--build-arg VERSION=1.0`.
  - `--no-cache:` Este parâmetro instrui o Docker a não usar o cache durante o processo de construção, o que significa que todas as camadas serão reconstruídas a partir do zero.
  - Outros parâmetros incluem `--pull` para forçar a atualização da imagem base durante a construção, `--squash` para comprimir as camadas em uma única camada, entre outros.
- **PATH | URL | . | - :** Este é o caminho para o contexto de construção, que pode ser um caminho local no sistema de arquivos (PATH), uma URL para um repositório Git (URL), `.` para ler o diretório local ou `-` para o diretório padrão.
  - Se você estiver construindo a partir de um diretório local, geralmente você apenas especifica o caminho para o diretório onde seu Dockerfile está localizado.
  - Se você estiver construindo a partir de um repositório Git, você especifica a URL do repositório Git.
  - Se você usar `.`, o contexto de construção será lido a partir da entrada padrão. Isso pode ser útil para a construção a partir de um Dockerfile gerado dinamicamente ou a partir de um pipe.
  - Se a opção for o diretório padrão do *Docker*, usa-se o `-`.

Como estamos rodando a CLI dentro da própria pasta que queremos, usaremos o comando:

02648216103 - HENRIQUE AMORIM BARRETO

```shell
docker build -t welcome-to-docker .
```

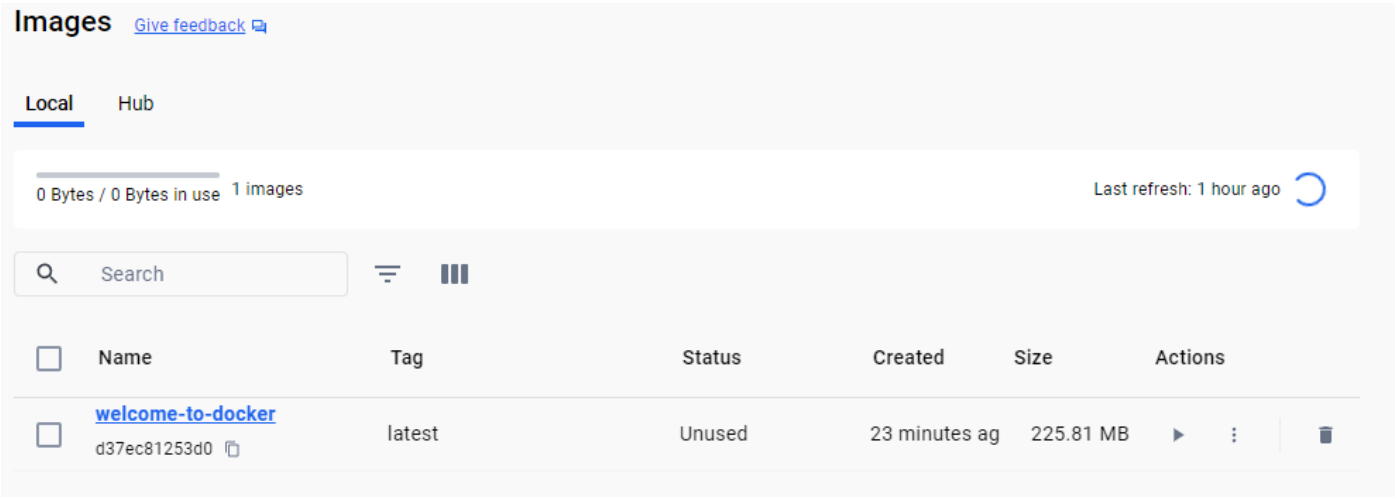
No comando temos:

- `docker build` → cria a imagem
- `-t` → procura a *tag* para criação da imagem específica, no caso welcome-to-docker
- `.` → define que o conteúdo está no diretório local

Ao rodarmos isso, teremos como retorno, no CLI, as seguintes informações:

```
PS C:\Users\Avell\Documents\welcome-to-docker> docker build -t welcome-to-docker .
[+] Building 200.6s (12/12) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                             0.2s
 => => transferring dockerfile: 669B                                                                                             0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                                                3.3s
 => [auth] library/node:pull token for registry-1.docker.io                                                                      0.0s
 => [internal] load .dockerignore                                                                                                0.1s
 => => transferring context: 52B                                                                                                 0.0s
 => [internal] load build context                                                                                                0.3s
 => => transferring context: 708.73kB                                                                                            0.2s
 => [1/6] FROM docker.io/library/node:18-alpine@sha256:0085670310d2879621f96a4216c893f92e2ded827e9e6ef8437672e1bd72f437         12.1s
 => => resolve docker.io/library/node:18-alpine@sha256:0085670310d2879621f96a4216c893f92e2ded827e9e6ef8437672e1bd72f437          0.0s
 => => sha256:0085670310d2879621f96a4216c893f92e2ded827e9e6ef8437672e1bd72f437 1.43kB / 1.43kB                                   0.0s
 => => sha256:aacbcec05180c1dd8c33dba8a9c42b75dbfdd659aa57617497f1ce2c5d83d889 1.16kB / 1.16kB                                   0.0s
 => => sha256:c8eb770fbfacf54104162cc9035c478ddb7d8dc15dca5298af028257f1dbdb3f 7.14kB / 7.14kB                                   0.0s
 => => sha256:eb6c7c29ba4d368f2428cacd291f7821b750fac3b1fb65b937ef855c573cdf97 40.24MB / 40.24MB                                 6.3s
 => => sha256:3d4a65156edf0208c8421995310d9e662e7ee63e2bcae660efb02f6c4ddef6a9 2.34MB / 2.34MB                                   1.3s
 => => sha256:4abcf20661432fb2d719aaf90656f55c287f8ca915dc1c92ec14ff61e67fbaf8 3.41MB / 3.41MB                                   2.0s
 => => sha256:5bdb6c27eb32087b71a9dde411c1f1eeb87563c0445f89db4eb7639d2cf50f45 450B / 450B                                       1.6s
 => => extracting sha256:4abcf20661432fb2d719aaf90656f55c287f8ca915dc1c92ec14ff61e67fbaf8                                        0.7s
 => => extracting sha256:eb6c7c29ba4d368f2428cacd291f7821b750fac3b1fb65b937ef855c573cdf97                                        4.9s
 => => extracting sha256:3d4a65156edf0208c8421995310d9e662e7ee63e2bcae660efb02f6c4ddef6a9                                        0.2s
 => => extracting sha256:5bdb6c27eb32087b71a9dde411c1f1eeb87563c0445f89db4eb7639d2cf50f45                                        0.0s
 => [2/6] WORKDIR /app                                                                                                           0.6s
 => [3/6] COPY package*.json ./                                                                                                  0.1s
 => [4/6] COPY ./src ./src                                                                                                       0.1s
 => [5/6] COPY ./public ./public                                                                                                 0.1s
 => [6/6] RUN npm install     && npm install -g serve     && npm run build     && rm -fr node_modules                           177.0s
 => exporting to image                                                                                                           6.9s
 => => exporting layers                                                                                                          6.8s
 => => writing image sha256:d37ec81253d0256f5b152b5d1cb124b6b383dc42c1a53c86d9b5ae3370271f16                                     0.0s
 => => naming to docker.io/library/welcome-to-docker                                                                             0.0s
```

E, se formos ao *Docker Desktop*, ao irmos em imagens, teremos a seguinte definição:



E pronto, nossa imagem está criada!



HORA DE
PRATICAR!

(CEBRASPE/DPE RO/2022) Um arquivo do tipo Dockerfile contém as configurações de um contêiner; para que seja possível, no final, executar esse contêiner, deve ser usado o comando

a) docker os.
b) docker exec.
c) docker build.
d) docker volume.
e) docker image.

Comentários:

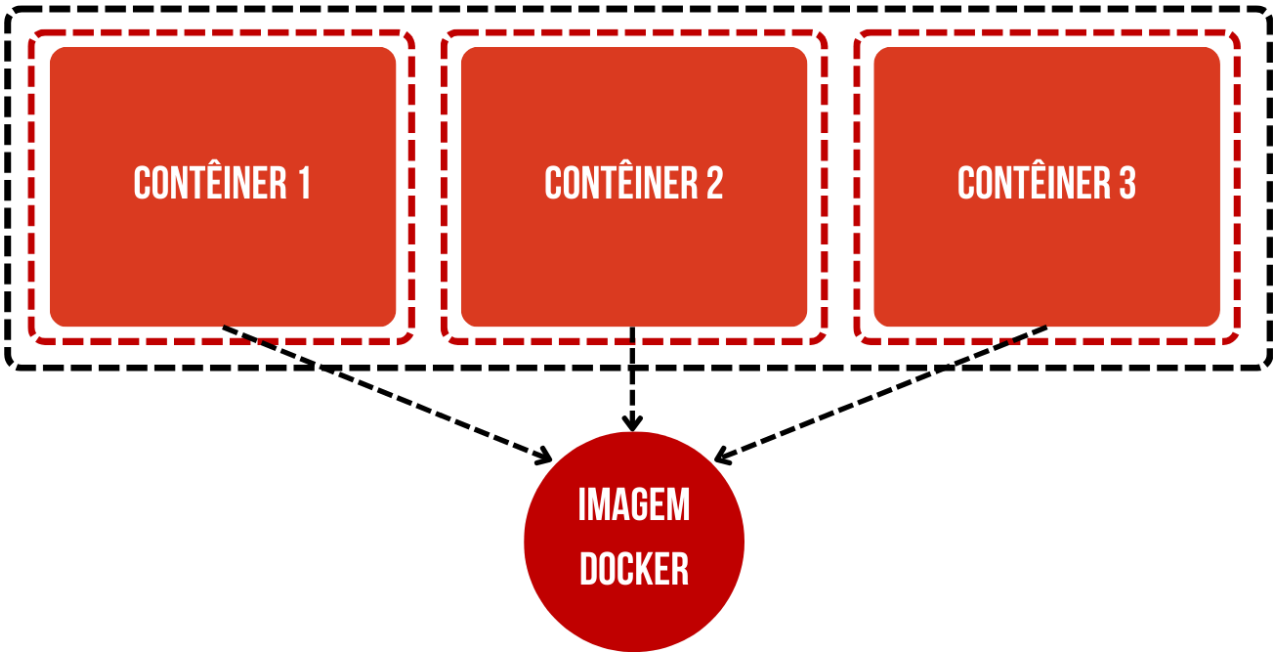
Um pouco de imprecisão – não é o contêiner em si que está sendo executado, e sim a imagem. Mas, com essa licença poética, o comando responsável por executar (criar) a imagem de fato é o docker build.

Quanto aos outros comandos:

- docker os – esse comando não existe. O que existe é docker ps, que lista os contêineres em execução no sistema
- docker exec – executa um comando dentro de um contêiner em execução
- docker volume – gerencia os volumes Docker, mecanismos usados para persistir os dados gerados pelo contêiner
- docker imagem – usado para gerenciar as imagens Docker no sistema

Sendo assim, correta a letra C. *(Gabarito: Letra C)*

Um cuidado que temos que ter é a diferenciação entre imagem e contêiner – para não incorrer no mesmo erro que a Cebraspe. Os arquivos que contêm todas as informações e valores para criar um contêiner são as imagens. Os contêineres são *deploys* de uma imagem – ou seja, o contêiner só existe enquanto estiver rodando, de forma efêmera.



Criando Contêineres

Agora, com nossa imagem em mãos, podemos criar o nosso contêiner. Isso pode ser feito através do Docker Desktop ou, como as bancas costumam cobrar, por comando em CLI. Veja a sintaxe genérica:

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Alguns dos parâmetros que podem ser passados na criação são:

- **`OPTIONS:`** Esses são os diferentes parâmetros que você pode passar para personalizar o comportamento do contêiner. Alguns dos parâmetros comuns incluem:
  - **`-d, --detach:`** Este parâmetro inicia o contêiner em segundo plano (modo "detached"), o que significa que ele continuará em execução mesmo que você saia do terminal. Isso é útil para serviços em execução contínua.
  - **`-p, --publish:`** Este parâmetro mapeia as portas do contêiner para portas do host. Por exemplo, -p 8080:80 mapeia a porta 80 do contêiner para a porta 8080 do host.
  - **`--name:`** Este parâmetro permite dar um nome específico ao contêiner em vez de usar um nome gerado automaticamente. Por exemplo, --name meu_contêiner.
  - **`--volume, -v:`** Este parâmetro monta um volume do host dentro do contêiner, permitindo compartilhar dados entre o host e o contêiner. Por exemplo, -v /caminho/local:/caminho/contêiner.
  - **`--env, -e:`** Este parâmetro define variáveis de ambiente dentro do contêiner. Por exemplo, -e VARIAVEL=valor.
  - **`--restart:`** Este parâmetro define a política de reinicialização do contêiner em caso de falha. Temos 4 políticas:
    - No (padrão) – não reinicializa automaticamente

    - Always – o Docker sempre tentará reiniciar o contêiner, independentemente do motivo de encerramento
    - Unless-stopped – semelhante ao "always", porém não haverá reiniciação quando a parada for explicitamente praticada pelo usuário (comando ´docker stop´)
    - On-failure – somente reiniciará o contêiner se o código de status for diferente de zero, apontando algum erro. Podemos, ainda, especificar um limite de tentativas de reinicialização com ´--restart-failure´
  - **`--expose:`** Este parâmetro informa ao Docker que o contêiner expõe a(s) porta(s) especificada(s) durante o tempo de execução. No entanto, isso não publica as portas para fora do contêiner. Ele é mais uma indicação para quem estiver usando o contêiner, especificando quais portas ele espera que estejam disponíveis para serem conectadas a outros contêineres ou serviços. Por exemplo, --expose 8080 indica que o contêiner expõe a porta 8080, mas a porta ainda deve ser mapeada usando -p para que seja acessível externamente.
- **`IMAGE:`** Este é o nome da imagem que você deseja usar para criar o contêiner. Pode ser uma imagem do Docker Hub, um repositório privado ou uma imagem local no seu sistema.
- **`COMMAND (opcional):`** Este é o comando a ser executado dentro do contêiner. Se não for especificado, o comando padrão definido na imagem será executado. Por exemplo, /bin/bash para iniciar um shell dentro do contêiner.
- **`ARG... (opcional):`** Estes são os argumentos passados para o comando dentro do contêiner. Por exemplo, se o comando for echo, você pode passar argumentos como `docker run minha_imagem echo "Olá, Mundo!"`.



CUIDADO! Não confunda o comando ´docker run´, para iniciar o contêiner, com a diretiva ´run´ no Dockerfile, que passa comandos a serem executados ao formar a imagem. São dois objetivos diferentes!

Continuando no nosso exemplo, vamos dar *deploy* na imagem de contêiner que criamos. Basta digitar no CLI o código abaixo:

```
SHELL
docker run -d -p 8088:3000 --name welcome-to-docker welcome-to-docker
```

Onde:

- **`docker run`** → comando para criarmos e iniciarmos um contêiner a partir de uma imagem
- **`-d`** → indica que estamos rodando o contêiner no modo *detached*, ou seja, em segundo plano
- **`-p`** → indica que estamos mapeando a porta 3000 do contêiner para a porta local 8088. Assim, quando qualquer tráfego for direcionado à porta 8080 do host, ele será redirecionado para a porta 3000 do contêiner
- **`--name welcome-to-docker`** → dá um nome específico ao contêiner
- **`welcome-to-docker`** → indica a imagem que está sendo usada para criar o contêiner



(FCC/TRT 18/2023) Um técnico deseja usar o Keycloak no Docker, instalado e em condições ideais.

```
___I_____8080:8080 -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin
quay.io/keycloak/keycloak:20.0.0 start-dev
```

Para iniciar o Keycloak exposto na porta local 8080, criando um usuário inicial admin, com senha admin, a lacuna I deve ser preenchida por:

a) docker run -v
b) docker container start on port
c) docker run -p
d) docker start --port
e) docker container run -d port

Comentários:

Queremos criar um contêiner, a partir de uma imagem, que exponha a porta local 8080 para a porta 8080 do contêiner. Para isso, precisaremos substituir a parte inicial pelo comando de deploy

de contêiner, com a opção de publish para a pota 8080 – da seguinte forma: *docker run -p*. *(Gabarito: Letra C)*

(FCC/MPE PB/2023) Considere um ambiente em que o Docker esteja instalado e funcionando em condições ideais. Uma Analista digitou os seguintes comandos:

```
$ docker container create -i -t --name mppbcontainer alpine
$ docker container start --attach -i mppbcontainer
```

O comando que seria equivalente aos dois comandos acima é:

a) $ docker run -rm --name mppbcontainer/alpine
b) $ docker run -it --name mppbcontainer alpine
c) $ docker container create && start --name mppbcontainer alpine
d) $ docker exec -i -t --name mppbcontainer//alpine
e) $ docker exec container --name mppbcontainer alpine

Comentários:

Questão difícil! Vamos lá, precisamos primeiro entender o que envolve os dois primeiros comandos.

```
$ docker container create -i -t --name mppbcontainer alpine
```

`docker container create` → o comando "*container create*" é uma forma de criarmos um contêiner sem iniciá-lo (ao contrário do *run*, que cria e roda automaticamente). Não se preocupe com os comandos, teremos um apanhado para vermos todos os comandos relevantes para prova no final da aula.
`-i` → indica que o contêiner será iniciado no modo interativo. Com isso, o terminal do contêiner será mantido aberto após a criação, permitindo a interação com ele.
`-t` → aloca um pseudo-terminal TTY para o contêiner, comando necessário para manter o terminal interativo
`--name mppbcontainer` → da um nome específico ao contêiner
`alpine` → indica a imagem que será usada para criar o contêiner

```
$ docker container start --attach -i mppbcontainer
```

`docker container start` → inicia um contêiner previamente criado
`--atach` → indica que o terminal do contêiner deve ser anexado ao terminal atual do host. Isso aponta que todas as interações de CLI do contêiner serão redirecionadas ao terminal do sistema operacional host.

`-i` → inicia o contêiner no modo interativo
`mppbcontainer` → o nome do contêiner que desejamos iniciar

Podemos congregar esses dois códigos um para criar o contêiner e outro para rodá-lo, com o comando `docker run`. Precisamos passar como parâmetro o terminal interativo, o que pode ser feito de forma separada (-i -t) ou agregada (-it). Por fim, indicamos o nome `mppbcontainer` e a imagem (alpine) que queremos. O comando fica assim:

```
$docker run -it --name mppbcontainer alpine
```

Correta, portanto, a letra B. *(Gabarito: Letra B)*

Docker Compose

Bom, o que vimos até agora foi o processo para operacionalizar um único contêiner. Porém, imagine se quisermos operacionalizar um grande conjunto de contêineres... como faremos?

Para isso, usamos o Docker Compose. O Compose é uma ferramenta fornecida pelo Docker que permite definir e gerenciar aplicativos multi-contêiner de forma mais fácil e simplificada. Ele utiliza arquivos YAML para configurar os serviços do seu aplicativo, permitindo que você defina as dependências, volumes, redes e outras configurações necessárias para cada contêiner em um único arquivo.



Um arquivo YAML (YAML Ain't Markup Language) é um formato de serialização de dados legível por humanos que é frequentemente usado para configuração e definição de dados estruturados. É comumente usado em ambientes de desenvolvimento de software, especialmente em contextos onde a legibilidade e a simplicidade são prioridades.

Um arquivo YAML consiste em pares de chave-valor e listas, com uma estrutura de dados hierárquica representada por indentação. Isso significa que a estrutura do arquivo é definida pela indentação dos elementos, tornando-o fácil de ler e entender. YAML é frequentemente usado para configurações de aplicativos, arquivos de manifesto, dados de configuração de infraestrutura e muito mais.

Vamos usar, novamente, o exemplo trazido pelo próprio Docker. Nele, e na maioria dos arquivos multi-contêiner, teremos um arquivo chamado compose.yaml. Nele, teremos todas as orientações para criarmos o aplicativo. Esse exemplo do Docker envolve a criação de dois contêiners para um aplicativo – o *todo-app* e o *todo-database*. Vamos destrinchar o arquivo YAML e entender o que é cada diretiva presente no documento.

YAML

```yaml
services:
  todo-app:
    build:
      context: ./app
    depends_on:
      - todo-database
```

```yaml
    environment:
      NODE_ENV: production
    ports:
      - 3000:3000
      - 35729:35729
    develop:
      watch:
        - path: ./app/package.json
          action: rebuild
        - path: ./app
          target: /usr/src/app
          action: sync
  todo-database:
    image: mongo:6
    ports:
      - 27017:27017
```

Nesse arquivo, temos:

- **services:** Isso indica que estamos definindo serviços para o Docker Compose. Cada serviço representa um contêiner Docker.
- **todo-app:** Este é o nome do serviço que estamos definindo. Ele representa o aplicativo de tarefas pendentes.
  - **build:** Esta seção define como construir a imagem Docker para este serviço. O contexto de construção é o diretório ./app, onde está localizado o código-fonte do aplicativo.
  - **depends_on:** Esta seção especifica que o serviço todo-app depende do serviço todo-database, ou seja, o banco de dados deve ser iniciado antes do aplicativo.
  - **environment:** Aqui são definidas variáveis de ambiente para o serviço. Neste caso, define-se a variável NODE_ENV como production.
  - **ports:** Esta seção mapeia as portas do contêiner para as portas do host. Por exemplo, 3000:3000 mapeia a porta 3000 do contêiner para a porta 3000 do host.
  - **develop:** Esta seção contém configurações específicas para o desenvolvimento. Ela define um monitoramento (`watch`) para o código-fonte do aplicativo, onde qualquer alteração nos arquivos package.json ou no diretório ./app resultará em uma

reconstrução do contêiner (`rebuild`) ou na sincronização (`sync`) do código-fonte com o contêiner.

- **`todo-database:`** Este é o nome do serviço que representa o banco de dados associado ao aplicativo de tarefas pendentes.
  - **`image:`** Esta seção especifica a imagem Docker a ser usada para este serviço. Neste caso, usa-se a imagem mongo:6 do MongoDB versão 6.
  - **`ports:`** Assim como no serviço todo-app, esta seção mapeia as portas do contêiner para as portas do host. Por exemplo, 27017:27017 mapeia a porta 27017 do contêiner para a porta 27017 do host.

Podemos ter uma penca de outras diretivas num documento. Vou listar apenas as principais, se não vamos ficar aqui a aula toda falando sobre elas. Ah, preciso ressaltar que a leitura delas é **extremamente** importante, já que tal assunto já foi alvo de cobrança em provas discursivas, como no cargo de TI para a SEF-MG – no caso, aplicado ao Kubernetes, se não me engano.

- **`container_name:`** Define um nome específico para o contêiner. Isso substitui o nome padrão gerado automaticamente.
- **`volumes:`** Define volumes a serem montados dentro do contêiner, permitindo persistência de dados entre execuções do contêiner.
- **`environment:`** Define variáveis de ambiente para o contêiner.
- **`restart:`** Define a política de reinicialização do contêiner em caso de falha.
- **`networks:`** Define redes personalizadas para conectar os serviços.
- **`command:`** Especifica o comando a ser executado quando o contêiner é iniciado.
- **`healthcheck:`** Define um comando para verificar a saúde do contêiner e determinar se ele está em um estado saudável.
- **`labels:`** Adiciona metadados ao contêiner, que podem ser usados para identificação ou organização.
- **`cap_add/cap_drop:`** Adiciona ou remove permissões de capacidade do contêiner.
- **`ulimits:`** Define limites de recursos do sistema para o contêiner, como limites de memória ou limites de CPU.
- **`network_mode:`** Define o modo de rede para o contêiner. Pode ser de três tipos:
  - **`bridge:`** modo padrão, onde é criada uma rede interna (*bridge network*), permitindo que os contêineres se comuniquem entre si
  - **`host:`** nesse modo o contêiner compartilha o namespace do host, ou seja, ele usa a pilha de rede do host diretamente, em vez de ter a sua isolada
  - **`none:`** o contêiner não possui uma configuração de rede, atuando isoladamente, sem se conectar com outros contêineres do *compose*

- `external_links:` Permite conectar o contêiner a contêineres externos, especificados pelo nome ou ID do contêiner e opcionalmente pelo *alias* de rede.
- `logging:` Configura as opções de registro para o contêiner, como o driver de log e suas opções.
- `tmpfs:` Monta uma partição de memória temporária no contêiner.

Para rodarmos um conjunto de contêineres com o *compose*, usamos o comando:

```
docker compose
```

Acompanhamos esse comando com alguns parâmetros:

- `up:` usado para criar e iniciar os serviços definidos no arquivo compose.yaml. Ele constrói as imagens necessárias (se ainda não estiverem construídas) e inicia os contêineres associados a esses serviços.
- `down:` usado para parar e remover todos os contêineres, redes e volumes associados aos serviços definidos no arquivo compose.yml. É basicamente o oposto do comando up, desligando todo o ambiente.
- `build:` constrói as imagens dos serviços definidos no arquivo compose.yml, sem iniciar os contêineres. Isso é útil quando você precisa apenas construir as imagens sem iniciar todo o ambiente.
- `start:` usado para iniciar os serviços definidos no arquivo compose.yml, sem reconstruir as imagens. Ele inicia os contêineres associados a esses serviços.
- `stop:` usado para parar os serviços definidos no arquivo docker-compose.yml, mas mantém os contêineres em execução. Isso é útil quando você precisa interromper temporariamente a execução dos serviços, sem removê-los.
- `restart:` usado para reiniciar os serviços definidos no arquivo docker-compose.yml, reiniciando os contêineres associados a esses serviços.
- `logs:` usado para exibir os logs de saída dos serviços definidos no arquivo docker-compose.yml. Ele mostra a saída padrão de cada contêiner em tempo real.
- `ps:` usado para listar os contêineres associados aos serviços definidos no arquivo docker-compose.yml, mostrando o estado atual de cada um.

Por fim, se quisermos criar o nosso cluster de contêineres com base no yaml, digitaremos:

```
docker compose up
```

(Inédita/Prof. Felipe Mathias) Rafael, responsável pelo DevOps da empresa SisCOM, precisa iniciar uma aplicação que envolve um conjunto de aplicativos. Durante o desenvolvimento do arquivo YAML, Rafael está trabalhando na diretiva “depends_on”. Assinale a alternativa que representa o que é definido nessa diretiva.

a) Define o modo de rede para o contêiner, permitindo a comunicação com outros contêineres.
b) Especifica as variáveis de ambiente a serem configuradas para o contêiner.
c) Controla a ordem de inicialização dos serviços, garantindo que determinados serviços sejam iniciados antes de outros.
d) Define volumes a serem montados dentro do contêiner, permitindo a persistência de dados.
e) Configura as opções de registro para o contêiner, como o driver de log e suas opções.

Comentários:

A diretiva "depends_on" cria dependências, especificando uma ordem de inicialização dos aplicativos. Por exemplo, se A depende de B, B deve ser inicializado primeiro.

Nese sentido, está correto o apontamento da letra C. *(Gabarito: Letra C)*

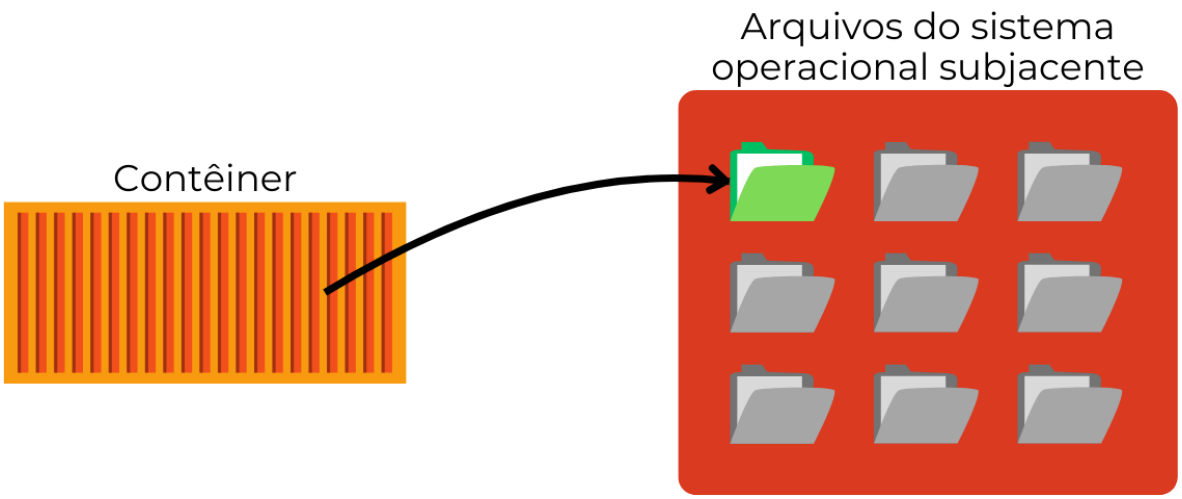


Persistência de Dados

O Docker isola todo o conteúdo do contêiner, incluindo os dados. Porém, podemos ter casos em que queremos persistir os dados do contêiner para o sistema operacional, para tornar essa interação definitiva. Por exemplo, uma criação de usuário, ou interações específicas de um usuário (já que podemos ter um deploy de 1 contêiner a cada usuário).

Com esse objetivo, usamos os Volumes. Um *volume* é uma local dentro do sistema de arquivos do sistema operacional subjacente, que é gerenciado pelo Docker. Dessa forma, podemos persistir os dados do contêiner mesmo após ele ter sido encerrado e/ou deletado.

Para criar e gerenciar os Volumes, usamos o comando `docker volume`.

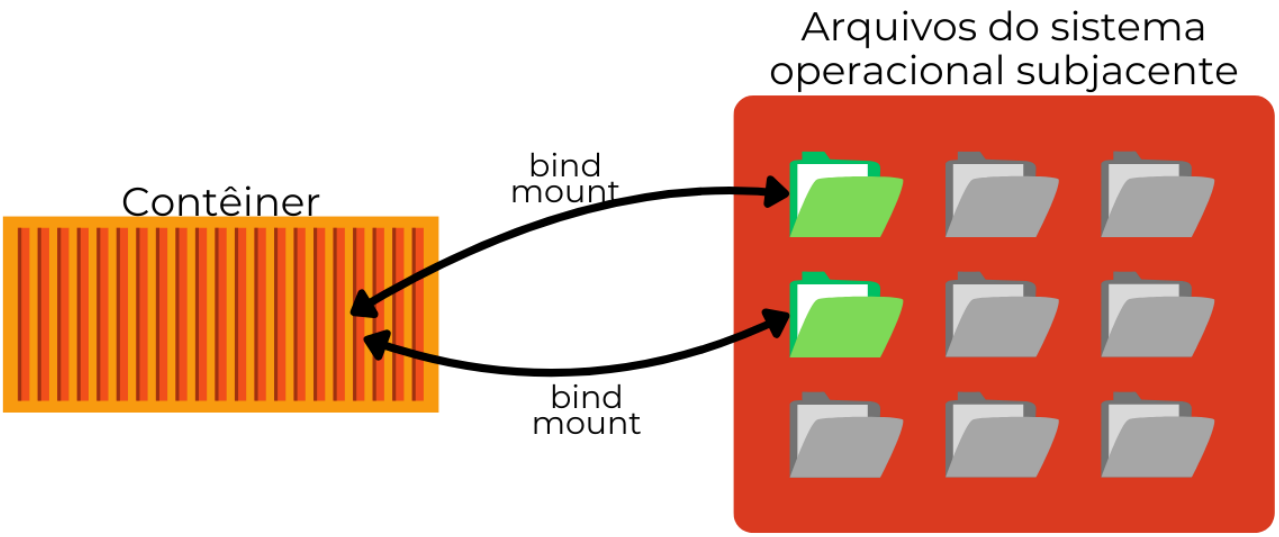


A criação de volumes também pode se dar no arquivo compose.yaml, através de diretivas. É bem simples, basta abrirmos a diretiva *volumes* e apontarmos onde a pasta local, e, em seguida, a pasta do contêiner (local:contêiner)

YAML

```yaml
services:
  todo-database:
    image: mongo:6
    volumes:
      - database:/data/db
    ports:
      - 27017:27017
```

Para que possamos montar esse diretório de arquivo do sistema host dentro de futuros contêineres, usamos o mecanismo de Bind Mount. Com isso, temos uma conexão direta entre o *host* e os contêineres. O ponto negativo é que isso acaba quebrando um pouco o isolamento do contêiner.



Esse *mount bind*, novamente, é feito no *compose.yaml*, dentro da diretiva de volumes.

YAML

```yaml
services:
  todo-app:
    ...
    volumes:
      - ./app:/usr/src/app
      - /usr/src/app/node_modules
```

O *volumes* indica ao compose para montar a pasta local ./app em /usr/src/app no contêiner para o serviço todo-app. Este bind mount em particular sobrescreve o conteúdo estático do diretório /usr/src/app no contêiner e cria o que é conhecido como um *contêiner de desenvolvimento*. A segunda instrução, /usr/src/app/node_modules, impede que o bind mount sobrescreva o diretório node_modules do contêiner para preservar os pacotes instalados no contêiner.

Comandos CLI

O intuito nesse capítulo é reunir todos os comandos que podem ser usados no Docker. Você pode fazer isso caso tenha o Docker instalado – basta abrir qualquer ferramenta de CLI e digitar `docker --help`. Vou destacar aqueles que mais importantes, e que acabam sendo os que mais caem em provas.

Comando	Descrição
`docker attach`	Acessa o contêiner e trabalha a partir dele
`docker build`	Cria uma imagem a partir de instruções de um Dockerfile
`docker commit`	Cria uma imagem a partir de um contêiner
`docker cp`	Copia arquivos ou diretórios do contêiner para o host
`docker create`	Cria um novo contêiner, sem iniciá-lo
`docker diff`	Exibe alterações feitas no filesystem do contêiner
`docker events`	Exibe os eventos do contêiner em tempo real
`docker exec`	Executa uma instrução dentro do contêiner, sem precisar o attach
`docker export`	Exporta um container para um arquivo .tar
`docker history`	Exibe o histórico de comandos que foram executados
`docker images`	Lista as imagens disponíveis no host
`docker import`	Importa uma imagem .tar para o host
`docker info`	Exibe informações do host
`docker inspect`	Exibe o *json* com todas as configurações do contêiner
`docker kill`	Interrompe abruptamente um contêiner em execução
`docker load`	Carrega a imagem de um arquivo .tar
`docker login`	Registra ou faz login em um servidor de registro
`docker logout`	Faz logout do servidor de registro
`docker logs`	Exibe os logs de um contêiner
`docker -p`	Abre uma porta do host e do contêiner
`docker network`	Gerencia as redes do Docker
`docker node`	Gerencia as redes do Docker Swarm

docker pause	Pausa o contêiner
docker port	Lista as portas mapeadas
docker ps	Lista todos os contêineres ativos
docker pull	Faz o pull de uma imagem Docker
docker push	Faz o push de uma imagem Docker
docker rename	Renomeia um contêiner existente
docker restart	Reinicia um contêiner que está rodando ou parado
docker rm	Remove um ou mais contêineres
docker rmi	Remove uma ou mais imagens
docker run	Cria um contêiner a partir de uma imagem, e o inicia
docker save	Salva a imagem em um arquivo .tar
docker search	Procura por uma imagem dentro do Hub
docker service	Gerenciamento dos serviços do Docker
docker start	Inicia um contêiner que esteja parado
docker stop	Para um contêiner que esteja rodando
docker swarm	Plataforma de orquestração de vários contêineres
docker tag	Coloca tag em uma imagem
docker unpause	Inicia um contêiner que estava pausado
docker update	Atualiza a configuração dos contêineres
docker version	Exibe as versões de API, Client e Server do host
docker volume	Gerencia os Volumes do Docker
docker wait	Aguarda o retorno da execução de um contêiner para iniciar outro determinado contêiner

(VUNESP/CM FERNANDÓPOLIS/2022) Um dos recursos utilizados para o gerenciamento de contêineres Docker são os comandos inseridos por meio da interface de linha de comando (Docker CLI).

O comando utilizado para listar os contêineres que estão sendo executados e informações relacionadas a estes é o:

a) docker ps
b) docker rm
c) docker node
d) docker inspect
e) docker search

Comentários:

Para verificarmos os contêineres que estão sendo executados, usamos o comando *docker ps*. *(Gabarito: Letra A)*

(COMPERVE/TJ RN/2020) Os volumes são mecanismos utilizados para persistir os dados gerados e usados pelos containers do Docker. Embora as montagens de ligação dependam da estrutura de diretórios da máquina host, os volumes são completamente gerenciados pelo Docker.

Considerando que um analista queira criar um volume de nome my-volume dentro de um docker, ele deve executar o comando

a) docker volume create my-volume
b) docker create volume my-volume
c) docker run create volume my-volume
d) docker create run volume my-volume

Comentários:

Para criarmos um volume dentro de determinado contêiner, usamos o comando *docker volume create <nome_do_volume>* – no caso da questão, como queremos o nome my-volume, o comando será *docker volume create my-volume*. *(Gabarito: Letra A)*

(FGV/TRT 16/2022) Docker é uma plataforma que permite criar e compartilhar aplicativos e microsserviços em contêineres.

O comando utilizado para executar um contêiner novo é o

a) docker composes.
b) docker create.
c) docker build.
d) docker exec.
e) docker run.

Comentários:

Temos dois comandos que podem executar um contêiner novo:

- docker run – cria um contêiner a partir de uma imagem, e o inicializa
- docker start – inicia um contêiner existente

Nesse sentido, o único dos comandos que aparece na questão é o da letra E, que é nosso gabarito.
(Gabarito: Letra E)

Docker Swarm

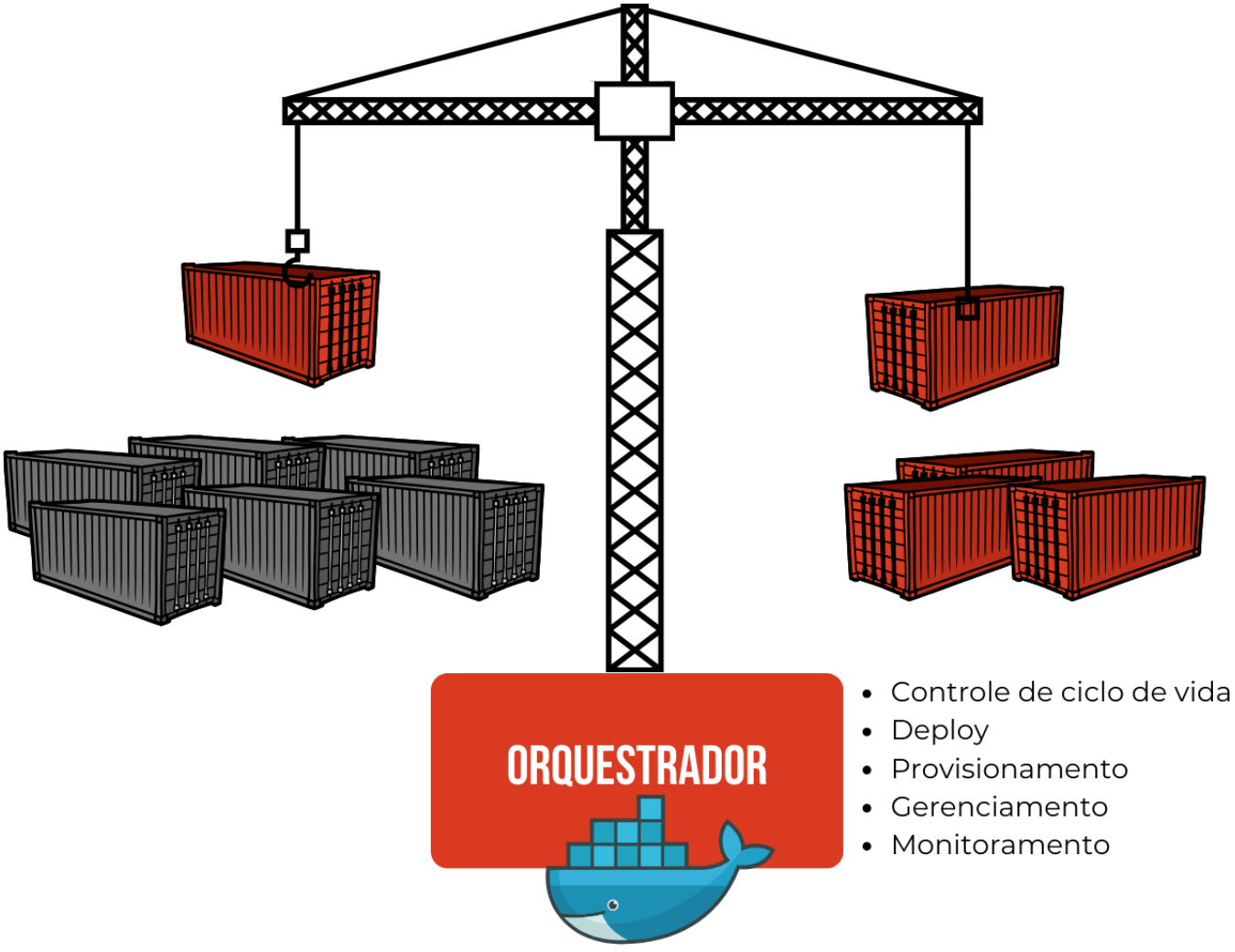
Orquestração de Contêineres

A orquestração de contêineres é uma prática de gerenciamento de contêineres, que envolve controlar, coordenar o *deployment*, a escalabilidade, a disponibilidade e, principalmente, automatizar todo esse processo.

Pense, por exemplo, que você é auditor fiscal e precisa acessar uma área privativa do serviço do Estado, para promover alguma operação. Ao fazer *login*, uma das abordagens recomendadas é a criação de um contêiner (ou de vários, se essa atividade usa um conjunto de contêineres) para lidar com essa sessão.

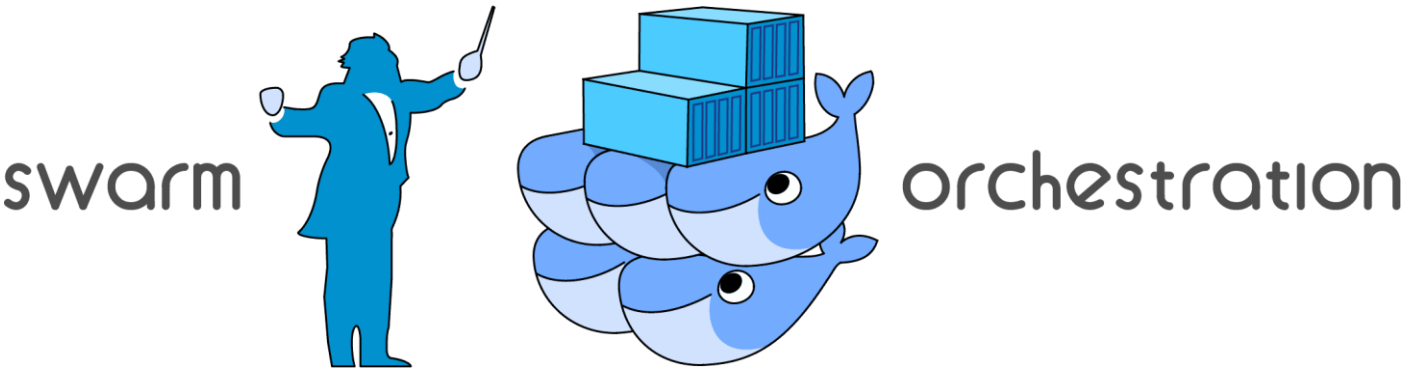
Pense, se fossemos fazer um *deploy* para cada acesso, a criação, a extinção, o controle do ciclo de vida do contêiner, quantos funcionários a empresa não teria que ter só para lidar com isso? E o tempo que isso demoraria?

Nesse contexto, surge o Docker Swarm.

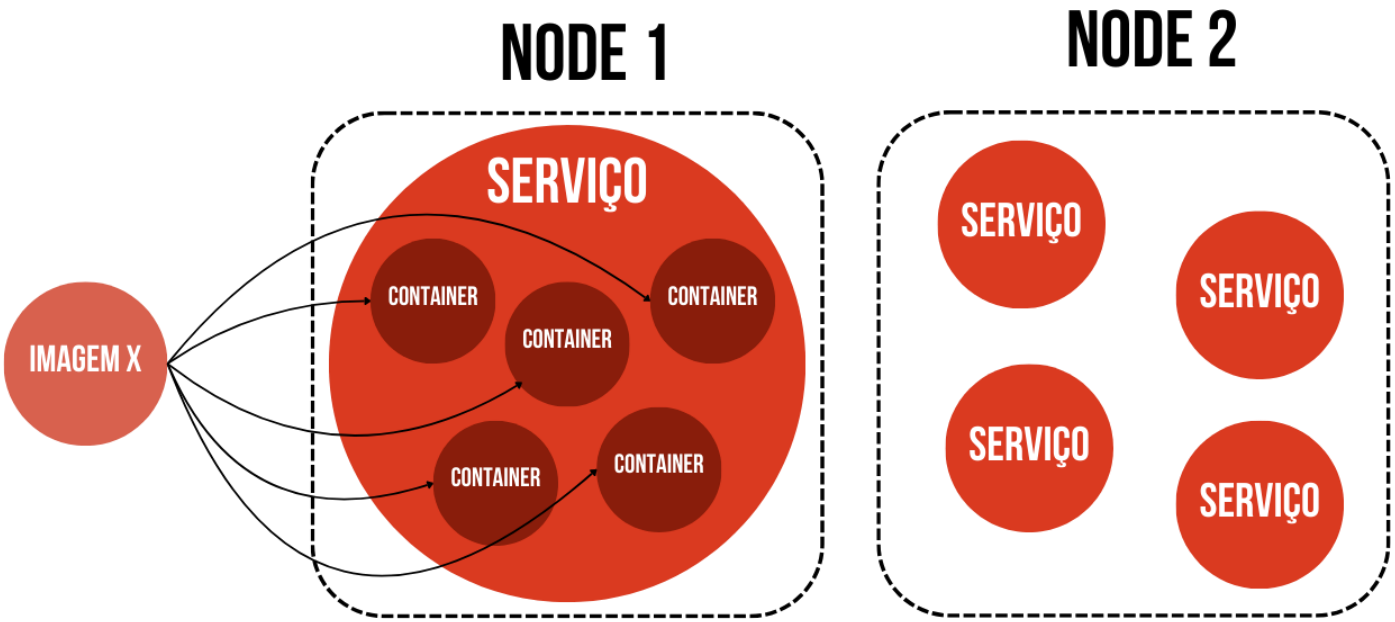


Para "saber" o que ele precisa fazer, os orquestradores baseiam-se em políticas definidas pelo usuário e em eventos. Dessa forma, usualmente em um arquivo YAML ou JSON, o usuário define quando e o que o orquestrador deve fazer, em resposta a determinados eventos. Por exemplo, podemos ter o número de usuários acessando determinado serviço configurado e, a partir do ponto máximo, monitorado através de solicitações HTTP, uma replicação (cópia) do contêiner atual é feita.

O Docker Swarm é uma ferramenta de orquestração de Contêineres nativa do Docker, que faz toda a parte de gerenciamento do ciclo de vida de contêineres. A partir de uma rede sobreposta (a *docker network*), o Swarm faz a descoberta de serviços e o balanceamento de carga.



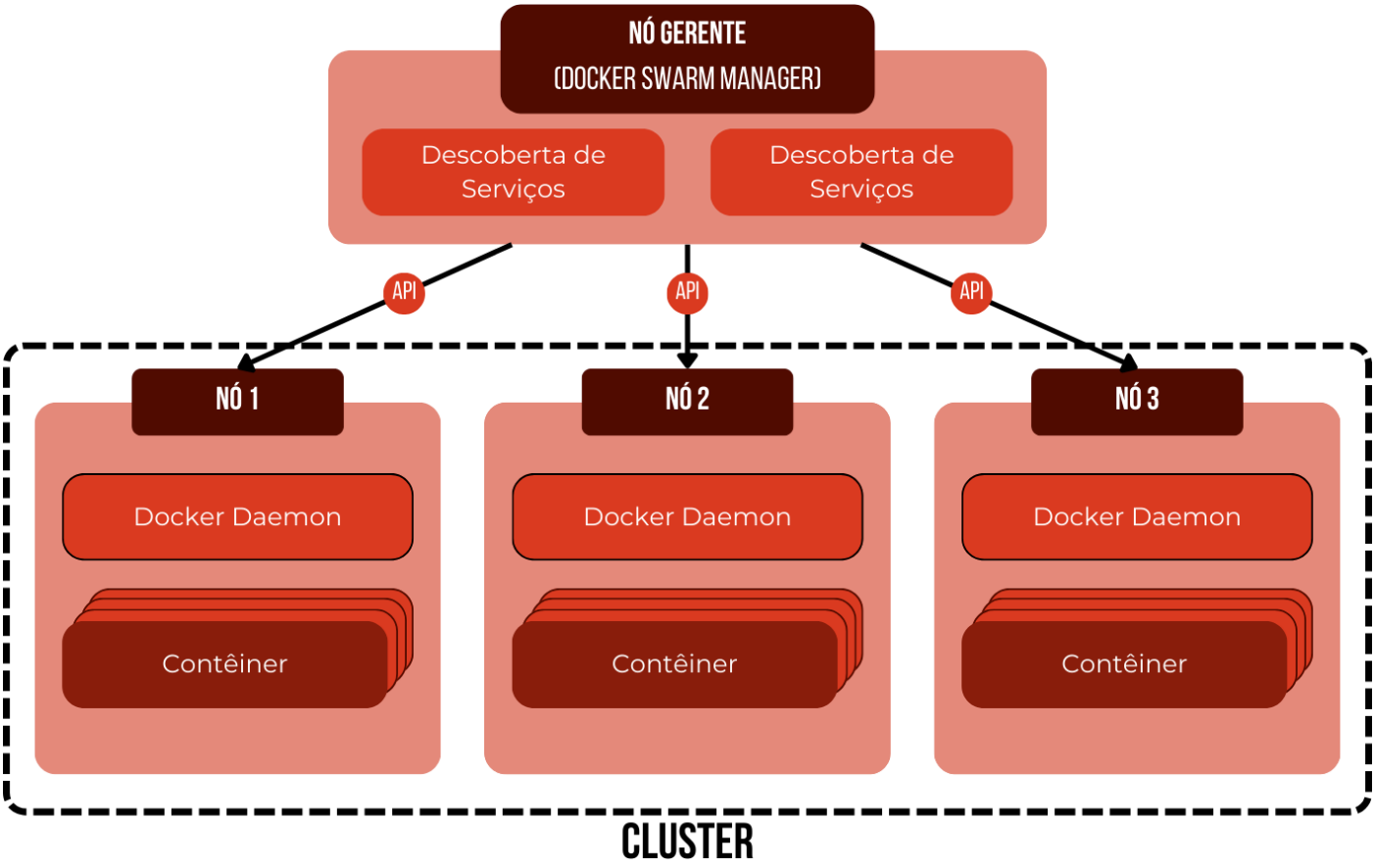
O Swarm gira em torno do conceito de serviços. Os serviços são constituídos a partir de múltiplos contêineres de uma mesma imagem, que são implementados dentro de um nó (*node*) – sendo que, em cada nó, podemos ter um ou mais serviços.



Temos dois tipos de nós no Docker:

- Nó gerente (manager node): responsável por gerenciar e monitorar os nós, de forma a criá-los, replicá-los e enviar tarefas a eles
- Nó trabalhador (worker node): recebe a tarefa do gerente e a executa

Essa comunicação entre gerente-trabalhador é feita a partir do protocolo HTTP, por meio de uma API específica do Docker – a Docker API. Cada nó terá seu próprio Docker *Daemon*, que receberá essa tarefa e a alocará para o container respectivo. Para saber para qual serviço a solicitação deve ser mandada, usamos a Descoberta de Serviços (ou *"Service Discovery"*), que roteia o tráfego de trabalho para o serviço específico que deve lidar com ele.



Podemos operar o Swarm em dois modos:

- Global: nesse modo, o Swarm manterá os contêineres em todos os nós escravos e master (ou nó trabalhador e gerente). Isso mantem replicas dos contêineres em todos os nós disponíveis do cluster
- Replicado: aqui o Swarm irá implementar os contêineres no número de replicas definidas pelo usuário.

Para inicializarmos o cluster do Docker Swarm, usamos o comando:

```
docker swarm init
```

Ao usarmos o comando, se não existir um cluster ativo, iremos iniciar o cluster com o nó atual (a sua máquina) como o nó gerente. Além disso, são gerados dois valores:

- Token de junção (join token): valor usado para permitir que outros nós se juntem ao cluster. Esse token é específico para o papel que o nó desempenhará (ou seja, gerente ou trabalhador) e possui um prazo de validade
- Swarm ID: um identificador único para o cluster Swarm criado

Se quisermos adicionarmos mais nós ao nosso cluster existente, usamos o comando abaixo em cada nó desejado:

```
docker swarm join --token <token>
```

Podemos também bloquear determinado nó de receber réplicas, definindo o modo de operação para "Drain Node", ou nó dreno.

> Réplicas referem-se à quantidade de instâncias de um serviço que devem ser mantidas em execução no cluster todo, ou em um nó específico.

Criando Serviços

Serviços, como já falei para vocês, é uma coletânea de contêineres criados a partir de uma mesma imagem Docker. Usualmente, cada contêiner será responsável por um serviço da aplicação (veremos mais sobre na aula de microsserviços), fazendo jus ao nome. Cada serviço recebe um conjunto de tarefas, que é alocada dentro dos contêineres do serviço.
Para criarmos um serviço específico, usamos o comando abaixo:

```
docker service create [OPÇÕES] imagem_docker
```

Dentro de opções, podemos passar os seguintes parâmetros:

- `--replicas` → especifica o número de réplicas que devem ser mantidas em execução
- `--name` → atribui um nome ao serviço
- `--constraint` → define restrições de onde os contêineres do serviço pode ser executados, com base na *tag* do nó. Por exemplo: `--constraint node.role=worker`
- `--publish` → mapeia portas de contêineres no host
- `--mount` → monta volumes em contêineres, por exemplo: `--mount type=volume, source=meu-volume, target=/app`
- `--env` → define variáveis de ambiente para o serviço

- `--restart-condition` → especifica a condição de reinicialização do contêiner. Temos 4 possibilidades:
- `no:` não haverá reinício automático
- `on-failure:` contêiner será reiniciado automaticamente apenas se falhar
- `always:` contêiner sempre será reiniciado
- `unless-stoped:` contêiner sempre será reiniciado, exceto se tiver sido parado explicitamente pelo usuário
- `--mode` → define o modo (global ou replicado) que o serviço irá operar
- `--limit-xx` → define limites para o consumo de algum recurso por parte do contêiner, como CPU (--limit-cpu) e memória (--limit-memory)
- `--detach` → Executa o serviço em segundo plano

Gerenciando Nós

Bom, temos nossos nós criados, definimos os serviços e estamos com o cluster em funcionamento. Agora, precisamos de ferramentas para monitorar as atividades dos nossos nós, e fazer um gerenciamento geral deles.

Para isso, usamos o comando:

```
docker node [OPÇÕES]
```

Dentre as opções, incluem-se:

- `ls` → lista todos os nós no cluster Docker Swarm. Ele fornece informações como o ID do nó, o hostname, o status (como ativo ou inativo), se o nó é um gerenciador ou um trabalhador, e a versão do Docker em execução no nó.
- `inspect <NODE_ID or NODE_NAME>` → fornece informações detalhadas sobre um nó específico no cluster. Ele pode ser usado para obter informações como endereço IP, labels, recursos disponíveis, e configurações específicas do nó.
- `update` → usado para atualizar a configuração de um nó no cluster. Você pode usá-lo para adicionar ou remover etiquetas, ajustar recursos (como CPUs ou memória), ou modificar outras configurações do nó.
- `promote <NODE_ID or NODE_NAME>` → usado para promover um nó de trabalhador a gerenciador. Isso pode ser útil se você deseja adicionar mais nós gerenciadores ao cluster para melhorar a redundância e a disponibilidade.
- `demote <NODE_ID or NODE_NAME>` → usado para rebaixar um nó de gerenciador a trabalhador. Você pode usar isso para redistribuir a carga de trabalho entre os nós gerenciadores e trabalhadores no cluster.
- `rm <NODE_ID or NODE_NAME>` → remove um nó do cluster. Isso pode ser útil se você deseja aposentar um nó antigo ou se deseja reduzir o tamanho do cluster.

02648216103 - HENRIQUE AMORIM BARRETO

Um cuidado que você deve ter é que, enquanto o comando `docker ps` lista os contêineres ativos, no Swarm não temos esse comando. Aqui, usamos apenas o `docker node ls` para listar todos os nós.

Configuração de Políticas – Compose

A **configuração das políticas** do Swarm é feita de forma muito semelhante (para não dizer idêntica) ao Docker Compose – inclusive usamos o mesmo arquivo, **compose.yaml**. Aqui, definimos alguns tópicos, como:

- Número de réplicas de serviço
- Restrições de implantação (por exemplo, onde um serviço deve ser implantado)
- Redes a serem utilizadas pelos serviços
- Configurações de volumes
- Configurações de saúde do serviço
- Configurações de atualização de serviço

Um exemplo de YAML é esse:

YAML

```yaml
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    deploy:
      replicas: 3
      placement:
        constraints:
          - node.labels.type == worker
    networks:
      - my_network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost"]
      interval: 30s
      timeout: 10s
      retries: 3

  db:
    image: mysql:latest
    environment:
      MYSQL_ROOT_PASSWORD: example
      MYSQL_DATABASE: my_database
    deploy:
      replicas: 1
    networks:
      - my_network
    volumes:
      - db_data:/var/lib/mysql
```

A instrução YAML acima cria dois serviços, um *web* e outro banco de dados. Vamos explorar as diretivas do YAML para entender tudo que podemos colocar aí. Lembrando que tudo o que veremos aqui é aplicável ao Docker Compose, e vice-versa.

Definindo o Objeto

Começamos o *compose* com a definição do objeto. Aqui, no nosso caso, começaremos definindo o **serviço** – usando a diretiva *services*. No nosso exemplo acima, criamos dois serviços: o *web* e o *db*.

Imagem Base

Definido o serviço, precisamos escolher a imagem base para ele. Temos três formas de fazer isso:

Definindo uma imagem pré-definida do Hub usando a tag *image*. Exemplo:

YAML
```yaml
services:
    alpine:
        image: alpine:latest
```

Definindo uma imagem local, com a tag *build*, definindo o local de acordo com a Dockerfile. Exemplo:

YAML
```yaml
services:
    app:
        build: ./app
```

Definimos o contexto e a nossa Dockerfile específica, dentro do *build*. Exemplo:

YAML
```yaml
services:
    app:
        build:
            context: ./dir
            dockerfile: Dockerfile.dev
```

Portas

A exposição de portas se dá de forma muito similar ao que vimos no Dockerfile. Podemos expor as portas de duas formas:

- Expor somente a porta do serviço no contêiner, mas não do host:

YAML
```yaml
expose:
 - "3000/tcp"
 - "8000/udp"
```

- Expor as portas do contêiner e do host

```yaml
expose:
 - "8080:80/udp" #lembrando, temos host:container/protocolo
```

Comandos

Os *commands* são ações executadas assim que o contêiner for iniciado, agindo como uma espécie de substituto para os comandos que você digitaria no CLI do contêiner. Podemos, por exemplo, rodar um *npm run* assim que o contêiner for montado.

```yaml
app:
  ...
  command:
    - 'npm run start'
```

Volumes

Podemos criar os volumes diretamente no YAML, através do *bind mount*. Nele,

```yaml
volumes:
  - /opt/data:/var/lib/mysql #caminho origem:caminho contêiner
  - datavolume:var/lib/mysql #aqui estamos atribuindo um nome
```

Dependências

Já vimos um pouco das dependências – que definem ordens na criação dos serviços. Dessa forma, se A depende de B, criaremos o serviço B antes para não termos problemas de referência e funcionamento.

```yaml
ghost: #nome do serviço
    container_name: ghost
    image: ghost
    depends_on: [db] #db será iniciado antes
```

Variáveis de Ambiente

Variáveis de ambientes são usadas para configurar os dados no contêiner. Esse parâmetro é usado se o contêiner é dependente do sistema operacional, de forma a atribuir nomes a variáveis.

```yaml
web:
  environment:
    - NODE_ENV=production
```

Também podemos anexar um arquivo *.env*. Esse é um arquivo que contém todas as nossas variáveis de ambiente.

```yaml
web:
  env_file:
    - variables.env
```

Rede

A rede (*network*) define as regras de comunicação entre contêineres, e entre os contêineres e o sistema *host*. As redes, em si, são objetos, portanto têm o mesmo nível de hierarquia que os serviços. Dessa forma, definimos uma rede dentro do YAML como um objeto, atribuindo-a um nome, e, em cada contêiner, apenas fazemos referência ao nome da rede – já que as configurações estarão no objeto *network*.

As redes são customizáveis, e podemos preencher, dentre outros valores, os seguintes:

- driver: define o tipo de rede que usaremos. Os tipos disponíveis são:
  - bridge: formato padrão, onde cada contêiner possui sua própria rede, e é conectado a uma rede interna privada do *host*, permitindo comunicação apenas entre contêineres num mesmo *daemon* host

  - host: remove o isolamento de rede entre o contêiner e o *host*, usando somente a rede do *host*. Essa forma é útil quando precisamos de uma performance otimizada, ou quando o contêiner lida com muitas portas.
  - overlay: permite a conexão entre vários *daemons* do Docker, permitindo que tenhamos comunicação entre diferentes nós. Dessa forma, podemos remover o roteamento a nível de sistema operacional.
  - ipvlan: cria interfaces de rede virtuais que compartilham endereços IP com o *host* – tanto endereços IPv4 quanto IPv6. Dessa forma, os contêineres são acessíveis diretamente pela rede do host, como se estivessem conectados a ela
  - macvlan: também cria interfaces de rede virtual, agora atribuindo um endereço MAC a cada contêiner, permitindo que cada um tenha seu endereço IP e MAC próprio
  - none: remove todas as interfaces de um contêiner, deixando-o isolado.
- driver_opts: define configurações adicionais para o driver. Cada driver tem suas opções, e, honestamente, as bancas não costumam ir tão a fundo.

Abaixo, um exemplo de configuração para uma rede *overlay*.

YAML

```yaml
services:
  app1:
    image: nginx:latest
    networks:
      - my_overlay_network

networks:
  my_overlay_network:
    driver: overlay
    driver_opts:
      encrypted: "true"
```



(FGV/TJ RN/2023) A analista Maria gerencia a aplicação WebJus. A WebJus requer a execução de dois containers Docker que necessitam de comunicação entre si. No entanto, para fins de balanceamento de carga, cada container da WebJus foi alocado em um servidor diferente. A fim

de configurar a comunicação entre os containers da WebJus de forma simples, Maria criou a rede Docker NetJus, que é capaz de conectar containers rodando em servidores diferentes.

A NetJus utiliza o driver de rede Docker:

a) host;
b) ipvlan;
c) bridge;
d) overlay;
e) macvlan.

Comentários:

Temos que cada contêiner está em um servidor diferente – portanto, cada contêiner WebJus se comunica com um *daemon* diferente. Para permitir comunicação entre *daemons* diferentes (e, consequentemente, entre servidores e nós), precisamos usar a configuração do tipo *overlay*. *(Gabarito: Letra D)*

Healthcheck

A diretiva *healthcheck* é usada para definir verificações de saúde dos contêineres. Dessa forma, periodicamente o Docker checa se está tudo funcionando perfeitamente em cada um dos contêiners, verificando a sua integridade. Cada tipo de contêiner aceita um *healtcheck* diferente. Vamos falar apenas dos parâmetros genéricos – que podem cair na sua prova.

- test: [plataforma, comando] → define o comando a ser executado para verificar a saúde
- interval → define o intervalo da aplicação do teste
- timeout → define o tempo limite para cada verificação
- retries → define quantas vezes o Docker tentará executar o comando, antes de considerá-lo como "não saudável".

Tomando como exemplo o arquivo que trouxe no começo do capítulo:

YAML

```yaml
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost"]
  interval: 30s
  timeout: 10s
  retries: 3
```

Assim, definimos o teste como o comando `curl -f http://localhost`, a ser executado no CMD, a cada 30s, com limite para 10s em cada teste. Se, após 3 tentativas, não tivermos sucesso, o contêiner será considerado como não saudável.

(PROPGE UFSCAR/2019) Considere o arquivo docker-compose.yml apresentado a seguir:

```
1   version: '2.1'
2   services:
3     A:
4       image: ubuntu:14.04
5       depends_on: [ B ]
6       command: 'nc -v B 80'
7     B:
8       image: nginx:stable-alpine
9       command: '/bin/sh -c "sleep 5; nginx -g daemon\ off\;"'
10      healthcheck:
11        test: ["CMD", "nc", "-z", "localhost", "80"]
12        interval: 1s
13        retries: 10
```

Executando-se o comando *docker-compose up* no diretório que contém esse arquivo, obtém-se a saída a seguir.

```
Creating network "questao_default" with the default driver
Creating questao_B_1 ... done
Creating questao_A_1 ... done
Attaching to questao_B_1, questao_A_1
A_1  | nc: connect to B port 80 (tcp) failed: Connection refused
questao_A_1 exited with code 1
```

Assinale a alternativa que explica, de forma correta, o motivo plausível para essa saída ter sido obtida.

a) A versão 14.04 do Ubuntu, usada como imagem para o serviço A, não vem com o utilitário nc.
b) A opção depends_on está com sintaxe incorreta. Ela deveria conter uma lista de itens, cada um iniciado por um hífen.
c) A opção depends_on não espera os serviços subirem e passarem no healthcheck, a menos que seja usada com a condição service_healthy.
d) A opção depends_on está especificada no serviço errado. Ela deveria ter sido especificada dentro do serviço B, indicando uma dependência para com o serviço A.
e) A opção depends_on é considerada legada. Em vez dela, deveria ter sido utilizada a opção links.

Comentários:

Temos um exemplo de um código com dependências – A depende de B, portanto B deve ser iniciado primeiramente. Só que temos um probleminha, no *Docker,* quando apontamos uma dependência, o *compose* só inicia o aplicativo antes, não espera ele estar pronto para ligar o próxima.

Por exemplo, um *compose* que envolva um banco de dados e uma aplicação, usualmente o banco de dados será inicializado antes, mas não estará "pronto" antes da aplicação, já que deve demorar mais para carregar todos os dados – por isso, usualmente, prevemos mecanismos de resiliência para dados indisponíveis.

Nesse sentido, ao prevermos um *health check* no serviço B, teremos, no início da execução, alguns problemas – já que A estará realizando pedidos a B sem ele estar completamente operacional. Nesses casos, é útil usar a condição *service_healthy*, para iniciarmos a verificação somente após ser constatado que os erviço está saudável. Nesse sentido, correto o apontamento da letra C. *(Gabarito: Letra C)*

02648216103 - HENRIQUE AMORIM BARRETO

QUESTÕES COMENTADAS

01. (FUNDEP/CRO MG/2023) O Dockerfile é um arquivo de configuração do Docker que permite a declaração de comandos para automatizar o processo de criação e a configuração de imagens e containers.

A esse respeito, numere a COLUNA II de acordo com a COLUNA I, relacionando os comandos disponíveis no Dockerfile com suas respectivas definições.

COLUNA I

1. FROM
2. EXPOSE
3. ENV

COLUNA II

( ) Declara imagem para criação do container.
( ) Declara uma variável de ambiente.
( ) Declara a porta que será aberta no container.

Assinale a sequência correta.

a) 1 2 3
b) 1 3 2
c) 3 1 2
d) 2 3 1

Comentários:

Ótima questão para revisarmos alguns parâmetros, seja do Dockerfile, seja do Compose.

( ) Declara imagem para criação do container.
Para declararmos uma imagem na *Dockerfile*, usamos o comando FROM.

( ) Declara uma variável de ambiente.
As variáveis da imagem são definidas na diretiva ENV.

( ) Declara a porta que será aberta no container.
A declaração das portas que serão expostas está em EXPOSE. Atenção, esse comando não abre as portas – apenas declara as portas que deverão ser abertas.

Nesse sentido, temos 1 3 2 como sequência.

Gabarito: Letra B

---

02. (FUNDATEC/CIGA SC/2023) O que é um container Docker?

a) Uma máquina virtual.
b) Uma imagem.
c) Uma instância executável de uma máquina virtual.
d) Uma instância executável de uma imagem.
e) Um pacote de código-fonte.

Comentários:

Um container, seja Docker, seja Linux, é uma instância virtual executável, criada a partir de um arquivo de imagem. Ela difere de uma virtualização (letra A) por utilizar recursos do sistema operacional subjacente para simular um sistema operacional interno às aplicações que estão sendo virtualizadas.

Gabarito: Letra D

---

03. (VUNESP/UNICAMP/2023) O uso do software Docker tem sido comum no sistema operacional Linux para a criação e execução de aplicações na forma de containers. Nesse contexto, uma imagem de container representa

a) uma janela de interface gráfica onde a saída produzida pela execução do container é apresentada.
b) um container em execução, que se reflete em um ou mais processos no sistema operacional que executa o container.
c) um arquivo de log gerado durante a execução do container, registrando informações importantes para avaliação do desempenho da aplicação.
d) um arquivo no formato JPG ou PNG para exibição da aplicação no navegador, quando esta for publicada em repositórios de containers na web.
e) um arquivo com todo o conteúdo necessário para a execução de um container, incluindo a própria aplicação e suas dependências.

Comentários:

Uma imagem é um arquivo que congrega tudo que é necessário para rodarmos um aplicativo, com o estado necessário que queremos – código da aplicação, dependências, dados, entre outros.

Essa imagem é a "estrutura" da aplicação e serve como fonte para a criação dos contêineres. Nesse sentido, a alternativa com a explicação correta é a letra E.

Gabarito: Letra E

---

04. (AOCP/IF MA/2023) Suponha que você seja um analista de infraestrutura em uma empresa de tecnologia e esteja trabalhando em um projeto que envolve o uso de Docker para executar aplicativos em contêineres. Sua equipe busca garantir a segurança dos contêineres, sem comprometer a escalabilidade e a portabilidade. Nesse cenário, qual é a maneira eficaz de melhorar a segurança ao executar aplicativos em contêineres com Docker?

a) Executar todos os contêineres com privilégios de root.
b) Utilizar imagens de base não oficiais para contêineres.
c) Implementar políticas de segurança usando Docker Secrets e Docker Content Trust.
d) Executar várias instâncias de aplicativos diferentes em um único contêiner.
e) Ignorar as práticas de isolamento de recursos do Docker.

Comentários:

Vamos analisar cada afirmativa, procurando a que aponta uma abordagem segura na execução de contêineres.

a) Errado. É justamente o contrário – devemos evitar usar privilégios de root.
b) Errado. Novamente, devemos evitar usar imagens não oficiais.
c) Certo. O Docker conta com duas estruturas importantes de segurança: o *Docker Content Trust*, que oferece um serviço de assinaturas digitais para garantir segurança com pacotes remotos, e o *Docker Secrets*, plataforma de gerenciamento de segredos (como senhas e *tokens*) do ecossistema Docker.
d) Errado. Cada contêiner deve executar uma única instância.
e) Errado. Devemos isolar, ao máximo, os recursos, de forma a aumentarmos a segurança da aplicação.

Sendo assim, correta a letra C.

Gabarito: Letra C

---

05. (CEBRASPE/EMPRE/2023) Na virtualização por contêineres, o mecanismo que engloba recursos do sistema em uma abstração é conhecido como

a) control groups.
b) union file systems.
c) bare metal.
d) namespace.
e) inter process communication.

Comentários:

Na virtualização tradicional, quem faz a abstração é o *hypervisor*. Porém, nos contêineres, não temos mais a figura do hipervisor. Quem "assume" a responsabilidade pela abstração do sistema subjacente são os *namespaces* (Linux) e os *silos* (Windows).

Gabarito: Letra D

---

06. (VUNESP/TJ RS/2023) Deseja-se executar um *container* Docker no qual um diretório específico dentro do *container* seja mapeado em um diretório específico da máquina *host*. Esse diretório do *host* deve ser especificado pelo usuário do Docker por meio de seu caminho absoluto no comando de execução do *container*. A solução para esse problema consiste em utilizar

a) uma imagem.
b) um volume.
c) um *bind mount*.
d) um *tmpfs mount*.
e) uma rede virtual.

Comentários:

Quando falamos de persistir dados, temos duas abordagens: os *Volumes* ou os *bind mount*.

Os Volumes são armazenados dentro de um diretório específico no host, ou em algum sistema de armazenamento remoto configurado para esse uso específico. É o caso dos bancos de dados – os dados e interações são armazenados apenas no *host*, e os contêineres fazem apenas referência a esses arquivos. Os volumes são gerenciados pelo Docker, garantindo persistência após o encerramento do contêiner.

Já o *Bind Mount* cria um mapeamento entre um diretório do host, e um diretório do contêiner. Dessa forma, conseguimos acessar e modificar arquivos no host a partir de um contêiner, e vice-versa. Aqui não temos gerenciamento por parte do Docker, e não há persistência após o encerramento da aplicação.

Podemos perceber a diferença no *compose* também. Nos *bind mounts*, temos uma atribuição entre host:contêiner. Já nos volumes, temos uma definição de um objeto, que compartilha o nível de hierarquia de um serviço, por exemplo.

Nesse sentido, para atender ao comando da questão, precisamos de um *bind mount*.

Gabarito: Letra C

---

07. (VUNESP/TJ RS/2023) O seguinte comando da ferramenta Docker serve para remover todos os volumes locais não usados.

```
docker volume prune –all
```

Nesse contexto, os volumes "não usados" são aqueles que

a) estão vazios.
b) não são referenciados por nenhum container.
c) não são usados há um determinado período de tempo, cujo valor default depende das configurações do Docker.
d) são anônimos.
e) não contêm nenhum arquivo de imagem.

Comentários:

O *prune* é um comando usado para remover volumes não usados – que, para o Docker, são aqueles que não são utilizados por nenhum contêiner. A verificação de uso/não uso é feita a partir da análise de referências: se um volume não é referenciado por nenhum contêiner, ele é considerado sem uso.

Gabarito: Letra B

---

08. (CEBRASPE/PREF. FORTALEZA/2023) Tendo em vista que, no atual cenário de desenvolvimento de aplicações web, é essencial considerar princípios, como consistência e escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue o item subsequente.

Um exemplo prático de conteinerização de aplicação é a utilização do Docker para criar um ambiente consistente; nesse caso, é correto criar um arquivo Dockerfile por meio do comando docker build.

Comentários:

Um erro sutil, mas grande: com o comando *docker build*, nós criamos uma imagem a partir de um Dockerfile, e não o Dockerfile em si.

Gabarito: Errado

---

09. (CEBRASPE/PREF. FORTALEZA/2023) Tendo em vista que, no atual cenário de desenvolvimento de aplicações web, é essencial considerar princípios, como consistência e

escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue o item subsequente.

A adoção de tecnologias de conteinerização, como o Docker, permite à equipe orquestrar vários contêineres em conjunto, por exemplo, um contêiner para a aplicação Node.js e outro, para o banco de dados MongoDB.

Comentários:

Perfeito! Essa é justamente um dos grandes motivos para os contêineres: separação de responsabilidades. Cada contêiner tem sua responsabilidade, sua função, seu "minimundo", funcionando de forma isolada e sem afetar as outras aplicações.

Gabarito: Certo

---

10. (CEBRASPE/MPE RO/2023) A seguir, é apresentado um arquivo Dockerfile para a criação de um contêiner.

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
MD ["node", "src/index.js"]
EXPOSE 3000
```

A partir das informações precedentes e considerando que se esteja no mesmo diretório onde se encontra o arquivo Dockerfile, assinale a opção que apresenta o comando a ser executado para criar uma imagem do contêiner com o nome node-js.

a) docker cp -t node-js .
b) docker build -t node-js .
c) docker push -t node-js .
d) docker image -t node-js .
e) docker pull -t node-js .

Comentários:

Questão suave! Temos um Dockerfile – a forma de construir um contêiner a partir de um Dockerfile é usando o comando *docker build*. A questão só colocou uma alternativa com essa função, então a vida ficou fácil.

O outro termo que aparece na sintaxe, o -t, indica a atribuição de uma *tag*.

Gabarito: Letra B

---

11. (CEBRASPE/MPE RO/2023) Assinale a opção correspondente ao comando, em Docker, que permite ao usuário fazer o download de uma imagem específica ou um conjunto de imagens do Docker Hub.

a) docker attach
b) docker node ls
c) docker pull
d) docker rmi
e) docker rm /redis

Comentários:

O comando responsável por "puxar" imagens do Hub é o *docker pull*. Sua contrapartida, para enviar, é o *docker push*. Quanto aos demais comandos:

- docker attach: anexa a entrada, saida e erro padrão a um contêiner em execução
- docker node ls: lista os serviços e contêineres do node
- docker rmi: remove uma imagem
- docker rm /redis: remove um contêiner redis

Portanto, correta a letra C.

Gabarito: Letra C

---

12. (CEBRASPE/DATAPREV/2023) No que se refere a conceitos de contêineres e orquestração de contêineres, julgue o item a seguir.

É possível visualizar o consumo de recursos pelo contêiner no Docker, por meio do comando `docker container ls`.

Comentários:

Errado! Para visualizar o consumo de recursos de um contêiner usamos o comando *docker status*.

Gabarito: Errado

---

13. (CEBRASPE/DPE RO/2023) Em programação, os contêineres são utilizados em larga escala no desenvolvimento, testes e, principalmente, na produção de softwares, permitindo rodar múltiplos sistemas isolados dentro de um sistema operacional real.

A esse respeito, julgue os itens a seguir.

> I. Em um contêiner, é possível utilizar diversos recursos e ferramentas, porém tudo roda em um mesmo sistema operacional; em caso de pane, todas as funcionalidades serão afetadas.
> II. Ferramentas embasadas nos contêineres Linux oferecem aos usuários acesso sem precedentes a aplicações, além da habilidade de implementar com rapidez e de ter total controle sobre as versões e distribuição.
> III. As ferramentas de contêiner, incluindo o Docker, fornecem um modelo de implantação com base em imagens, facilitando o compartilhamento de uma aplicação ou conjunto de serviços, incluindo todas as dependências deles em vários ambientes.

Assinale a opção correta.

a) Apenas o item I está certo.
b) Apenas o item II está certo.
c) Apenas os itens I e III estão certos.
d) Apenas os itens II e III estão certos.
e) Todos os itens estão certos.

Comentários:

Vamos analisar cada uma das afirmativas.

I. Errado. É uma afirmativa mal construída – se o problema for no sistema operacional subjacente sim, teremos um erro. Mas a afirmativa mais se aproxima do conceito de virtualização, já que contêiners são executados de forma isolada e um problema em um deles não afetaria os demais.
II. Certo. Contêineres, por empacotarem todos os requisitos para a aplicação rodar, de fato não precisam de nada precedente a eles.
III. Certo. Perfeito galera – o Docker cria imagens de aplicativos e seus requisitos, e, a partir das imagens, criamos os contêineres.

Corretas, portanto, a II e III.

Gabarito: Letra D

---

14. (CEBRASPE/DPE RO/2023) A orquestração automatiza a implantação, o gerenciamento, a escala e a rede dos contêineres. As ferramentas de orquestração de contêineres fornecem um framework para gerenciar arquiteturas de microsserviços e contêineres em escala, e muitas delas são usadas no gerenciamento do ciclo de vida dos contêineres; entre elas, o Docker Swarm é uma plataforma

02648216103 - HENRIQUE AMORIM BARRETO

a) que permite utilizar diversos recursos e ferramentas, como Apache e PHP, porém tudo rodando em um mesmo sistema operacional.
b) de código aberto criada pelo Google para operações de implantação de contêiner, aumento e redução e automação em clusters de hosts.
c) de orquestração de contêiner de código aberto, sendo o mecanismo de clusterização nativo para e pelo Docker, utilizando sua mesma linha de comando.
d) que roda sobre o Kubernetes instalado em sistema operacional na versão Enterprise da Red Hat, agregando opções de monitoramento, integração e entrega contínua.
e) usada pela Amazon para fornecer outros serviços aos clientes, como DNS, balanceamento, segurança e monitoramento, se integrando

Comentários:

O Docker Swarm é a plataforma de orquestração de contêineres da Docker, que opera em código aberto – sendo o mecanismo nativo para orquestração do Docker. Sendo assim, a alternativa que apresenta a definição correta é a letra C.

Gabarito: Letra C

---

15. (FGV/CGU/2022) Uma das estratégias para reduzir o tamanho de imagens Docker consiste em:

a) combinar comandos RUN em um único comando;
b) substituir a imagem base por uma versão mais recente;
c) separar comandos RUN complexos em comandos menores;
d) reordenar os comandos de forma que o cache seja utilizado com maior frequência;
e) compactar arquivos a serem copiados para a imagem e descompactá-los durante a sua geração.

Comentários:

Vamos analisar cada afirmativa, procurando aquela que irá apontar uma prática que permite reduzir o tamanho das imagens Docker.

a) Certo. A cada comando definido no RUN do Dockerfile, temos uma criação de uma camada de imagem – mais camadas, maior o tamanho. Se agregarmos vários comandos juntos, teremos uma redução do número de camadas, reduzindo o tamanho do arquivo.
b) Errado. Isso não garante um tamanho menor.
c) Errado. Vide letra A – devemos agregar, não separar.
d) Errado. A ordem dos comandos não afeta o tamanho. O que podemos fazer é desabilitar o *cache* para diminuir o tamanho.
e) Errado. Não é recomendada a compactação de arquivos numa imagem.

Portanto, correta a letra A.

Gabarito: Letra A

---

16. (FGV/SEFAZ AM/2022) Leia o fragmento a seguir.

> *"A plataforma Docker usa uma arquitetura do tipo _____. O cliente Docker conversa com o daemon do Docker, que constrói, executa e distribui _____ Docker. O cliente e o daemon do Docker podem ser executados em um mesmo sistema ou se conectar um cliente do Docker a um daemon remoto. O cliente Docker e o daemon se comunicam usando _____ ou uma interface de redes."*

Assinale a opção cujos itens completam corretamente as lacunas do fragmento acima, na ordem apresentada.

a) MVC – imagens – chamadas RPC ou *bluetooth*.
b) *thin client* – contêineres – wireless ou *bluetooth*.
c) *serverless* – componentes – chamadas MPI ou RPC.
d) cliente-servidor – contêineres – API REST ou soquetes UNIX.
e) *mesh app and service* – imagens – *API RESTFULL ou wireless*.

Comentários:

Vamos preencher cada campo.

- A arquitetura do Docker é do tipo cliente-servidor
- O Docker foca na construção e na distribuição de contêineres
- A comunicação é feita, primariamente, por APIs Rest. É possível, também, o uso de soquetes UNIX.

Portanto, temos como resposta a letra D.

Gabarito: Letra D

---

17. (FGV/SEFAZ AM/2022) Com relação à segurança dos contêineres Docker, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

> ( ) Possuem isolamento no nível do processo no sistema operacional e isolamento adicIonal, usando recursos especiais tais como *namespaces* e *cgroups*.

02648216103 - HENRIQUE AMORIM BARRETO

( ) Aproveitam-se dos mecanismos *interprocess communication* padrão, tais como sinais, pipes e sockets, onde cada contêiner possui a sua própria *network stack*.
( ) Os contêineres possuem um *multi level security* no sistema operacional do host e os recursos físicos que são gerenciados por um *hypervisor* compartilhado.

As afirmativas são, na ordem apresentada, respectivamente,

a) F – V – V.
b) F – F – V.
c) V – F – V.
d) V – F – F.
e) V – V – F.

Comentários:

Vamos analisar cada assertiva.

( ) Possuem isolamento no nível do processo no sistema operacional e isolamento adicIonal, usando recursos especiais tais como namespaces e cgroups.
Verdadeiro. De fato, os contêineres usam esses recursos para isolamento e abstração.

( ) Aproveitam-se dos mecanismos interprocess communication padrão, tais como sinais, pipes e sockets, onde cada contêiner possui a sua própria network stack.
Verdadeiro. Os mecanismos de comunicação são os mais variados – e, de fato, cada contêiner possui sua própria pilha de rede.

( ) Os contêineres possuem um multi level security no sistema operacional do host e os recursos físicos que são gerenciados por um hypervisor compartilhado.
Falso. Quem usa hypervisor são as virtualizações, não os contêineres.

Sendo assim, temos V-V-F, correta a letra E.

Gabarito: Letra E

---

18. (FCC/TJ CE/2022) No Linux, em condições ideais, o Docker é disponibilizado com três redes por padrão. Essas redes oferecem configurações específicas para o gerenciamento do tráfego de dados e

a) deve-se, para visualizar as redes no Docker, utilizar o comando: docker network -show
b) todos os containers que estão na rede None poderão se comunicar via protocolo TCP/IP. Se uma pessoa souber qual é o endereço IP do container que deseja conectar, é possível enviar tráfego para ele, pois estão todos na mesma rede IP (172.17.0.0/24).

c) a rede Host tem como objetivo isolar o container para comunicações externas. A rede não recebe qualquer interface para comunicação externa. A única interface de rede IP será a localhost. Essa rede, normalmente, é utilizada para containers que manipulam apenas arquivos de backup, sem necessidade de enviá-los via rede para outro local.
d) a rede None tem como objetivo entregar para o container todas as interfaces existentes no docker none. Isso agiliza a entrega dos pacotes, uma vez que não há host no caminho das mensagens. Mas o uso de um host pode ser importante para a segurança e a gerência do tráfego.
e) Bridge é a rede padrão para qualquer container iniciado no Docker, a menos que seja, explicitamente, associada outra rede a ele. Os containers da rede default bridge podem acessar uns aos outros somente através de seus endereços IP, a menos que se utilize a opção --link

Comentários:

Vamos lá, encontrar a alternativa correta.

a) Errado. Para visualizar as redes, usamos *docker ls*.
b) Errado. Em drivers de rede *none*, temos uma ausência de redes – o que gera um isolamento do contêiner.
c) Errado. Essa definição é a definição do driver *None*, não do *Host*.
d) Errado. Como não temos redes, não há de se falar em entrega de pacotes numa rede *None*.
e) Certo. As redes *Bridge* são as redes padrões do Docker, e, de fato, sem a opção –link, só conseguimos acessar os contêineres de uma mesma rede (mesmo *daemon*).

Portanto, correta a letra E.

Gabarito: Letra E

---

19. (FCC/TRT 19/2022) A fim de utilizar o Docker em sua organização, um Analista necessitou conhecer os principais componentes dessa plataforma, tais como:

I. Software que roda na máquina onde o Docker está instalado. Recebe comandos do cliente a partir de Command Line Interfaces ou API's REST.
II. Mecanismo usado para criar imagens e containers.
III. Coleção de imagens hospedadas e rotuladas que juntas permitem a criação do sistema de arquivos de um container. Pode ser público ou privado.
IV. Repositório usado para hospedar e baixar diversas imagens. Pode ser visto como uma plataforma de Software as a Service (SaaS) de compartilhamento e gerenciamento de imagens.

Os itens de I a IV correspondem, correta e respectivamente, a Docker

a) Conteiner – Compose – Swarm – Image.
b) Daemon – Image – Container – Registry.
c) Engine – Swarm – Hub – Registry.
d) Daemon – Engine – Registry – Hub.
e) Conteiner – Engine – Compose – Hub.

Comentários:

Vamos analisar cada item.

I. O Software que recebe os comandos a partir de CLI ou APIs é o *daemon* do Docker.
II. Para criarmos imagens e contêineres, usamos o Engine.
III. Aqui temos a definição de Registry – um aplicativo server-side, stateless, que permite distribuir imagens Docker
IV. O repositório para hospedarmos imagens Docker é o Hub.

Então, temos como resposta a letra D – Daemon, Engine, Registry e Hub.

Gabarito: Letra D

---

20. (FAPEC/UFMS/2022) Sobre os conceitos de virtualização e conteinerização, analise as assertivas a seguir.

I- Na virtualização, cria-se uma nova camada de computação, na qual uma máquina virtual permite a virtualização de toda a infraestrutura de TI (rede, servidores e storage) através de um sistema operacional próprio (hipervisor).
II - Conteinerização (contêiner) é um processo de computação que utiliza uma pequena fração do recurso de hardware para viabilizar a execução de aplicativos de forma mais eficiente e isolada. No caso do contêiner, permite a virtualização de aplicativos de software, utilizando o sistema operacional do host em um Kernel compartilhado.
III -As máquinas virtuais são medidas em megabyte. Elas contêm, no máximo, a aplicação e os arquivos necessários para executá-la. Além disso, elas costumam ser utilizadas para empacotar funções individuais que realizam tarefas específicas, os famosos microsserviços. Como são leves e possuem um sistema operacional compartilhado, as VMs são muito fáceis de migrar entre vários ambientes.

IV -Os contêineres são medidos em gigabyte. Eles costumam ter seu próprio sistema operacional, o que possibilita a execução simultânea de várias funções com uso intenso de recursos. Por terem um número maior de recursos à disposição, contêineres são capazes de abstrair, dividir, duplicar e emular por inteiro servidores, sistemas operacionais, desktops, bancos de dados e redes.

Estão corretas, apenas, as assertivas:

a) I e II.
b) II e III.
c) III e IV.
d) I e III.
e) II e IV.

Comentários:

Ótima questão da banca FAPEC. Antes de ir para ela, vamos lembrar da nossa comparação contêiner x virtualização.

Comparação	Contêiner	Virtualização
Isolamento	Usa *namespaces* ou *silos*	Usa hipervisores
Overhead	Menor	Maior
Velocidade de Inicialização	Maior	Menor
Tamanho	Pequeno	Grande
Portabilidade	Alta	Média
Flexibilidade	Menor	Maior
Implementação	Simples	Complexa
Persistente	Não	Sim

Com isso em mente, vamos analisar cada um dos itens.

I. Certo. Perfeita a definição de virtualização.
II. Certo. É exatamente esse o funcionamento dos contêineres.
III. Errado. As máquinas virtuais usualmente são pesadas – na ordem dos GigaBytes. Isso pois carregam toda a estrutura necessária para criar uma máquina, como sistema operacional, disco rígido virtual, entre outros.
IV. Errado. Os contêineres são extremamente leves, nunca chegando na ordem de GigaBytes. Isso pois não trazem uma abstração completa, como as máquinas virtuais.

Sendo assim, corretos os itens I e II.

Gabarito: Letra A

LISTA DE QUESTÕES

01. (FUNDEP/CRO MG/2023) O Dockerfile é um arquivo de configuração do Docker que permite a declaração de comandos para automatizar o processo de criação e a configuração de imagens e containers.

A esse respeito, numere a COLUNA II de acordo com a COLUNA I, relacionando os comandos disponíveis no Dockerfile com suas respectivas definições.

COLUNA I

1. FROM
2. EXPOSE
3. ENV

COLUNA II

( ) Declara imagem para criação do container.
( ) Declara uma variável de ambiente.
( ) Declara a porta que será aberta no container.

Assinale a sequência correta.

a) 1 2 3
b) 1 3 2
c) 3 1 2
d) 2 3 1

02. (FUNDATEC/CIGA SC/2023) O que é um container Docker?

a) Uma máquina virtual.
b) Uma imagem.
c) Uma instância executável de uma máquina virtual.
d) Uma instância executável de uma imagem.
e) Um pacote de código-fonte.

03. (VUNESP/UNICAMP/2023) O uso do software Docker tem sido comum no sistema operacional Linux para a criação e execução de aplicações na forma de containers. Nesse contexto, uma imagem de container representa

a) uma janela de interface gráfica onde a saída produzida pela execução do container é apresentada.

b) um container em execução, que se reflete em um ou mais processos no sistema operacional que executa o container.
c) um arquivo de log gerado durante a execução do container, registrando informações importantes para avaliação do desempenho da aplicação.
d) um arquivo no formato JPG ou PNG para exibição da aplicação no navegador, quando esta for publicada em repositórios de containers na web.
e) um arquivo com todo o conteúdo necessário para a execução de um container, incluindo a própria aplicação e suas dependências.

04. (AOCP/IF MA/2023) Suponha que você seja um analista de infraestrutura em uma empresa de tecnologia e esteja trabalhando em um projeto que envolve o uso de Docker para executar aplicativos em contêineres. Sua equipe busca garantir a segurança dos contêineres, sem comprometer a escalabilidade e a portabilidade. Nesse cenário, qual é a maneira eficaz de melhorar a segurança ao executar aplicativos em contêineres com Docker?

a) Executar todos os contêineres com privilégios de root.
b) Utilizar imagens de base não oficiais para contêineres.
c) Implementar políticas de segurança usando Docker Secrets e Docker Content Trust.
d) Executar várias instâncias de aplicativos diferentes em um único contêiner.
e) Ignorar as práticas de isolamento de recursos do Docker.

05. (CEBRASPE/EMPRE/2023) Na virtualização por contêineres, o mecanismo que engloba recursos do sistema em uma abstração é conhecido como

a) control groups.
b) union file systems.
c) bare metal.
d) namespace.
e) inter process communication.

06. (VUNESP/TJ RS/2023) Deseja-se executar um *container* Docker no qual um diretório específico dentro do *container* seja mapeado em um diretório específico da máquina *host*. Esse diretório do *host* deve ser especificado pelo usuário do Docker por meio de seu caminho absoluto no comando de execução do *container*. A solução para esse problema consiste em utilizar

a) uma imagem.
b) um volume.
c) um *bind mount*.
d) um *tmpfs mount*.
e) uma rede virtual.

07. (VUNESP/TJ RS/2023) O seguinte comando da ferramenta Docker serve para remover todos os volumes locais não usados.

```
docker volume prune –all
```

Nesse contexto, os volumes “não usados” são aqueles que

a) estão vazios.
b) não são referenciados por nenhum container.
c) não são usados há um determinado período de tempo, cujo valor default depende das configurações do Docker.
d) são anônimos.
e) não contêm nenhum arquivo de imagem.

08. (CEBRASPE/PREF. FORTALEZA/2023) Tendo em vista que, no atual cenário de desenvolvimento de aplicações web, é essencial considerar princípios, como consistência e escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue o item subsequente.

Um exemplo prático de conteinerização de aplicação é a utilização do Docker para criar um ambiente consistente; nesse caso, é correto criar um arquivo Dockerfile por meio do comando docker build.

09. (CEBRASPE/PREF. FORTALEZA/2023) Tendo em vista que, no atual cenário de desenvolvimento de aplicações web, é essencial considerar princípios, como consistência e escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue o item subsequente.

A adoção de tecnologias de conteinerização, como o Docker, permite à equipe orquestrar vários contêineres em conjunto, por exemplo, um contêiner para a aplicação Node.js e outro, para o banco de dados MongoDB.

10. (CEBRASPE/MPE RO/2023) A seguir, é apresentado um arquivo Dockerfile para a criação de um contêiner.

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
MD ["node", "src/index.js"]
EXPOSE 3000
```

A partir das informações precedentes e considerando que se esteja no mesmo diretório onde se encontra o arquivo Dockerfile, assinale a opção que apresenta o comando a ser executado para criar uma imagem do contêiner com o nome node-js.

a) docker cp -t node-js .
b) docker build -t node-js .
c) docker push -t node-js .
d) docker image -t node-js .
e) docker pull -t node-js .

11. (CEBRASPE/MPE RO/2023) Assinale a opção correspondente ao comando, em Docker, que permite ao usuário fazer o download de uma imagem específica ou um conjunto de imagens do Docker Hub.

a) docker attach
b) docker node ls
c) docker pull
d) docker rmi
e) docker rm /redis

12. (CEBRASPE/DATAPREV/2023) No que se refere a conceitos de contêineres e orquestração de contêineres, julgue o item a seguir.

É possível visualizar o consumo de recursos pelo contêiner no Docker, por meio do comando `docker container ls`.

13. (CEBRASPE/DPE RO/2023) Em programação, os contêineres são utilizados em larga escala no desenvolvimento, testes e, principalmente, na produção de softwares, permitindo rodar múltiplos sistemas isolados dentro de um sistema operacional real.

A esse respeito, julgue os itens a seguir.

I. Em um contêiner, é possível utilizar diversos recursos e ferramentas, porém tudo roda em um mesmo sistema operacional; em caso de pane, todas as funcionalidades serão afetadas.
II. Ferramentas embasadas nos contêineres Linux oferecem aos usuários acesso sem precedentes a aplicações, além da habilidade de implementar com rapidez e de ter total controle sobre as versões e distribuição.
III. As ferramentas de contêiner, incluindo o Docker, fornecem um modelo de implantação com base em imagens, facilitando o compartilhamento de uma aplicação ou conjunto de serviços, incluindo todas as dependências deles em vários ambientes.

Assinale a opção correta.

a) Apenas o item I está certo.
b) Apenas o item II está certo.
c) Apenas os itens I e III estão certos.
d) Apenas os itens II e III estão certos.
e) Todos os itens estão certos.

14. (CEBRASPE/DPE RO/2023) A orquestração automatiza a implantação, o gerenciamento, a escala e a rede dos contêineres. As ferramentas de orquestração de contêineres fornecem um framework para gerenciar arquiteturas de microsserviços e contêineres em escala, e muitas delas são usadas no gerenciamento do ciclo de vida dos contêineres; entre elas, o Docker Swarm é uma plataforma

a) que permite utilizar diversos recursos e ferramentas, como Apache e PHP, porém tudo rodando em um mesmo sistema operacional.
b) de código aberto criada pelo Google para operações de implantação de contêiner, aumento e redução e automação em clusters de hosts.
c) de orquestração de contêiner de código aberto, sendo o mecanismo de clusterização nativo para e pelo Docker, utilizando sua mesma linha de comando.
d) que roda sobre o Kubernetes instalado em sistema operacional na versão Enterprise da Red Hat, agregando opções de monitoramento, integração e entrega contínua.
e) usada pela Amazon para fornecer outros serviços aos clientes, como DNS, balanceamento, segurança e monitoramento, se integrando

15. (FGV/CGU/2022) Uma das estratégias para reduzir o tamanho de imagens Docker consiste em:

a) combinar comandos RUN em um único comando;
b) substituir a imagem base por uma versão mais recente;
c) separar comandos RUN complexos em comandos menores;
d) reordenar os comandos de forma que o cache seja utilizado com maior frequência;
e) compactar arquivos a serem copiados para a imagem e descompactá-los durante a sua geração.

16. (FGV/SEFAZ AM/2022) Leia o fragmento a seguir.

"A plataforma Docker usa uma arquitetura do tipo _____. O cliente Docker conversa com o daemon do Docker, que constrói, executa e distribui _____ Docker. O cliente e o daemon do Docker podem ser executados em um mesmo sistema ou se conectar um cliente do Docker a um daemon remoto. O cliente Docker e o daemon se comunicam usando _____ ou uma interface de redes."

Assinale a opção cujos itens completam corretamente as lacunas do fragmento acima, na ordem apresentada.

a) MVC – imagens – chamadas RPC ou *bluetooth*.
b) *thin client* – contêineres – wireless ou *bluetooth*.
c) *serverless* – componentes – chamadas MPI ou RPC.
d) cliente-servidor – contêineres – API REST ou soquetes UNIX.
e) *mesh app and service* – imagens – *API RESTFULL ou wireless*.

17. (FGV/SEFAZ AM/2022) Com relação à segurança dos contêineres Docker, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

( ) Possuem isolamento no nível do processo no sistema operacional e isolamento adicIonal, usando recursos especiais tais como *namespaces* e *cgroups*.
( ) Aproveitam-se dos mecanismos *interprocess communication* padrão, tais como sinais, pipes e sockets, onde cada contêiner possui a sua própria *network stack*.
( ) Os contêineres possuem um *multi level security* no sistema operacional do host e os recursos físicos que são gerenciados por um *hypervisor* compartilhado.

As afirmativas são, na ordem apresentada, respectivamente,

a) F – V – V.
b) F – F – V.
c) V – F – V.
d) V – F – F.
e) V – V – F.

18. (FCC/TJ CE/2022) No Linux, em condições ideais, o Docker é disponibilizado com três redes por padrão. Essas redes oferecem configurações específicas para o gerenciamento do tráfego de dados e

a) deve-se, para visualizar as redes no Docker, utilizar o comando: docker network -show
b) todos os containers que estão na rede None poderão se comunicar via protocolo TCP/IP. Se uma pessoa souber qual é o endereço IP do container que deseja conectar, é possível enviar tráfego para ele, pois estão todos na mesma rede IP (172.17.0.0/24).
c) a rede Host tem como objetivo isolar o container para comunicações externas. A rede não recebe qualquer interface para comunicação externa. A única interface de rede IP será a localhost. Essa rede, normalmente, é utilizada para containers que manipulam apenas arquivos de backup, sem necessidade de enviá-los via rede para outro local.

02648216103 - HENRIQUE AMORIM BARRETO

d) a rede None tem como objetivo entregar para o container todas as interfaces existentes no docker none. Isso agiliza a entrega dos pacotes, uma vez que não há host no caminho das mensagens. Mas o uso de um host pode ser importante para a segurança e a gerência do tráfego.
e) Bridge é a rede padrão para qualquer container iniciado no Docker, a menos que seja, explicitamente, associada outra rede a ele. Os containers da rede default bridge podem acessar uns aos outros somente através de seus endereços IP, a menos que se utilize a opção --link

19. (FCC/TRT 19/2022) A fim de utilizar o Docker em sua organização, um Analista necessitou conhecer os principais componentes dessa plataforma, tais como:

I. Software que roda na máquina onde o Docker está instalado. Recebe comandos do cliente a partir de Command Line Interfaces ou API's REST.
II. Mecanismo usado para criar imagens e containers.
III. Coleção de imagens hospedadas e rotuladas que juntas permitem a criação do sistema de arquivos de um container. Pode ser público ou privado.
IV. Repositório usado para hospedar e baixar diversas imagens. Pode ser visto como uma plataforma de Software as a Service (SaaS) de compartilhamento e gerenciamento de imagens.

Os itens de I a IV correspondem, correta e respectivamente, a Docker

a) Conteiner – Compose – Swarm – Image.
b) Daemon – Image – Container – Registry.
c) Engine – Swarm – Hub – Registry.
d) Daemon – Engine – Registry – Hub.
e) Conteiner – Engine – Compose – Hub.

20. (FAPEC/UFMS/2022) Sobre os conceitos de virtualização e conteinerização, analise as assertivas a seguir.

I- Na virtualização, cria-se uma nova camada de computação, na qual uma máquina virtual permite a virtualização de toda a infraestrutura de TI (rede, servidores e storage) através de um sistema operacional próprio (hipervisor).
II - Conteinerização (contêiner) é um processo de computação que utiliza uma pequena fração do recurso de hardware para viabilizar a execução de aplicativos de forma mais eficiente e isolada. No caso do contêiner, permite a virtualização de aplicativos de software, utilizando o sistema operacional do host em um Kernel compartilhado.
III -As máquinas virtuais são medidas em megabyte. Elas contêm, no máximo, a aplicação e os arquivos necessários para executá-la. Além disso, elas costumam ser utilizadas para empacotar funções individuais que realizam tarefas específicas, os famosos microsserviços.

Como são leves e possuem um sistema operacional compartilhado, as VMs são muito fáceis de migrar entre vários ambientes.

IV -Os contêineres são medidos em gigabyte. Eles costumam ter seu próprio sistema operacional, o que possibilita a execução simultânea de várias funções com uso intenso de recursos. Por terem um número maior de recursos à disposição, contêineres são capazes de abstrair, dividir, duplicar e emular por inteiro servidores, sistemas operacionais, desktops, bancos de dados e redes.

Estão corretas, apenas, as assertivas:

a) I e ll.
b) II e III.
c) lll e IV.
d) I e III.
e) II e lV.

02648216103 - HENRIQUE AMORIM BARRETO

GABARITO

GABARITO



1. Letra B
2. Letra D
3. Letra E
4. Letra C
5. Letra D
6. Letra C
7. Letra B
8. Errado
9. Certo
10. Letra B
11. Letra C
12. Errado
13. Letra D
14. Letra C
15. Letra A
16. Letra D
17. Letra E
18. Letra E
19. Letra D
20. Letra A

KUBERNETES

Introdução



Originalmente criado pela Google, **Kubernetes** , ou **K8s**, é um sistema ***open-source*** de **orquestração e escalamento de sistemas compostos de contêineres**. Atualmente, o Kubernetes é mantido pela *Cloud Native Computing Foundation* (CNFC).

Kubernetes é, de longe, o sistema mais popular de orquestração, sendo suportado pelas mais diversas plataformas de nuvem, fornecendo uma alta abstração para definir suas aplicações conteinerizadas.

> A **orquestração de contêineres** é uma prática de gerenciamento de contêineres, que envolve controlar, coordenar o *deployment*, a escalabilidade, a disponibilidade e, principalmente, **automatizar** todo esse processo.

Antes de entrarmos em conceitos mais profundos, vamos ter uma visão geral da estrutura do Kubernetes.

Estrutura Geral

O K8s, assim como o Docker, segue a arquitetura **cliente-servidor**, trabalhando com *nodes*, ou nós, em conjunto – formando um ***cluster K8s***. Aqui, trabalhamos com dois tipos de nós:

- **Nó Master:** é o responsável por gerenciar e coordenar os trabalhos no cluster. Um conjunto de nós master formam um plano de controle.
- **Nó Trabalhador (Worker/Slave):** são os nós que recebem os trabalhos para serem executados, hospedando de fato os contêineres. Um conjunto de nós de trabalho formam as máquinas de computação.

Dentro dos **nós trabalhadores**, onde os contêiners rodam, temos três componentes principais:

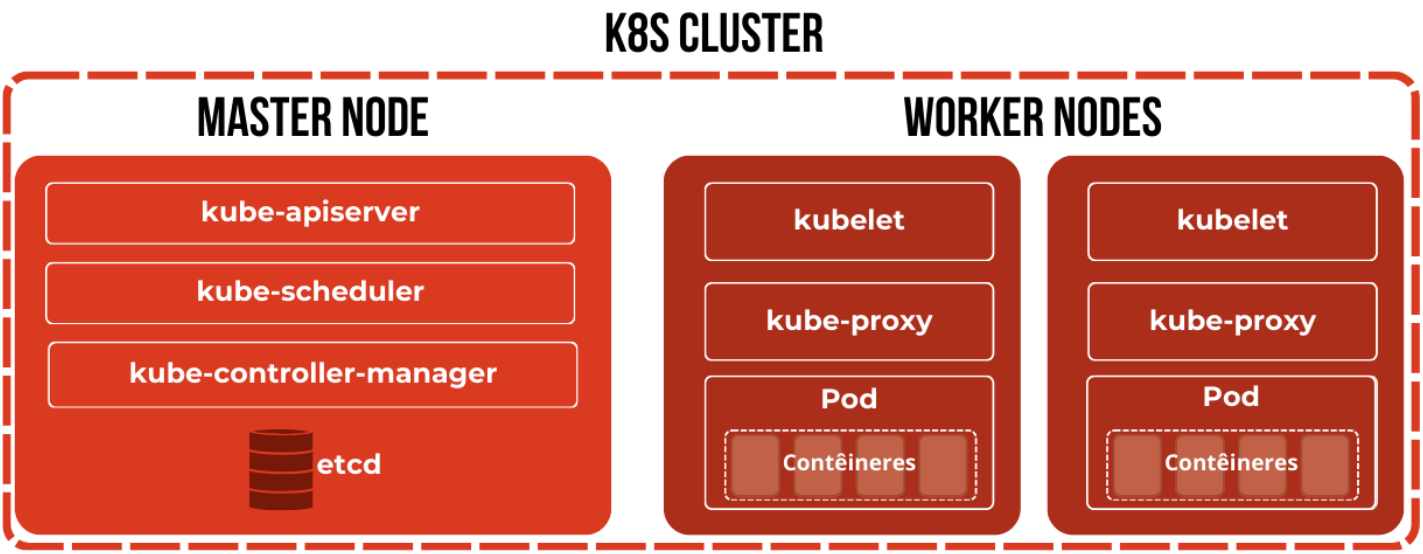
- **kubelet:** é o agente responsável por comunicar com o plano de controle (nó master), assim como funcionam os *daemons* no Swarm. Ele que executa as instruções recebidas, e garante que o nó esteja rodando de forma própria.
- **kube-proxy:** é uma rede de proxy rodando em cada nó, responsável por comunicação dentro e fora do cluster
- **Criador de contêineres:** o Kubernetes, por si só, não cria os contêineres, precisando de ferramentas externas (chamadas também de *container runtime*), como o Docker. Os contêineres, dentro do Kubernetes, são agrupados em estruturas chamadas ***pod*** – que estudaremos logo mais.

Já o **nó master** (falo no singular, mas ***pod*emos ter vários nós master** para maior disponibilidade), onde todo o plano de controle roda, também tem alguns componentes importantes:

- **kube-apiserver:** ferramenta responsável por expor a Kubernetes API, responsável pela comunicação com o cluster k8s. Ele age como uma espécie de *front-end* para o plano de controle.

02648216103 - HENRIQUE AMORIM BARRETO

- **etcd:** um banco de dados distribuído, do tipo chave-valor, que armazena os dados relacionados ao cluster.
- **kube-scheduler:** é o responsável por alocar os *pods* nos nós trabalhadores disponíveis, levando em conta vários fatores, como recursos disponíveis, restrições, entre outros
- **kube-controller-manager:** o controlador do K8s é responsável pro observar um objeto e garantir que o seu estado atual esteja de acordo com o especificado



Objetos

Objetos, no Kubernetes, são entidades persistentes no seu sistema. O K8s usa essas entidades para representar o estado do cluster, descrevendo, dentre outros:

- Que contêineres estão rodando, e onde
- Que recursos estão alocados a cada aplicação
- As políticas ao redor dessas aplicações, como réplicas e políticas de reinicialização

Um objeto é um "registro de intenção" e, uma vez criado, o K8s trabalhará constantemente em cima desse objeto para garantir que ele exista. Esses objetos nada mais são que estruturas de dados, em YAML ou JSON, criados a partir da API Kubernetes.

Quase todos os objetos do Kubernetes possuem dois campos de objeto aninhados que governam a sua configuração: os objetos `spec` e `status`. O `spec` irá definir as características que você quer que o recurso tenha – **seu estado desejado**. Já o `status` irá descrever o estado atual do objeto, atualizado constantemente pelos componentes do Kubernetes – e gerenciado pelo plano de controle.

Abaixo, um exemplo de um *deployment*, que é um objeto que *pod*e representar uma aplicação rodando no *cluster*. Não se preocupe em entender tudo agora – até o final da aula você conseguirá tranquilamente.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Esse arquivo YAML recebe o nome de **manifesto**. Por convenção, os manifestos são escritos em YAML – mas nada impede que ele seja escrito em JSON. Aliás, o Kubernetes interpreta o manifesto como um JSON, já que, se o arquivo estiver em YAML, será feita uma conversão automática para JSON pelo `kubectl`.

Após criarmos o arquivo de manifesto, *pod*emos **criar** o *deployment* com o comando `kubectl apply`, passando o arquivo .yaml como um argumento. Por exemplo:

```
kubectl apply -f https://k8s.io/examples/application/deployment.yaml
```

E pronto, temos o objeto persistido dentro do ecossistema do Kubernetes. Veja que "apply" é um comando estranho para criar – e as bancas pegam justamente nesse detalhe. Olhe uma questão da prova da Câmara dos Deputados.

(QUESTÃO INÉDITA/PROF. VINÍCIUS BORGES) Fernando está gerenciando um cluster Kubernetes e precisa implantar um novo pod que hospedará um contêiner com uma aplicação web.

Ele descreveu todas as especificações do objeto (imagem 'minhaapp/web:1.0', políticas de reinício e portas) em um arquivo de manifesto chamado pod.yaml.

Com base nessas informações, assinale a opção que indica o comando necessário para criar ou atualizar o recurso no cluster utilizando esse arquivo.

a) kubectl run pod.yaml --image=minhaapp/web:1.0 -restart=Always
b) kubectl create pod pod.yaml --image=minhaapp/web:1.0 -restart=Always
c) kubectl apply -f pod.yaml

d) kubectl create deployment pod.yaml -image=minhaapp/web:1.0
e) kubectl expose pod pod.yaml --port=80 --target-port=8080

Comentários:

O comando para criarmos qualquer objeto dentro do Kubernetes a partir de um YAML é o apply -f, onde o -f indica que devemos ler a definição do objeto antes de criá-lo, para saber do que se trata (um serviço, pod, deployment, etc – veremos mais sobre durante a aula). Nesse sentido, o gabarito correto é a letra C.

Um cuidado é com o comando run, que é usado no Docker para criarmos e rodarmos determinado contêiner.
(Gabarito: Letra C)

kubectl

O `kubectl` é a ferramenta de Interface de Linha de Comando (CLI) do Kubernetes, responsável por comunicar os nós com o plano de comando, usando a API K8s. A sintaxe do *kubectl* é simples, muito similar ao que vemos no Linux, e bem declarativa – e segue como padrão:

```
kubectl [comando] [tipo] [nome] [flags]
```

Onde:

- **kubectl** – indica que estamos falando com o CLI do Kubernetes
- **comando** – especifica a operação que queremos realizar, como `create`, `get`, `describe` e `delete`.
- **tipo** – especifica o tipo de recurso que queremos. Os tipos de recurso são *case-insensitive*, isso é, não diferenciam maiúsculas de minúsculas, e tmabém não diferenciam plural, singular e formas abreviadas (por exemplo, *pod*, *pods* e po)
- **nome** – especifica o nome do recurso
- **flags** – especifica flags opcionais, para refinar o comando

Veremos vários comandos ao longo da aula, então não se preocupe em decorar os comandos agora – apenas entender a estrutura da sintaxe. Alguns exemplos de comandos incluem:

- `kubectl get pods`
- `kubectl get services`
- `kubectl descibre pod <nome_do_pod>`
- `kubectl apply -f arquivo-deploy.yaml`
- `kubectl delete service <nome_do_serviço)`

Os comandos em uma CLI *pod*em ser de dois tipos:

- Imperativos: operamos diretamente em objetos em funcionamento (*live objects*)
- Declarativos: operamos em arquivos de configuração locais, que não estão “ao vivo”

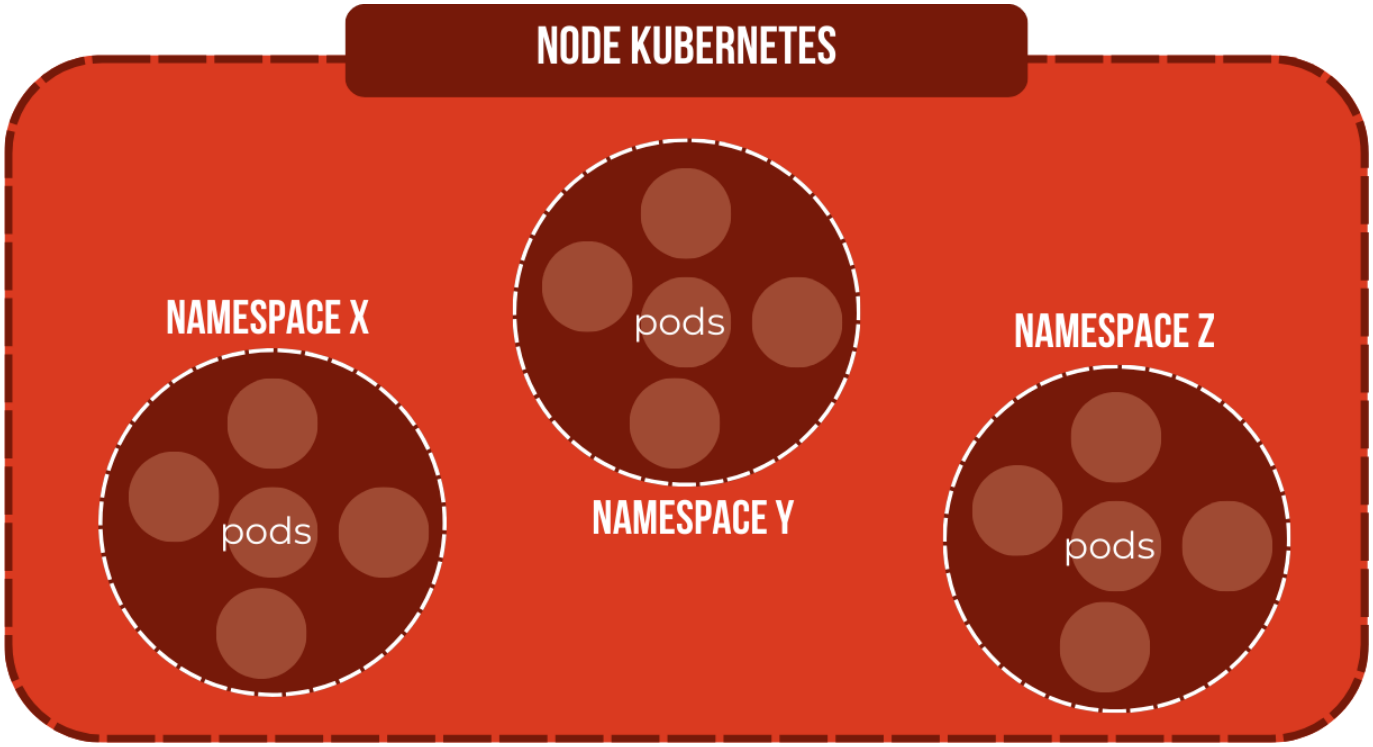
namespaces

Os ***namespaces*** fornecem mecanismos para isolar grupos de recursos dentro de um único cluster, criando um “espaço virtual” dentro dele. Assim, cada *namespace* fornece um escopo para os recursos dentro dele,

ajudando a organizar e gerenciar o aplicativo. Dessa forma, dentro de cada *namespace* os recursos devem ter nomes únicos – mas, graças a essa separação, *pod*emos ter nomes repetidos em outros *namespaces*.

Com isso, conseguimos:

- **Isolamento de recursos:** Os namespaces permitem isolar recursos como *pods*, serviços e volumes, evitando conflitos entre aplicativos que *pod*em estar em execução no mesmo cluster.
- **Organização de aplicativos:** Você *pod*e usar namespaces para organizar seus aplicativos com base em ambientes (por exemplo, desenvolvimento, teste, produção) ou equipes de desenvolvimento, facilitando a gestão e visualização dos recursos relacionados a esses aplicativos.
- **Controle de acesso**: Os namespaces *pod*em ser usados para controlar o acesso aos recursos do Kubernetes, permitindo que diferentes equipes ou usuários tenham permissões específicas apenas em determinados namespaces.
- **Limitação de recursos:** É possível definir políticas de limitação de recursos (como CPU e memória) em nível de namespace, garantindo que os recursos sejam distribuídos de forma justa entre os diferentes aplicativos em execução no cluster.
- **Escalabilidade:** Os namespaces permitem que você dimensione seus aplicativos de forma mais eficiente, permitindo que você crie namespaces adicionais conforme necessário para acomodar mais aplicativos ou equipes.



nodes

Você já sabe o que é e quais são os nós, agora vamos para detalhes mais técnicos. Primeiramente, **como criar um nó**. Os nós nada mais são que um objeto no K8s, e, para tanto, o criamos a partir de um manifesto – nesse caso, o manifesto é especificamente feito em JSON, já que ainda não temos o kubectl para converter o YAML em JSON.

JSON

```
{
    "kind": "Node",
    "apiVersion": "v1",
    "metadata": {
      "name": "10.240.79.157",
      "labels": {
        "name": "meu-primeiro-node-k8s"
      }}}
```

*Pod*emos adicionar o nó ao cluster de duas formas:

- Manualmente
- Por um registro automático, partindo do kubelet



Relembrando: o **kubelet** é um agente que roda em cada nó de um cluster Kubernetes. Ele garante que os containers estejam rodando nos *pods* conforme configurado. O kubelet recebe informações sobre qual *pod* deve rodar em um nó através do API Server do Kubernetes e garante que os containers nesses *pods* estejam rodando e saudáveis. Ele também lida com a execução de comandos em containers, como execução de comandos remotos ou logs.

Caso queiramos a segunda opção (que é ativada, por padrão), devemos apontar a flag na criação `--register-node = true`. Caso isso ocorra, o kubelet é iniciado com as seguintes opções:

- **`--kubeconfig`** – caminho para autenticar as credenciais do nó no servidor API
- **`--cloud-provider`** – como se comunicar com o fornecedor de serviços na nuvem
- **`--register-node`** – registra automaticamente no serivdor API
- **`--register-with-taints`** – registra o nó com um par de *taints* (trio de dados `<chave>=<valor>:<efeito>`, separado por vírgulas)

A partir disso, *pod*emos gerenciar nossos nós com o nosso *kubectl*. Vamos ver as principais interações que *pod*emos ter.

Comando	Descrição
`kubectl get node`	Lista todos os nós de um cluster
`kubectl describe node <nome>`	Descreve detalhes de um nó específico

`kobectl cordon <nome>`	Marca um nó como não programável (unschedulable), bloqueando a adição de novos *pods*
`kubectl uncordon <nome>`	Desbloqueia o nó de adições de novos *pods*
`kubectl drain <nome>`	"Drena" o nó, movendo todos os *pods* para outros nós
`kubectl delete node <nome>`	Exclui permanentemente um nó do sistema

(Inédita/Prof. Felipe Mathias) Rafaela é a responsável por monitorar o Kubernetes do SisFISCO, garantindo que todos os nós tenham o correto funcionamento. Em determinada manhã, se deparou com um problema no nó "*node2*", causando instabilidade nos *logins*.

De forma a diminuir os impactos da instabilidade, Rafaela optou por usar um comando que bloqueia a criação de novos *pods* dentro desse nó. Assinale a alternativa que aponta o comando usado por Rafaela.

a) kubectl get node2
b) kubectl drain node2
c) kubectl cordon node2
d) kubectl uncordon node2
e) kubectl -f node2

Comentários:

Para deixarmos um nó com o status de não agendável, bloqueando a adição de novos *pods* a ele, usamos o comando *kubectl cordon <nome_do_nó>*. Para retirarmos dessa situação, *pod*emos usar o comando *kubectl uncordon <nome_do_nó>*. Nesse sentido, para bloquearmos a adição, correta a letra C. *(Gabarito: Letra C)*

Um ponto importante dos nós é o seu **status**. Para verificarmos se um nó está disponível, funcionando corretamente e conforme o estado esperado, usamos uma notificação chamada de **heartbeat**. Esse sinal é uma requisição HTTP que permite monitorar a performance dos cluster k8s.

Quem faz esse monitoramento é o **node controller**, que forma o nosso plano de controle. Dentre suas funções, está justamente a de monitoramento da saúde dos nós. A sua responsabilidade é de alterar o campo `.status`, de um estado ideal `ready`, para, caso haja algum problema, o estado `unknown` ou `false`.

Se o nó permanecer inalcançável, é iniciada uma **evicção** do nó inalcançável. Por padrão, esse tempo de intervalo é de 5 minutos – que *pod*e ser alterado usando a flag, no `kube-controller-manager`, `--node-monitor-period`.
A evicção é responsável por limpar o nó, *por* por *pod*. A velocidade de evicção é definida na diretriz `--node-eviction-rate`, que, por padrão, é de 0.1. Isso significa que não iremos remover mais de 1 nó a cada 10 segundos.

Para desligarmos um node, temos duas abordagens:

- **Desligamento gracioso:** o controlador envia um sinal de desligamento para o kubelet, indicando que o nó será desligado. O kubelet, então, executa um processo de desligamento controlado, garantindo que todos os *pods* sejam encerrados de forma limpa e completa, começando pelos *pods* regulares, e terminando pelos *pods* críticos.
- **Desligamento forçado:** há um desligamento direto do nó pelo controlador, sem comunicação com o kubelet. Essa forma de desligamento só é indicada para momentos críticos.

*Pod*emos usar a ferramenta `GracefulNodeShutdown` para o encerramento gracioso, que irá mudar o status do nó para `NotReady`, bloqueando a adição de novos *pods*. O shutdown recebe duas configurações:

- **shutdownGracePeriod:** especifica a duração total do desligamento, sendo composta pelo tempo de desligamento dos *pods* regulares, mais os *pods* críticos.
- **shutdownGracePeriodCritical*Pods*:** especifica o tempo de encerramento específico aos *pods* críticos.

Os *pods* críticos são baseados no **nível de prioridade** da sua classe. Quanto maior o valor do nível de prioridade. Outra forma de fazer um desligamento gracioso é através da drenagem de um nó, definida pelo comando CLI:

```
kubectl drain <nome>
```

Coleta de Lixo

Coleta de Lixo é o nome dado ao conjunto de ferramentas e mecanismos que o Kubernetes usa para limpar os recursos de um *cluster*. Isso permite a limpeza de recursos como:

- *Pods* encerrados
- Trabalhos completados
- Objetos sem referências
- Imagens e contêineres sem uso
- Entre outros

Os objetos em Kuberenetes conectam-se uns aos outros através de **referências de proprietário** (*owner references*). Essas referências informam uma relação de dependência entre objetos – como vimos no Swarm, aula passada. Essas referências são repassadas para o plano de controle antes de realizar a limpeza, para não termos uma dependência obrigatória sendo encerrada. Usualmente, a limpeza automática se dá apenas em objetos que não possuem nenhuma dependência.
Porém, *pod*e ser o caso de termos objetos que possuem dependências e, ao encerrar a dependência principal, termos a necessidade de encerrar esses elementos dependentes. Esse processo recebe o nome de **deleção em cascata**.

Temos duas abordagens para a deleção em cascada:

- **Foreground Cascading Deletion:** na deleção em primeiro plano (*foreground*), colocamos o proprietário do objeto, também chamado de objeto pai, num estado chamado *deletion in progress*. Isso bloqueia o objeto proprietário, e sinaliza ao controlador que devemos começar a deletar os recursos dependentes (objetos filhos). Após todas as dependências serem removidas, removemos o objeto principal.

- **Background Cacading Deletion:** na deleção em segundo plano (*background*), temos a deleção direta do objeto proprietário, feita diretamente pela API Kubernetes, e, posteriormente, a deleção dos objetos filhos feita pelo controlador.

*Pod*emos ter também uma política específica para a deleção de contêineres não usados, que ignorará as referências. Ela considerará três variáveis principais:

- **minAge** – idade mínima para que o contêiner possa ser deletado
- **MaxPer*Pod*Container** – quantidade máxima de contêineres considerados "mortos" dentro de um *pod*
- **MaxContainers** – número máximo de contêineres "mortos", no total do cluster

Parece um detalhe bobo – mas preste bastante atenção no min/max, já que as bancas costumam brincar com isso. Veja:



(FGV/BBTS/2023) *Garbage Collection* é um dos componentes arquiteturais do Kubernets e é um termo utilizado por vários mecanismos de limpeza de recursos em um *cluster*.Com relação aos mecanismos de *Garbage Collection* do Kubernets, versão 1.27, analise as afirmativas a seguir e assinale (**V**) para a verdadeira e (**F**) para a falsa.

I. Existem dois tipos de exclusão em cascata: *Foreground cascading deletion* e *Background cascading deletion*.
II. A limpeza dos contêineres não utilizados é baseada nas variáveis *LowAge*, *HighPerPodContainer* e *HighContainers*.
III. O ciclo de vida de uma imagem de contêiner é realizado pelo *Image Manager-Kubelet* que considera os limites de uso de disco definidos por *MaxThresholdPercent* e *MinThresholdPercent* para tomar decisões de limpeza.

As afirmativas são, respectivamente,

a) F – V – V.
b) F – F – V.
c) V – V – V.
d) V – F – F.
e) V – V – F.

Comentários:

02648216103 - HENRIQUE AMORIM BARRETO

Vamos analisar cada um dos itens.

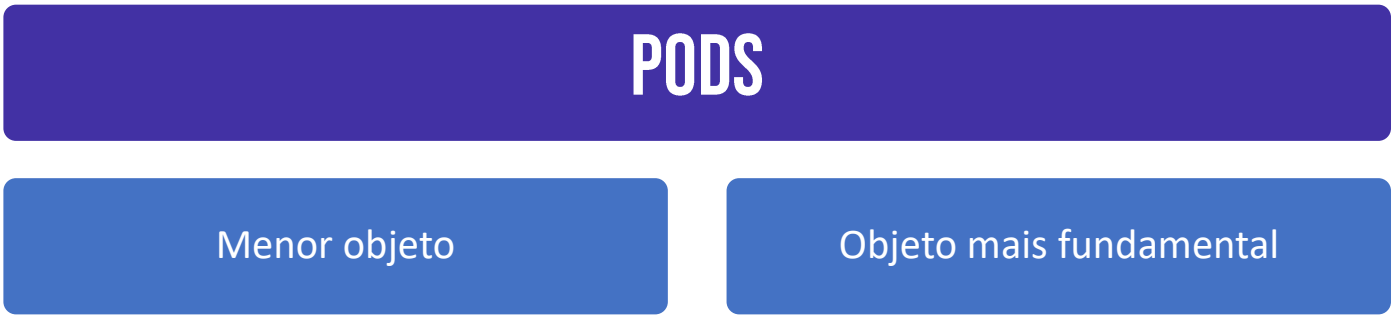
I. Certo. De fato, temos dois tipos de exclusão em cascata – a *foreground* e a *background*.
II. Errado. Galera, cuidado! Os valores são minAge, MaxPer*Pod*Container e MaxContainers.
III. Errado. Não vimos ainda, mas também houve uma mudança nos parâmetros aqui – o correto seria HighThresholdPercent e LowThresholdPercent.

Portanto, temos V – F – F. *(Gabarito: Letra D)*

Pods

Vamos falar agora sobre os elementos mais importantes do Kubernetes – as diferentes **cargas de trabalho**, ou ***workloads***. Essas cargas de trabalho *pod*em ser aplicativos, serviços, tarefas ou processos que são executados em contêineres dentro do ambiente Kubernetes. Kubernetes oferece vários tipos de workloads para lidar com diferentes necessidades de implantação e escalonamento de aplicativos.

Os *pods* são as **menores** e **mais fundamentais** cargas de trabalho do Kubernetes.



Antes mesmo de eu te explicar o restante, decore o que eu passei acima: os *pods* são as menores e mais fundamentais unidades do Kubernetes. Veja o porquê de decorar isso:



(IBFC/TJ MG/2022) A menor unidade de computação implantável que se *pod*e criar e gerenciar no Kubernetes são denominados tecnicamente como sendo:

a) bean
b) shell
c) kernel
d) *pod*
e) hem

Tem outra...

(FGV/MPE GO/2022) Uma equipe de especialistas em infraestrutura está implantando o orquestrador Kubernetes em um servidor do MP-GO que possui vários containers Docker. Foi decidido que alguns desses containers serão agrupados na unidade básica de operações do Kubernetes.

Para isso, a equipe deve definir um novo

a) statefulSet.
b) replicaSet.
c) service.
d) ingress.
e) *pod*.

Não preciso nem dizer qual é o gabarito dessas questões acima – e de muitas outras que cobram a mesma coisa, né? Obviamente que são os ***pods*!**

Bom, retomando a explicação *rs*... os *pods* é um grupo de um ou mais contêineres, que compartilham recursos de armazenamento e processamento, e uma localziação que especifica como rodar esses contêineres. Eles são a unidade mínima de um cluster Kuberenetes. Agora, você deve estar pensando "*ué, mas não temos os contêineres?*".

E bom, temos sim – mas o K8s tem uma abordagem um pouco diferente. Ao invés de rodar o contêiner diretamente no nó, o Kubernetes "embrulha" o conjunto de contêineres que você precisa num cápsula, chamada de ***pod***.

Um *pod*, usualmente, roda um único contêiner – mas nada impede que tenhamos vários contêineres dentro dele. O importante é saber que todos os contêineres dentro de um mesmo *pod* rodaram num mesmo nó, compartilhando a rede, armazenamento e recursos.

Outra característica do *pod* é que, uma vez que ele entra em atividade, ele é **imutável**. Uma vez criados, seus detalhes fundamentais, como endereço de IP, identificação (nome) e especificação, não *pod*em ser alterados. É necessário criar um novo *pod* com as alterações desejadas. Detalhe – isso não impede a alteração das imagens internas (que criam os contêineres), apenas as especificações do *pod* em si.

Com isso, garantimos alguns benefícios:

- **Confiabilidade:** Ao substituir um *pod* antigo por um novo, você garante que o novo *pod* inicie em um estado conhecido e previsível, evitando problemas causados por mudanças não planejadas.
- **Simplicidade:** A imutabilidade simplifica o gerenciamento de aplicativos, pois você não precisa se preocupar com alterações inesperadas no estado dos *pods* existentes.
- **Reversibilidade:** Se algo der errado com o novo *pod*, você *pod*e simplesmente encerrá-lo e o *pod* antigo continuará funcionando, minimizando o impacto de problemas durante as atualizações.

Para criarmos um *pod*, primeiramente fazemos as especificações no seu manifesto:

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

Esse é um exemplo simples e reduzido de um manifesto YAML para criar um *pod*. Vamos explorar os parâmetros que *pod*emos definir, de forma aninhada (ou seja, atendendo à hierarquia do YAML), para que você saiba responder todas as questões que *pod*em cair sobre isso. Lembrando que, o que garante a hierarquização em um YAML é a endentação – não vá me esquecer isso na hora da prova!

- **apiVersion** → a versão da API Kubernetes que está sendo usada
- **kind** → o tipo de objeto que será criado. No nosso caso, um *pod*.
- **metadata** → metadados associado ao *pod*, como `name`, `labels`, entre outros.
- **spec** → especificações acerca do *pod* – com destaque para o contêiner que irá rodar nele
  - **container** → especificações do contêiner (ou dos contêineres) que fará parte do *pod*. Segue a mesma lógica que vimos no Docker.
  - **name** → especifica o nome do contêiner.
  - **ports** → definições acerca das conexões de rede.
    - **containerPort** → define a porta do contêiner para ser mapeada
    - **hostPort** → define a porta do host para ser mapeada
    - **protocol** → define o protocolo (TCP, UDP, etc) que será usado
  - **resources** → define os recursos que serão usados pelo contêiner, abrangendo. Temos dois tipos de recursos:
    - **requests** → especifica a quantidade de recursos mínima requisitada pelo contêiner. Usualmente, são duas definições: CPU e *memory*.
    - **limits** → estabelece um limite superior aos recursos alocados
    - **env** → define variáveis de ambiente
    - **volumeMounts** → define montagens de volume para o contêiner
- **volume** → define os volumes disponíveis para o *pod*
  - **name** → especifica o nome do volume
  - **emptyDir** → define o tipo de volume que é inicializado como vazio e é apagado quando o *pod* é removido

Leia cada linha dessa lista acima, procurando entender tudo. Essa é a estrutura base de um YAML para um *pod* – dificilmente teremos um campo a mais em questões que não vimos aí. E se vierem, são bem literais, não é difícil de entender.

Ciclo de vida do *pod*

O *pod* possui um ciclo de vida bem definido e delimitado, passando por algumas fases até concluir o seu objetivo. Assim como os contêineres, a vida de um *pod* é considerada efêmera – ao contrário de durável. As suas fases envolvem:

- **pending (pendente)** – o *pod* foi aceita no cluster, mas um ou mais contêineres ainda não estão preparados para rodar. Essa fase inclui o tempo que o *pod* aguarda para ser alocado, assim como o tempo para carregar as imagens.
- **running (rodando)** – o *pod* foi alocado a um nó, e todos os contêineres foram criados – e pelo menos um está funcionando.
- **succeeded (sucedido)** – todos os contêineres do *pod* foram terminados com sucesso e não serão reiniciados
- **failed (falha)** – todos os contêineres do *pod* foram terminados, e pelo menos um terminou com falha (status diferente de 0)
- **unkown (desconhecido)** – por alguma razão, não foi possível obter o estado do *pod*

É possível, também, que alguns comandos do `kubectl` indiquem que o *pod* está numa fase de terminação, ou `terminating`.



Cuidado para não confundir com os **estados** (status) dos **contêineres**. Ou seja, aqui estamos indo nos componentes particulares do *pod*. *Pod*emos ter três estados:

- **Running (rodando)** – indica que o contêiner está sendo executado sem problemas
- **Terminated (terminado)** – indica que, ou o contêiner completou seu objetivo, ou encontrou alguma falha no processo
- **Waiting (esperando)** – caso residual, ou seja, o contêiner não está nem rodando, nem foi terminado. Usualmente ele está rodando operações para completar a inicialização.

Podemos definir alguns *hooks* (ganchos) no manifesto de um *pod* para executar ações específicas em determinados estágios do ciclo de vida do pod. Esses ganchos permitem que você execute comandos personalizados antes ou depois de certas etapas, como antes do pod ser criado ou depois que ele for removido.

Os principais *hooks* são:

- **PrePull:** executado antes de um contêiner ser puxado (pull) do registro de contêiner. Ele pode ser usado para realizar verificações ou configurações adicionais antes que o contêiner seja puxado para o nó.
- **PreStart:** executado antes de um contêiner ser iniciado, mas após o volume ser montado. Pode ser usado para configurar o ambiente do contêiner, como a configuração de arquivos de configuração ou a realização de verificações de integridade.
- **PostStart:** executado imediatamente após um contêiner ser iniciado. É útil para realizar inicializações específicas, como a execução de scripts de configuração ou a configuração de variáveis de ambiente.
- **PreStop:** executado antes de um contêiner ser encerrado. Pode ser usado para realizar ações de limpeza ou para se preparar para o encerramento, como salvar dados ou notificar outros componentes.
- **PostStop:** executado após um contêiner ser encerrado. Ele é útil para realizar ações de limpeza ou para finalizar qualquer processo pendente após o contêiner ser encerrado.



Liveness probes

Galera, imaginem que vocês têm uma aplicação rodando em um contêiner e, de repente, ela entra em um loop infinito ou sofre um deadlock. A aplicação ainda está em execução, então o Kubernetes, por padrão, acharia que está tudo bem e não faria nada. É exatamente para evitar esse falso positivo que usamos o **Liveness Probe**.

O Liveness Probe é um mecanismo usado pelo kubelet para saber **quando deve reiniciar um contêiner**. Se o contêiner falhar repetidamente nessa verificação de saúde, o kubelet simplesmente **mata** esse contêiner e o reinicia. Isso ajuda a manter a aplicação disponível mesmo quando ocorrem bugs severos que travam o sistema.

Para fazer essa checagem, podemos configurar a sonda de algumas formas, como executando um comando dentro do contêiner ou fazendo uma requisição HTTP. Por exemplo, o kubelet pode executar um comando específico lá dentro e, se o comando retornar um valor numérico diferente de zero, indicando erro, o kubelet entende que o contêiner não está saudável e força a reinicialização.

Vale a pena não confundir com a Readiness Probe. Enquanto a Liveness Probe decide se o contêiner deve ser morto e reiniciado, a **Readiness Probe determina se o contêiner já está pronto para receber tráfego de rede**. São conceitos diferentes para momentos diferentes do ciclo de vida da aplicação.

Vamos ver como isso fica no nosso manifesto YAML. Prestem atenção na endentação, que fica dentro da especificação do contêiner.

```
apiVersion: v1
kind: Pod
metadata:
  name: meu-pod-liveness
spec:
  containers:
  - name: minha-aplicacao
    image: nginx:1.14.2
    livenessProbe:
      httpGet:
        path: /healthz
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
```

Vejam que definimos um caminho HTTP e também configuramos parâmetros como initialDelaySeconds e periodSeconds, que controlam o tempo de espera antes da primeira checagem e a frequência das checagens, respectivamente.

Um ponto que as bancas adoram usar para confundir o candidato é a relação entre o que configuramos na imagem do contêiner e o que configuramos no orquestrador. Vocês devem lembrar dos nossos estudos sobre Docker que existe uma instrução chamada **HEALTHCHECK**, colocada direto no Dockerfile, para testar se o contêiner está saudável e respondendo corretamente.

A grande pegadinha de prova é a seguinte: o Kubernetes **ignora completamente a instrução HEALTHCHECK do Docker**. Isso mesmo, galera. Se você gastou linhas e linhas configurando um teste de saúde maravilhoso no seu Dockerfile, o kubelet não vai olhar para ele na hora de decidir se o contêiner travou ou não.

No ecossistema do Kubernetes, **quem manda na verificação de saúde para fins de reinicialização é a Liveness Probe**, configurada diretamente no manifesto YAML do Pod ou do Deployment. Enquanto o HEALTHCHECK do Docker é uma configuração atrelada à imagem construída, a Liveness Probe é uma configuração atrelada à execução e orquestração daquela carga de trabalho no cluster.

A banca vai tentar te convencer de que o Kubernetes converte o HEALTHCHECK do Docker em uma Liveness Probe automaticamente, ou que eles trabalham em conjunto nativamente. Não caia nessa. Para o Kubernetes agir e reiniciar um contêiner travado, você obrigatoriamente precisa declarar e definir a sua Liveness Probe no manifesto do objeto. O Docker até usa o HEALTHCHECK para relatar o status do contêiner em ferramentas nativas como o Docker Swarm, mas quando subimos para o nível do K8s, essa responsabilidade muda de dono.

Contêineres

O que são contêineres e suas propriedades você já viu – até porque o Kubernetes exige um *container runtime*, como o Docker, para funcionar, já que ele não gera os contêineres. O ponto aqui é focar em alguns tipos de contêineres específicos. A saber:

- **Init Containers:** São contêineres que são executados antes do contêiner principal em um *pod*. Eles são usados para realizar tarefas de inicialização, como preparar o ambiente, baixar arquivos de

configuração ou inicializar bancos de dados, antes que o contêiner principal seja iniciado. Os contêineres Init são úteis para garantir que certas condições estejam em vigor antes que o contêiner principal comece a ser executado.

- **Ephemeral Containers:** São contêineres temporários que *pod*em ser adicionados a um *pod* em execução, sem precisar modificar o manifesto do *pod*. Eles são úteis para depuração, coleta de logs ou execução de tarefas ad hoc dentro do mesmo contexto de rede e recursos do *pod*. Os contêineres efêmeros são úteis para operações pontuais que não justificam a criação de um novo *pod*.
- **Sidecar Containers:** São contêineres que oferecem funcionalidades auxiliares para o contêiner principal em um *pod*, como logging, monitoramento, proxy, ou até mesmo funcionalidades de segurança. Eles compartilham o mesmo ciclo de vida e recursos do contêiner principal e são usados para estender ou complementar a funcionalidade do contêiner principal de uma maneira modular e desacoplada.

Qualidade de Serviço (QoS)

O K8s classifica os *Pods* em uma classe específica de qualidade de serviço – **Quality of Service (QoS)**. Essa classe influencia como diferentes *pods* são gerenciados. A classificação é feita a partir de pedidos de recurso dos contêineres dentro de um *pod,* juntamente com como esses pedidos se relacionam ao limite estabelecido.

Essa métrica de QoS é importante para a evicção dos *pods*, principalmente quando ela se dá devido ao excesso de consumo dos recursos. Temos 3 classes de QoS:

- **Guaranteed (garantida):** Os *pods* nesta classe têm requisitos de recursos especificados para CPU e memória. Eles recebem a mais alta prioridade de QoS e são garantidos para ter seus recursos sempre disponíveis. Se os recursos não estiverem disponíveis, o Kubernetes não programará mais *pods* nesse nó.
- **Burstable (expansível):** Os *pods* nesta classe têm requisitos mínimos de recursos, mas também *pod*em usar recursos adicionais se estiverem disponíveis. Eles têm uma prioridade intermediária de QoS e são adequados para cargas de trabalho que *pod*em precisar de mais recursos em momentos de pico, mas *pod*em operar com menos recursos em momentos normais.
- **BestEffort (Melhor Esforço):** Os *pods* nesta classe não têm requisitos de recursos especificados. Eles recebem a menor prioridade de QoS e *pod*em usar qualquer recurso disponível no nó. Esses *pods* são adequados para cargas de trabalho não críticas ou que *pod*em se adaptar facilmente a variações na disponibilidade de recursos.



Gerenciadores de Workload

De forma geral, o Kubernetes roda as aplicações dentro de *pods*, que comporta os contêineres. Porém, gerenciar cada *pod pod*e ser uma tarefa difícil e trabalhosa – vai que um *pod* falha e você não está disponível para acompanhar?

Nesse contexto, entram em trabalho os **gerenciadores de carga de trabalho**. Eles visam facilitar o trabalho de coordenar múltiplos *pods* e automatizar o processo.

Deployments

*Pod*e ser que eu não queria criar *pod por pod* – para isso, usamos o *deployment. Um* **Deployment** no Kubernetes é um recurso usado para gerenciar a implantação de aplicações em contêineres de forma declarativa.

Um Deployment define o estado desejado de um conjunto de *pods*, permitindo que o Kubernetes gerencie a criação e a atualização desses *pods* para atender a esse estado desejado. Ele garante que um número especificado de réplicas de um *pod* esteja em execução em todos os momentos, lidando automaticamente com a substituição de *pods* que falharam ou foram removidos.

Da mesma forma que os *pods*, definimos a estrutura da inicialização de um deployment através de um YAML. Veja um exemplo:

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: meu-deployment
  labels:
    app: minha-aplicacao
spec:
  replicas: 3
  selector:
    matchLabels:
      app: minha-aplicacao
  template:
    metadata:
      labels:
        app: minha-aplicacao
    spec:
      containers:
      - name: meu-container
        image: minha-imagem:tag
        ports:
        - containerPort: 80
      restartPolicy: Always
```

Veja que a estrutura é quase idêntica à do YAML para os *pods*. Porém, aqui temos algumas diferenças notáveis. Vamos "dissecar" essa estrutura – sempre prestando atenção na endentação e hierarquia.

- **`apiVersion`** → a versão da API Kubernetes que está sendo usada
- **`kind`** → o tipo de objeto que será criado. No nosso caso, um *deployment*
- **`metadata`** → metadados associado ao *pod*, como `name`, `labels`, entre outros.
- **`spec`** → o spec de nível mais alto (raiz) irá definir especificações do Deployment em si, como número de réplicas, seletor, template, entre outros
  - **`replicas`** → define a quantidade de réplicas do *pod* que o deployment deve manter em execução a todo momento
  - **`selector`** → seletor usado para definir quais *pods* são gerenciados pelo deploy
  - **`template`** → passa definições do *pod* que será criado
    - **`metadadata`** → metadados associados ao *pod*
    - **`spec`** → nosso segundo spec, agora passando especificações do *pod* que será criado
      - **`container`** → específica um ou vários contêineres que serão executados
    - **`restartPolicy`** → define a política de reinicialização, que *pod*e ser `always`, `onFailure` ou `never`.

Vamos ver como isso já foi cobrado? Aqui, uma questão discursiva na SEF MG!

(FGV/SEF MG/2023) Considere um Deployment para executar um servidor web Nginx em um cluster Kubernetes. O código YAML atual para o Deployment é o seguinte:

```
1   apiVersion: apps/v1
2   kind: Deployment
3   metadata:
4     name: nginx-deploy
5   spec:
6     selector:
7       matchLabels:
8         app: nginx
9     template:
10      metadata
11      labels:
12        app: nginx
13      spec:
14        containers:
15          name: nginx
16          image:nginx:1.16
```

Apresente um trecho de código que *pode* ser inserido no exemplo acima que define o número de réplicas para 3. Use a numeração de linhas original da coluna da esquerda como referência para indicar claramente onde o código deve ser inserido.

Apresente um trecho de código que *pode* ser inserido no exemplo acima que define o limite do consumo de CPU para 250m e o consumo de memória para 250Mi para cada réplica. Use a numeração de linhas original da coluna da esquerda como referência para indicar claramente onde o código deve ser inserido.

Apresente um trecho de código que *pode* ser inserido no exemplo acima que libere as portas HTTP e HTTPS para o servidor web. Use a numeração de linhas original da coluna da esquerda como referência para indicar claramente onde o código deve ser inserido.

Comentários:

Aluno, essa questão foi uma verdadeira pedrada na cara do aluno. Extremamente difícil -mas é pra isso que eu tô aqui, pra simplificar as coisas na sua vida. Não vou elaborar uma redação completa, mas vamos responder todos os 3 itens.

a)
Queremos definir o número de réplicas, igual a 3. Primeiro passo é saber o comando e onde vamos colocá-lo. O comando é simples, *replicas: 3*. Como se trata de uma definição do deployment, colocamos ele dentro do primeiro *spec*, que aborda as configurações dele. Lembrando sempre da endentação, já que ela garante a hierarquia do YAML.

O código será incluído ali na linha 6, logo após o *spec*. Veja:

YAML

```yaml
1   apiVersion: apps/v1
2   kind: Deployment
3   metadata:
4     name: nginx-deploy
5   spec:
6     replicas: 3
7     selector:
8       matchLabels:
9         app: nginx
10    template:
11      metadata:
12        labels:
13          app: nginx
14      spec:
15        containers:
16          name: nginx
17          image: nginx:1.16
```

b)

O limite de consumo de um contêiner é definida na sua especificação – lá no final, após a linha 15. Para isso, precisamos abrir uma tag nova: **resources**. Dentro dela, como a questão só quer um limite máximo, não quer um pedido mínimo, iremos definir somente isso. O código fica assim:

YAML

```
...
14      spec:
15        containers:
16          name: nginx
17          image: nginx:1.16
18          resources:
19            limits:
20              memory: 250Mi
21              cpu: 250m
```

c)

Por fim, essa última alternativa exigia um nível muito alto pois, além de definirmos a estrutura do YAML, precisávamos lembrar que a porta para o HTTP é a 80, e para o HTTPS é a 443. Essas portas devem ser abertas dentro do contêiner, e *pod*em ser abertas após o código que acabamos de inserir. Nosso código, ao fim, ficará assim:

YAML

```
1   apiVersion: apps/v1
2   kind: Deployment
3   metadata:
4     name: nginx-deploy
5   spec:
6     replicas: 3 #letra a
7     selector:
8       matchLabels:
9         app: nginx
10    template:
11      metadata:
12        labels:
13          app: nginx
14      spec:
15        containers:
16          name: nginx
17          image: nginx:1.16
18          resources: #letra b
19            limits:
20              memory: 250Mi
21               cpu: 250m
22            ports: #letra c
23            -  containerPort: 80
```

Um detalhe importante – sempre que formos numerar listas, como fizemos agora nas portas, que irão gerar chaves (do par chave:valor) repetidas, devemos inicializar cada item com um hífen.

Sets

Os **sets** são objetos controladores do Kubernetes que ajudam a gerenciar os *pods*, cada um com um objetivo específico. Vamos ver cada um deles.

ReplicaSet

O **ReplicaSet** tem como objetivo manter um número específico de réplicas dos *pods* esteja rodando a qualquer momento. Isso objetiva garantir a disponibilidade dos serviços no K8s.
As definições de um ReplicaSet, assim como a maioria dos objetos que vimos até agora, é feita através de um manifesto. Dentro dele, temos seletores que definem quais *pods* devem ser mantidos e quantas réplicas de cada.

*Pod*emos verificar os ReplicaSet disponíveis com o comando `kubectl get rs`.

StatefulSet

Os **StatefulSet** são as APIs de carga de trabalho para gerenciar aplicações que requerem uma atribuição de estado – chamadas de *stateful*. Dessa forma, eles gerenciam um grupo de *pods* que compartilham alguma especificação e exijam um armazenamento persistente (para armazenar os estados).

A diferença ente um StatefulSet e um *deployment* é que, enquanto no *deployment* temos um conjunto de *pods* intercambiáveis, não-únicos, no StatefulSet cada *pod* é único.

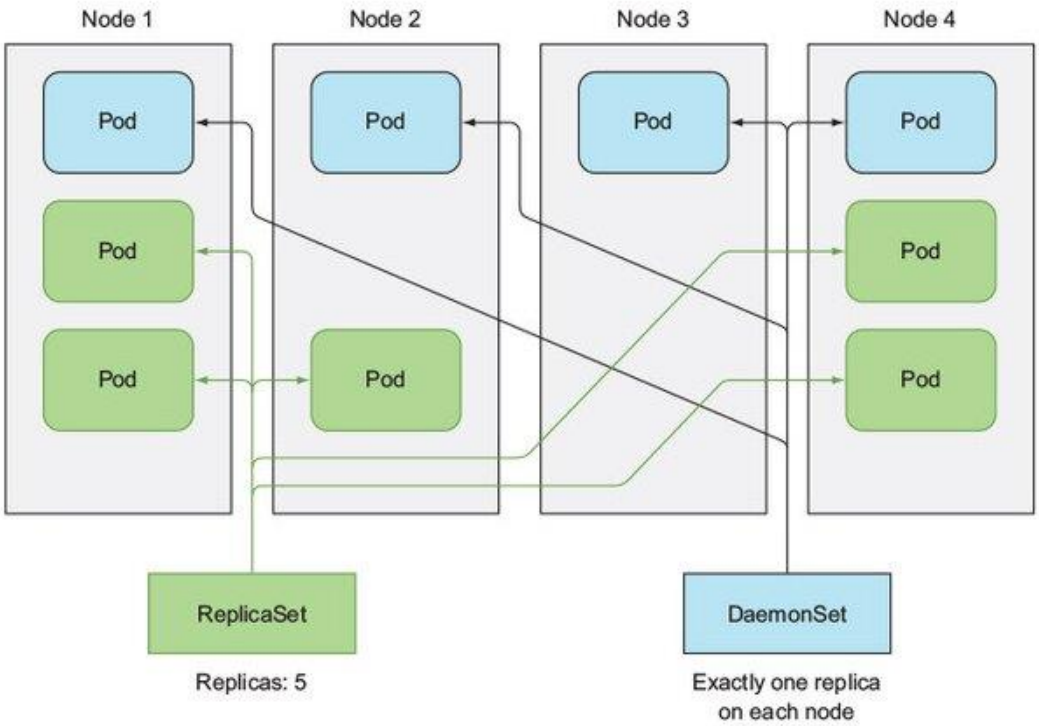
DaemonSet

O **DaemonSet** garante que todos, ou um determinado subconjunto, os nós rodem uma cópia de determinado *pod*. Conforme novos nós são adicionados ao *cluster*, o DaemonSet garante que esse nó terá ao menos uma unidade de determinado *pod*.

Um cuidado que temos que ter é que, ao deletarmos um DaemonSet, deletamos todos os *pods* criados por ele. Por isso, em alguns comandos de deleção usamos a diretiva `--ignore-daemonsets`, para ignorar os *pods* criados pelo Daemon.

ESQUEMATIZANDO

SET	DESCRIÇÃO
ReplicaSet	Garante que determinado número mínimo de *pods* estará disponível, a todo momento
StatefulSet	Gerencia um conjunto de *pods* do tipo *stateful*, que operam quando há necessidade de persistência de dados entre sessões
DaemonSet	Replica ao menos uma unidade de determinado *pod* a todos os nós, ou a um conjunto deles



(FGV/TJDFT/2022) A analista Ana precisa implantar o *pod* Log*Pod* no cluster de Kubernetes KCluster do TJDFT de forma que todos os nós elegíveis do KCluster executem uma cópia do Log*Pod*.

Para que o KCluster apresente uma cópia do Log*Pod* em cada nó elegível, de forma automática, Ana deve implantar o Log*Pod* utilizando o recurso do Kubernetes:

a) ReplicationController;

b) ReplicaSet;
c) StatefulSet;
d) DaemonSet;
e) EndpointSlice.

Comentários:

Como acabamos de ver, o *set* responsável por garantir que tenhamos pelo menos uma cópia de determinado *pod* em cada nó elegível é o DaemonSet. *(Gabarito: Letra D)*

Jobs & CronJobs

Já vimos um pouco, mas vamos aprofundar agora. Os **jobs**, ou **tarefas**, criam um ou mais *pods* e irão continuar a criar e executá-los até que um determinado número de *pods* apresente sucesso na sua execução. Eles são direcionados a executar determinada atividade que tenha um começo e fim bem definidos.

Um YAML configura os jobs também, para variar *rs...* abaixo, deixo um exemplo de um *job* que irá computar o número *pi* para 2000 casas após da vírgula, e printar.

YAML

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34.0
        command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
  backoffLimit: 4 #quantas vezes irá executar após a falha
```

Podemos definir algumas políticas de limpeza para nossas **tarefas**, ou ***jobs***, finalizadas. As tarefas são processos de curta duração, que são completadas após apontar o êxito na sua execução. Usualmente, são trabalhos que não precisam ser aplicados continuamente, e sim de forma periódica ou única.

É uma prática recomendada que os trabalhos sejam mantidos por um pouco mais de tempo após serem finalizados, dentro da API, de forma que possamos dizer se o trabalho foi completado ou falhou.

Para isso, especificamos um TTL – Time To Live – à tarefa a partir da especificação do campo `.spec.ttlSecondsAfterFinish`. Com isso, *pod*emos definir um tempo que a tarefa irá ter de sobrevida, contado a partir do tempo que o trabalho aponta como status o `complete` ou `failed`.

Quando é necessário agendar um trabalho, usamos os **Cronjobs**. Para isso, adicionamos uma tag `schedule: "* * * * *"` no YAML. Os valores devem ser preenchidos, respectivamente:

MINUTO	HORA	DIA DO MÊS	MÊS	DIA DA SEMANA
0-59	0-23	1-31	1-12	0-6

Por exemplo:

```
0 0 13 * 5
```

Executaremos o trabalho toda meia-noite de sexta-feira, além de todo dia 13. O asterisco indica *todos*. É uma estrutura muito similar aos *cronjobs* do Linux. Existem também alguns macros que executam a os Cronjobs de forma mais "legível".

MACRO	DESCRIÇÃO	EQUIVALÊNCIA
@yearly @annualy	Roda uma vez por ano, à meia noite do dia 1 de janeiro	0 0 1 1 *
@monthly	Roda uma vez, à meia noite do primeiro dia do mês	0 0 1 * *
@weekly	Roda uma vez por semana, na manhã de domingo	0 0 * * 0
@daily @midnight	Roda todo dia, à meia noite	0 0 * * *
@hourly	Roda a cada hora, no começo da hora	0 * * * *

(VUNESP/TJM SP/2023) Considere o seguinte arquivo YAML de manifesto de um CronJob do Kubernetes:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: hello
            image: busybox:1.28
            imagePullPolicy: IfNotPresent
            command:
            - /bin/sh
```

```
        - -c
        - date; echo Hello
      restartPolicy: OnFailure
```

Esse CronJob executa uma tarefa

a) a cada minuto.
b) uma vez por dia, às 01:28 horas.
c) uma vez por mês.
d) apenas quando ocorre uma falha no cluster.
e) apenas uma vez, dentro da data retornada pelo comando date do sistema operacional.

Comentários:

O cronjob define o seguinte cronograma: "* * * * *".

Como o asterisco indica a presença de todos, iremos executar todos os minutos, todas as horas, todos os dias da semana, todos os meses e todos os dia do mês – ou seja, iremos executar a tarefa a cada minuto. *(Gabarito: Letra A)*

Rede

Cada *pod* dentro de um cluster tem seu **IP único**, válido única e exclusivamente dentro do ambiente desse próprio *cluster*. Isso significa que não precisamos criar links entre os *pods* e quase nunca precisaremos mapear as portas dos contêineres ao host.

Dessa forma, temos uma organização limpa e clara, de forma a tratar os *pods* quase como máquinas virtuais ou a própria máquina física, da perspectiva de alocação de portas, nomeação, descoberta de serviços, balanceamento de carga, entre outros.

Existem duas **imposições** fundamentais que o K8s faz para todas as implementações de redes:

- *Pods pod*em comunicar-se com todos os outros *pods*, em qualquer outro nó, sem ferramentas de NAT
- Agentes do nó (como daemon, kubelete) *pod*em comunicar-se com todos os nós daquele determinado nó

Os endereços de IP do Kuberentes se aplicam dentro do contexto dos *pods* também – dessa forma, os contêineres dentro de um *pod* compartilham seu *namespace* de rede, incluindo seu endereço IP e MAC. Isso exige uma coordenação para não haver sobreposição de portas. Esse modelo recebe o nome de ***ip-per-pod.***

Temos alguma APIs que fornecem funcionalidades e soluções na rede, vamos estudá-las.

Serviço

No Kubernetes, serviço (**service**) é um método de expor uma aplicação de rede que está rodando em um ou mais *pods* no cluster. O objetivo é que você não precise modificar a aplicação para usar um serviço de descoberto desconhecido.
A aplicação é alocada em um *pod*, independentemente de ter sido desenvolvida para a nuvem, para uso local, ou em um ecossistema mais antigo, e o Service irá fazer com que o conjunto de *pods* que rode sua aplicação esteja disponível para os clientes externos interagirem.

> Imagine a situação – você cria uma aplicação a partir de um *deployment*. Esse objeto *pod*e criar e deletar *pods* dinamicamente. De um momento ao outro, não temos mais controle sobre quais estão saudáveis, quais estão no estado desejado – isso pois os *pods* são recursos efêmeros. Como cada *pod* tem seu IP, isso significa que teremos uma mudança constante, *pod*endo afetar dependências entre diferentes *pods*. Nesse sentido, entram os *services*, para manter um rastreamento dessas mudanças de endereço.

Para definir um serviço, assim como todo objeto no Kubernetes, temos um YAML.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Aqui, o que nos importa são as portas:

- **`protocol`** → representa o protocolo que será usado. No nosso exemplo, o TCP.
- **`port`** → o número da porta no qual o *service* ficará disponível ao mundo externo, para receber chamadas. No nosso exemplo, receberemos chamadas na porta 80.
- **`targetPort`** → número da porta nos *pods* selecionados (ali no *selector*).

Dessa forma, quando recebermos uma solicitação externa, uma chamada HTTP por exemplo, na porta 80, ela será redirecionada para o MyApp, o conjunto de *pods* que representa o aplicativo selecionado no *selector*.

*Pod*emos, também, fazer uma "referência cruzada", definindo um nome para determinada porta, e, ao criarmos o serviço, atribuímos esse nome, ao invés do número.

Temos **4 tipos diferentes de serviços**:

- **ClusterIP:** atribui um *IP interno fixo* ao serviço. Esse IP é acessível apenas de dentro do cluster, o que significa que o serviço é acessível apenas por outros *pods* no mesmo cluster. É útil para comunicação entre diferentes componentes dentro do cluster.
- **NodePort:** expõe o serviço em um *número fixo de porta em cada nó* do cluster. Isso significa que o serviço é acessível de fora do cluster usando o IP de qualquer nó e a porta designada. É útil para expor serviços para o mundo externo, mas não é recomendado para uso em produção devido a questões de segurança e escalabilidade.
- **LoadBalancer:** cria um balanceador de carga externo no provedor de nuvem (se suportado) e atribui um IP externo ao serviço. Esse IP é acessível de fora do cluster e o balanceador de carga distribui o tráfego entre os *pods* do serviço. É útil para expor serviços para o mundo externo com balanceamento de carga.
- **ExternalName:** é um tipo especial de serviço que não cria um endpoint de rede no cluster, mas sim retorna um nome de host externo (como um registro DNS) quando acessado. É útil para referenciar serviços externos ao cluster de forma transparente, sem a necessidade de IPs ou portas específicas.



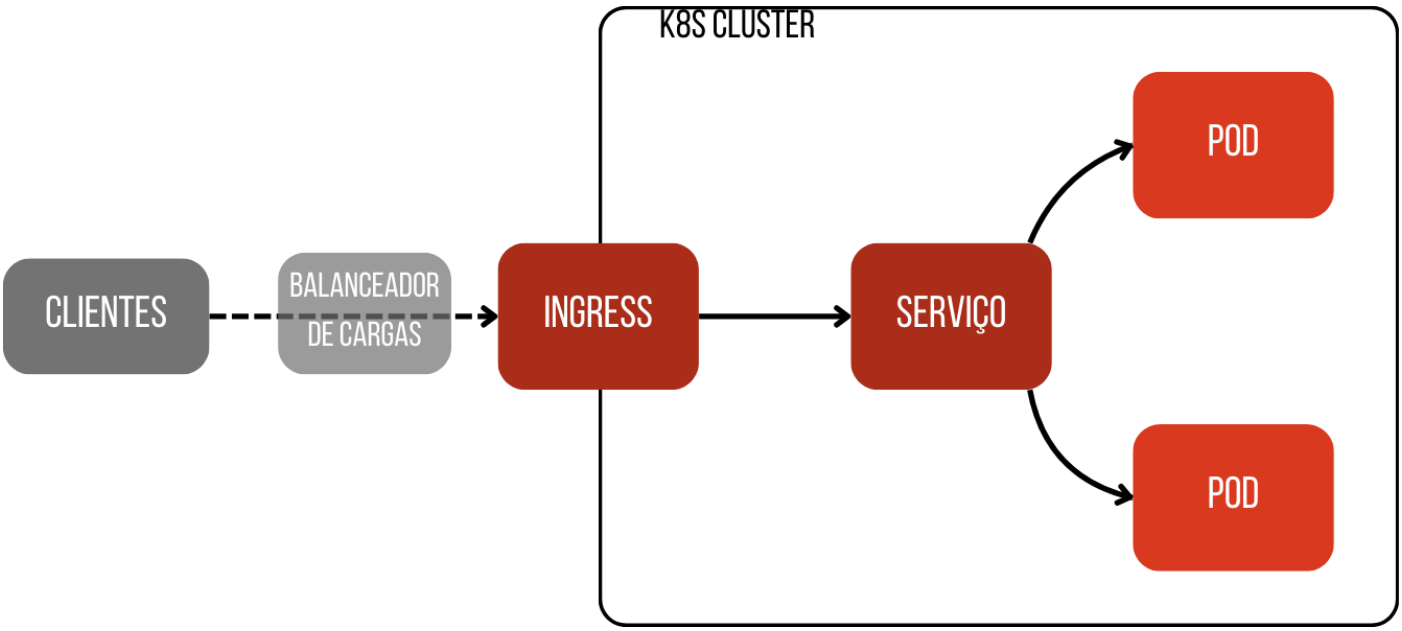
Em alguns casos, não é necessário nem balanceamento de carga, nem endereço de IP. Nesses casos, usamos os *headless services*, atribuindo o valor `none` ao endereço de IP do cluster.

Ingress

O ***Ingress*** (ingresso, numa tradução direta) é um objeto API que gerencia acessos externos a um serviço num *cluster*. Eles fornecem balanceamento de carga, tunelamento seguro (SSL/TLS), *hosting* virtual, entre outros.

De forma geral, o *ingress* é uma porta de entrada, que recebe as requisições HTTP externas ao *cluster* e, através de regras de roteamento, direciona essa requisição para serviços, que, por si, direcionam aos *pods*. Na imagem abaixo ficará mais fácil de entender.





Para implementar o *ingress*, é mandatório que tenhamos um ***ingress controller***. Apenas criar os recursos (YAML) do *ingress* não são suficientes. Usualmente, usamos os *ingress* fornecidos pela Nginx, como o *ingress-nginx*.

*Pod*emos ter alguns tipos de configurações de roteamento no *ingress*:

- **Ingress suportado por um unico service:** Nesse tipo de configuração, o Ingress direciona todo o tráfego para um único Service no cluster. É útil quando você deseja expor um único serviço para o mundo externo.
- **Simple Fanout:** Nesse tipo de configuração, o Ingress direciona o tráfego para diferentes Services com base no caminho da URL. Por exemplo, você *pod*e ter um Service para /app1 e outro Service para /app2, e o Ingress roteará o tráfego com base no caminho da URL.
- **Hospedagem virtual baseada em nome:** Nesse tipo de configuração, o Ingress direciona o tráfego com base no nome do host na solicitação. Por exemplo, você *pod*e ter um Service para app1.example.com e outro Service para app2.example.com, e o Ingress roteará o tráfego com base no nome do host na solicitação.
- **TLS:** O TLS (Transport Layer Security) é usado para criptografar o tráfego entre o cliente e o servidor.No Kubernetes, o Ingress *pod*e ser configurado para suportar TLS, o que permite que você exponha serviços de forma segura pela Internet.
- **Load Balancing:** O Ingress *pod*e ser usado para distribuir o tráfego entre diferentes *pods* de um mesmo Service. Isso é feito de forma transparente pelo Ingress, que *pod*e usar algoritmos de balanceamento de carga, como round-robin, para distribuir o tráfego de forma equilibrada entre os *pods*.

Aqui vale uma pausa para explicar os principais algoritmos de balanceamento:

- **Round Robin (RR):** O algoritmo distribui o tráfego de forma equitativa entre os *pods*, de forma sequencial. Cada nova solicitação é enviada para o próximo *pod* na lista de *pods* disponíveis. É simples e eficaz, mas *pod*e não ser ideal para cargas de trabalho desbalanceadas.

- **Least Connections (LC):** Ele direciona o tráfego para o *pod* com o menor número de conexões ativas no momento. Isso ajuda a distribuir a carga de forma mais equilibrada entre os *pods*, especialmente em situações onde alguns *pods pod*em ter mais conexões abertas que outros.
- **IP Hash:** O IP Hash usa o endereço IP do cliente para determinar para qual *pod* enviar o tráfego. Isso garante que solicitações do mesmo cliente sejam sempre direcionadas para o mesmo *pod*, o que *pod*e ser útil para manter o estado da sessão em aplicações que exigem isso.
- **Weighted Round Robin (WRR):** É semelhante ao Round Robin, mas permite atribuir pesos diferentes a cada *pod*. *Pods* com pesos mais altos recebem uma proporção maior de tráfego, o que *pod*e ser útil para lidar com cargas de trabalho desiguais.
- **Random (RAND):** O Random seleciona aleatoriamente um *pod* para receber o tráfego. Embora simples, *pod*e não ser ideal para cargas de trabalho onde a distribuição uniforme de tráfego é importante.

(FGV/SENADO/2022) Na configuração de uma aplicação que irá utilizar múltiplos contêineres, queremos definir o método de balanceamento adequado, no qual o tempo de resposta de cada servidor é usado para calcular o seu peso.
O nome desse método é

a) menor tempo de início lento de conexão.
b) ponderado fixo.
c) round robin ponderado.
d) round robin.
e) resposta ponderada.

Comentários:

Galera, existem diversos tipos de balanceamento. Nesse caso, estamos dimensionando a alocação de acordo com o tempo de resposta, atribuindo um peso adicional para balanceamento adequado. Nesse sentido, o método é chamado de *resposta ponderada*. *(Gabarito: Letra E)*

Endpoints

Em Kubernetes, os **endpoints** são objetos que representam um conjunto de IPs de *pod*s que realizam uma determinada tarefa. Eles são usados principalmente em conjunto com Services para direcionar o tráfego de rede para os *pod*s corretos.

Quando você cria um Service no Kubernetes, ele cria automaticamente um conjunto de endpoints que correspondem aos *pod*s selecionados pelo Service. Cada endpoint contém o IP e a porta de um *pod*, permitindo que o Service direcione o tráfego para esses *pod*s.

Nesse contexto, temos o **EndpointSlice**, uma API que referencia um conjunto de *endpoints* da rede. O plano de controle automaticamente cria essa API para qualquer serviço que tenha algum seletor especificado. Por padrão, cada Slice controla até 100 *endpoints*, *pod*endo ser configurado para chegar a um máximo de 1.000 *endpoints*.
Como comentei pouco antes a vocês, a efemeridade dos *pods* é um problema quando temos dependências, já que constantemente temos mudanças de endereços, ao extinguir e criar um novo *pod*. O EndpointSlice entra com o objetivo de fazer esse mapeamento dinâmico.

A grande vantagem do Slice é a garantia de uma maior granularidade – conseguimos controlar cada *endpoint*, sem a necessidade de fazer um agrupamento deles para ter o controle. Assim, cada *pod pod*e ser representado como um *endpoint* individual garantindo maior eficiência na escalabilidade da aplicação.

Alocação de Pods

Kubernetes Scheduler

No contexto do Kubernetes, ***scheduling*** ou agendar, significa assegurar que os *pods* estão alocados no nó correto, para que o `kubelet` consiga rodar neles. Um gerenciador de agendamento – chamado de ***kube-scheduler*** – observa novos *pods* criados que não possuem um nó designado, se tornando responsável por encontrar o nó mais adequado a esse *pod*. Quanto aos nós, aqueles que se adequam ao *pod* são chamados de ***feaseble***, algo como "viável", ou "possível".

A seleção do `kube-scheduler` é realizada em duas etapas:

- **Filtragem** – os passos de filtragem focam em encontrar os nós que são compatíveis com o *pod*, aplicando uma série de filtros. Por exemplo, o filtro `*Pod*FitsResources` checa se o nó tem os recursos necessários para armazenar o *pod*.
- **Pontuação** – com os nós compatíveis separados, é feita uma classificação para encontrar o mais compatível dentre eles. Uma pontuação é atribuída a cada um deles, e o com a maior pontuação é escolhido.

*Pod*emos configurar a filtragem e a pontuação de duas formas:

- Políticas de Agendamento – são configurados *predicados* para filtragem, e *prioridades* para a pontuação
- Perfis de Agendamento – são implementados *plugins* para fazer o agendamento em diferentes etapas, como `QueueSort`, `Filter`, `Bind`, entre outros.

Alocação de *pods*

*Pod*emos restringir um *pod* a um nó específico, ou definir preferências para ele rodar em um conjunto de nós. A abordagem mais recomendada é pelo uso de seletores de etiquetas (*label selectors*). Porém, *pod*em ter vezes que necessitamos de um refinamento ainda maior – por exemplo, escolher nós que possuam SSD assignado, que permitam acesso externo, entre outros. Nesses casos, temos quatro abordagens:

- nodeSelector
- Afinidade
- nodeName
- Restrições de topologia dos *pods*

02648216103 - HENRIQUE AMORIM BARRETO

ALOCAÇÃO DE PODS

NODESELECTOR | AFINIDADE | NODENAME | RESTRIÇÕES

NodeSelector

O `nodeSelector` é a forma mais simples de seleção de restrições. Adicionamos a tag no documento YAML de especificação do *pod* e especificamos as **etiquetas de nós** que queremos que recebam esses nós. Nesse caso, o *scheduler* irá analisar somente esses nós delimitados para definir a alocação dos *pods*.

Afinidade

A afinidade expande as possíveis restrições do seletor de nós, fornecendo uma linguagem mais expressiva e com mais controle sobre a lógica de seleção.

Por exemplo, *pod*emos identificar determinada regra como secundária, usando a tag *soft*, ou como primária, com a tag *preferred*. Dessa forma, *pod*emos o *scheduller* encontrará um nó mesmo que não tenha encontrado um 100% compatível.

A **afinidade**, ou ***node affinity***, serve para delimitarmos nós que queremos a alocação, de acordo com alguns parâmetros. Temos dois tipos de afinidade:

- `requiredDuringSchedulingIgoredDuringExecution` : o *scheduler* não *pod*e agendar o *pod* caso os parâmetros definidos não tenham compatibilidade
- `preferredDuringSchedulingIgoredDuringExecution` : o *scheduler* irá tentar encontrar algum nó que atenda aos parâmetros. Caso não encontre, será alocado em nós que cheguem próximo, medindo a afinidade.

A afinidade é definida no campo *spec* do YAML do *pod*. Veja:

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: topology.kubernetes.io/zone
            operator: In
            values:
            - antarctica-east1
            - antarctica-west1
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: another-node-label-key
            operator: In
            values:
            - another-node-label-value
```

Acima, por exemplo, temos as seguintes regras:

- O nó **deve** ter uma etiqueta com a chave `topology.kubernetes.io/zone`, e o valor dessa etiquete **deve** ser ou `antartica-east1` ou `antartica-west1`.
- O nó **preferivelmente** deve ter um etiqueta com a chave `another-node-label-key` e o valor `another-node-label-value`.

*Pod*emos também definir um peso, entre 1 e 100, para cada instância do *preferred*. Quando o *scheduler* encontrar afinidades, ele irá iterar por cada uma dessas preferências, atribuindo os pesos e as classificando.



(FGV/SEN/2022) Uma instituição acadêmica está desenvolvendo um ambiente de laboratório virtual em kubernetes, com alta disponibilidade. O sistema provisiona automaticamente um novo *pod* quando um aluno se autentica.

Um dos pré-requisitos do projeto é isolar os *pod*s dos alunos em um pool de nodes exclusivo em alta disponibilidade.

Assinale a opção que indica a funcionalidade do kubernetes adequada a esse pré-requisito.

a) Ingress.
b) Node affinity.
c) Limit Ranges.
d) *Pod* Overhead.
e) Node-pressure Eviction.

Comentários:

Temos uma situação de alocação de *nodes*. A questão fica fácil já que o único mecanismo de alocação nas alternativas é o *node affinity*. Porém, vamos entender o que acontece.

Sempre que um aluno faz o *log-in*, temos a criação de um *pod* específico para lidar com essa sessão do aluno. Precisamos designar esse nó para algum nó que forneça características de segurança e autenticação, como o uso de um *ingress*, por exemplo. Isso *pod*e ser feito de diversas formas – sendo a mais adequada a **afinidade**. Aqui, caso não encontremos uma compatibilidade 100% com o que queremos, conseguimos definir casos residuais que permitirão o *pod* a ser alocado em algum nó. (Gabarito: Letra B)

nodeName

O `nodeName` é uma forma mais direta de seleção de nós no seletor. Aqui, basicamente definimos o nome do nó e o *pod* é alocado a ele. Se o nó não existir, o *pod* não irá rodar e, em alguns casos, *pod*e ser deletado automaticamente.

Restrições de disseminação de tipologia

Restrições de disseminação de topologia de *pod*s, ou ***pod topology spread constraints***, são uma forma de especificar políticas no Kubernetes para garantir que os *pod*s de uma determinada aplicação sejam distribuídos de forma equilibrada e resiliente em relação à topologia do cluster. Isso é especialmente útil para garantir a alta disponibilidade e a tolerância a falhas em ambientes de vários nós.

Essas restrições permitem que você defina **regras sobre como os *pod*s de uma aplicação devem ser distribuídos** em relação a fatores como zonas de disponibilidade, hosts ou regiões do cluster. Por exemplo, você *pod*e especificar que os *pod*s devem ser distribuídos de forma que não mais de um determinado número deles seja colocado em um único nó ou em uma única zona de disponibilidade.

Isso ajuda a evitar a concentração de *pod*s em um único ponto de falha e melhora a disponibilidade da aplicação, distribuindo os *pod*s de forma mais equilibrada e resiliente em relação à topologia do cluster.

Pod Overhead

Em tecnologia da informação, o termo ***overhead*** se refere aos **custos adicionais ou recursos consumidos além do mínimo necessário para realizar uma determinada tarefa**. Esse termo é frequentemente usado para descrever a quantidade de recursos, como tempo, memória, largura de banda ou processamento, que são utilizados para suportar uma operação ou processo, mas que não estão diretamente relacionados à sua função principal.

Por exemplo, em virtualização, o overhead se refere aos recursos extras necessários para gerenciar e executar as máquinas virtuais, como a camada de virtualização e a gestão dos recursos compartilhados. Em redes de computadores, overhead pode se referir à quantidade de dados adicionais necessários para encapsular e transmitir os dados úteis, como os cabeçalhos de pacotes em protocolos de rede.

Nesse sentido, temos o ***pod overhead***. Quando rodamos um *pod* em um nó, esse *pod* consome uma determinada quantidade de recursos. Esses recursos **são adicionados** aos recursos necessários para rodar os contêineres dentro do *pod*. Então, digamos que o contêiner consuma 250mb de memória, e o *pod* mais 50mb – o *overhead* são esses 50mb a mais para rodar a infraestrutura do *pod*.

Podemos controlar esses parâmetros a partir de uma classe chamada `RuntimeClass` e, dentro dela, definindo o campo `overhead`. As *workloads* que forem criadas especificando a determinada classe irão levar esse *overhead* em conta para calcular as dinâmicas de carga.

Veja um exemplo:

YAML

```yaml
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: kata-fc
handler: kata-fc
overhead:
  podFixed:
    memory: "120Mi"
    cpu: "250m"
```

Quando formos criar um pod, por exemplo, referenciaremos dentro das `spec`, o campo referente à classe, como `runtimeClassName: kata-fc`.

Prioridade e Perempção

Cada *pod* pode ter sua prioridade. A prioridade indica a importância de determinado *pod* em relação a outros. Se determinado *pod* não é agendado, o *scheduler* tenta fazer a perempção (ou evicção) dos *pods* com menor prioridade, para fazer o agendamento daqueles com maior prioridade possível.

Para fazer uso desse mecanismo, usamos o uso das **classes de prioridade**, e, posteriormente, adicionamos elas na tag `priorityClassName` dentro do *template* do *pod.*

Abaixo, um exemplo de classe de prioridade:

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
```

Podemos evitar a perempção da classe, adicionando o campo `peremptionPolicy`, e atribuindo o valor `never`.

Evicção por pressão

A evicção por pressão dos nós é um processo em que o `kubelet` proativamente encerra os *pods* no nó para reclamar os recursos, por se tornarem escassos. Os recursos, como memória e espaço de armazenamento, são monitorados e, quando um desses recursos fica acima dos limites, o `kubelet` termina proativamente alguns dos *pods*. Podemos definir **sinais** e **limites** para a evicção ocorrer.

Os **sinais** são estados atuais de determinado recurso, em um determinado ponto do tempo. O `kubelet` irá fazer comparações frequentes entre os sinais e os limites definidos para decidir se deve ou não encerrar determinados *pods*.

Alguns sinais de evicção são:

- memory.available → demonstra a memória disponível
- nodefs.available → *fs* é uma abreviação de *file system*. Esse comando demonstra a disponibilidade de armazenamento no sistema de arquivos do nó

O `kubelet` lê informações de dois sistemas de arquivos específicos (*fyle system*):

- `nodefs` → sistema de arquivos do node
- `imagefs` → sistema de arquivos opcionais, relativo aos *container runtime*, usado para armazenar as imagens e as camadas de contêineres

Com isso, sobrepomos algumas ferramentas dos coletores de lixo com as *flags* de evicção. As principais flags que temos no Kubernetes são:

- `--eviction-hard` : Esta flag define a política de remoção de pods com base em limites rígidos. Quando ocorre pressão no nó, os pods que excedem os limites definidos por essa flag podem ser removidos.
- `--eviction-soft` : Esta flag define a política de remoção de pods com base em limites suaves. Durante um período de graça especificado, os pods podem exceder os limites sem serem removidos imediatamente, permitindo que o nó se recupere antes de tomar medidas mais drásticas.

- --maximum-dead-containers: Esta flag define o número máximo de containers mortos que são tolerados antes que um pod seja removido. Se um pod atingir ou exceder esse limite, ele será marcado para remoção.

Podemos modificar também essas configurações diretamente no YAML do kubelete.

YAML

```yaml
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
evictionHard:
  memory.available: "500Mi"
  nodefs.available: "1Gi"
  imagefs.available: "100Gi"
evictionMinimumReclaim:
  memory.available: "0Mi"
  nodefs.available: "500Mi"
  imagefs.available: "2Gi"
```

RESUMO

O QUE É O KUBERNETES?

O Kubernetes é uma plataforma de código aberto para automatização, escalonamento e gerenciamento de aplicativos em contêineres, fazendo a orquestração dos contêineres.

QUAL A ESTRUTURA DO KUBERNETES?

A estrutura do Kubernetes inclui "nós" (nodes), que são as máquinas físicas ou virtuais onde os contêineres são executados; "cluster", que é um conjunto de nós que funciona como uma única unidade; e "pods", que são grupos de um ou mais contêineres que compartilham recursos.

O QUE SÃO OBJETOS NO KUBERNETES?

Em Kubernetes, os objetos são elementos persistentes, que compreendem representações de estado do cluster, como contêineres, pods, serviços, volumes, etc. Todos os objetos são configurados a partir de um documento chamado manifesto, escrito em YAML ou JSON.

O QUE É UM POD?

Um "pod" é a menor unidade no Kubernetes que pode ser criada, programada e gerenciada. Um pod geralmente contém um ou mais contêineres que são executados juntos, compartilhando o mesmo contexto, recursos e rede.

O QUE É O KUBELET? E O KUBECTL?

O "kubelet" é um agente que roda em cada nó do cluster Kubernetes. Ele é responsável por garantir que os contêineres estejam sendo executados em um pod. Já o "kubectl" é uma ferramenta de linha de comando usada para interagir com clusters Kubernetes. Ele permite que os usuários executem comandos contra clusters Kubernetes para criar, inspecionar, atualizar e remover recursos.

QUAIS OS 3 “SETS” DO KUBERNETES?

Os três conjuntos principais de controladores, ou “sets”, no Kubernetes são o "ReplicaSet", que garante que um número especificado de réplicas de um pod esteja em execução; o "StatefulSet", que é usado para aplicativos que necessitam de identidades persistentes e ordem de inicialização específica; e o "DaemonSet", que garante que todos os nós ou um subconjunto de nós execute uma cópia de um pod, útil para tarefas de infraestrutura como monitoramento de logs ou armazenamento distribuído.

QUESTÕES COMENTADAS

01. (FGV/SEF-MG/2023) Kubernetes é um sistema de orquestração de contêineres *open-source* que automatiza a implantação, o dimensionamento e a gestão de aplicações em contêineres.

Em relação ao conceito de Kubernetes, assinale a afirmativa **incorreta**.

a) Kube-proxy é um *proxy* de rede executado e mantém regras de rede em cada máquina do cluster.
b) Kubernetes utilizam controladores que rastreiam pelo menos um tipo de recurso Kubernetes.
c) Kubernetes utiliza *contexts* como mecanismo para isolar grupos de recursos dentro de um único *cluster*.
d) Kubelet é um agente que é executado em cada máquina do *cluster*; ele garante que os contêineres estejam sendo executados em um Pod.
e) Os objetos do Kubernetes são entidades persistentes no Kubernetes e utilizam estas entidades para representar o estado do *cluster*.

Comentários:

Vamos analisar cada uma das alternativas.

a) Certo. O kube-proxy é um componente do Kubernetes responsável por gerenciar o tráfego de rede para os serviços que estão em execução no cluster. Ele opera a nível de cada nó do cluster e realiza várias funções importantes relacionadas à rede
b) Certo. Os controladores são responsáveis por rastrear e monitorar os recursos dentro do Kubernetes.
c) Errado. São usados ***namespaces*** para isolar grupos de recursos dentro de um cluster.
d) Certo. O *kubelet* é um agente presente em cada nó do *cluster*, que faz a comunicação com o plano de controle garantindo que tudo esteja "ok".
e) Certo. Os objetos do Kubenetes são, de fato, entidades persistentes, que recebem definições em arquivos YAML, e representam o estado do cluster.

Sendo assim, incorreta a letra C.

Gabarito: Letra C

---

02. (FCC/MPE PB/2023) Considere que a equipe de TI do Ministério Público utiliza o Kubernetes como ferramenta de controle de clusters e neste ambiente também encontra-se o Kubectl, uma ferramenta de linha de comando para gerenciar objetos e clusters Kubernetes, instalados e funcionando em condições ideais. Um Analista foi solicitado a apresentar os comandos para as seguintes questões relativas ao Kubectl:

I. Verificar se o Kubectl pode autenticar e acessar o cluster Kubernetes de um usuário.
II. Visualizar como está a configuração do Kubectl.
III. Buscar a lista de clusters definidos no kubeconfig.

Os comandos para realizar as tarefas I, II e III são, correta e respectivamente,

a) $ kubectl kubernetes-info / $ kubectl config_view / $ kubectl get-clusters

02648216103 - HENRIQUE AMORIM BARRETO

b) $ kubectl cluster-info / $ kubectl view_config / $ kubectl config list-clusters
c) $ kubectl kubectl-info / $ kubectl config view / $ kubectl list-clusters
d) $ kubectl kubernetes-info / $ kubectl view_config / $ kubectl get-clusters
e) $ kubectl cluster-info / $ kubectl config view / $ kubectl config get-clusters

Comentários:

Ao contrário do Docker, questões de Kubernetes costumam cobrar mais a estrutura que os comandos em si. Mas vamos lá, encontrar os comandos

I. Verificar se o Kubectl pode autenticar e acessar o cluster Kubernetes de um usuário.
Para isso, precisamos de informações acerca do cluster. O comando é `kubectl cluster-info`.
II. Visualizar como está a configuração do Kubectl.
Aqui, como já falamos diretamente com o kubectl quando digitamos os códigos, podemos ir diretamente para `kubectl view-config`.
III. Buscar a lista de clusters definidos no kubeconfig.
Para buscar (retornar) algum elemento, como o *select* do SQL, usamos o comando *get*. Nesse caso, usamos `kubectl config get-clusters`.

Sendo assim, a afirmativa que aponta todos os comandos corretos é a letra E.

Gabarito: Letra E

---

03. (FEPESE/PREF. B. CAMBORIÚ/2023) Assinale a alternativa que descreve corretamente o conceito de Serviço do Kubernetes.

a) A entrega de pods como instâncias de processos à aplicação.
b) A orquestração de containers ao longo de todo o cluster, com gerenciamento automatizado.
c) Uma forma de expor uma aplicação rodando sobre um conjunto de pods como um serviço de rede.
d) A distribuição de pods pelos nós do cluster, implementando um balanceamento de carga pró-ativo.
e) É o componente que monitora a existência de Pods recentemente criados sem nó designado, e lhes atribui um nó para que sejam executados.

Comentários:

Os serviços, no Kubernetes, são formas de expor aplicações para os serviços de rede, manuseando as mudanças de endereço IP e a criação/extinção dos contêineres. Nesse sentido, correto o apontamento da letra C.

Gabarito: Letra C

---

04. (FEPESE/PREF. B. CAMBORIÚ/2023) Analise as afirmativas abaixo sobre Kubernetes.

1. O Controle Plane é responsável por gerenciar os principais componentes e garantir que o funcionamento esteja de acordo com o estado almejado da aplicação.
2. O etcd é um repositório de chave-valor consistente e de alta disponibilidade.

3. O servidor de APIs do Kubernetes é denominado kube-container.

Assinale a alternativa que indica todas as afirmativas corretas.

a) É correta apenas a afirmativa 1.
b) São corretas apenas as afirmativas 1 e 2.
c) São corretas apenas as afirmativas 1 e 3.
d) São corretas apenas as afirmativas 2 e 3.
e) São corretas as afirmativas 1, 2 e 3.

Comentários:

Vamos analisar cada item.

1. Certo! O plano de controle gerencia toda a parte de controle do Kubernetes, a partir dos nós master.
2. Certo! O etcd é, de fato, o repositório de dados do Kubernetes no padrão chave-valor.
3. Errado! O servidor de APIs do Kubernetes é o kube-apiserver.

Portanto, corretos os itens 1 e 2.

Gabarito: Letra B

---

05. (AOCP/IFMA/2023) No Kubernetes, qual é a abordagem correta para implementar comunicação segura entre os componentes do cluster e garantir autenticação e autorização adequadas?

a) Desabilitar o controle de acesso baseado em função (RBAC) e permitir o acesso a todos os usuários e serviços.
b) Utilizar certificados autoassinados para todos os componentes do cluster e ignorar a validação de certificados.
c) Implementar políticas de segurança usando o etcd como armazenamento centralizado de credenciais e chaves de criptografia.
d) Configurar o Kubernetes para utilizar criptografia TLS, certificados válidos e controle de acesso baseado em função (RBAC).
e) Utilizar somente redes não criptografadas entre os componentes do cluster para melhorar o desempenho.

Comentários:

Vamos julgar cada afirmativa.

a) Errado. O Kubernetes, de fato, recomendo o uso de RBAC. Porém, o acesso não é liberado a todos os usuários, nem a todos os serviços.
b) Errado. Não é recomendado o uso de autoassinatura.
c) Errado. O etcd armazena os arquivos YAML para as configurações de cluster, por ser um banco de dados chave-valor. Porém, ele é um sistema distribuído, e a recomendação é que se use sistemas centralizados para armazenamento de ferramentas de segurança.
d) Certo. Todas essas práticas são recomendadas pelo Kubernetes.

e) Errado. O ideal é usar redes criptografadas, sempre que possível.

Sendo assim, correta a letra D.

Gabarito: Letra D

---

06. (VUNESP/TJM SP/2023) Em um arquivo YAML de definição de um objeto do Kubernetes, a descrição do estado desejado desse objeto, aplicável à grande parte dos tipos de objetos, é dada pelo campo

a) namespace.
b) metadata.
c) status.
d) desired.
e) spec.

Comentários:

As definições específicas de determinado objeto, como deployment e pods, é feita dentro do campo `spec`, no arquivo YAML.

Gabarito: Letra E

---

07. (FGV/SEFAZ AM/2022) Suponha a entrega de um laboratório (SaaS) como microsserviços na nuvem, onde cada aluno terá um espaço dedicado de memória, processamento, e poderá abrir uma interface Web e rodar códigos em Python. O espaço individual de cada aluno será criado a partir de uma mesma imagem.

Assinale a opção que apresenta a plataforma de entrega na nuvem para atender a essa demanda.

a) Big Query.
b) Azure Cosmos DB.
c) Azure Kubernetes.
d) Amazon Simple Storage Service.
e) Amazon Quantum Solutions Lab.

Comentários:

Apesar de ser uma questão sobre AWS, o conceito não é novo para vocês. Quando estamos trabalhando com *imagens*, estamos operacionalizando contêineres. No ecossistema da AWS, quem faz isso é o Azure Kubernetes – que nada mais é do que uma implantação do Kubernetes na nuvem.

Gabarito: Letra C

---

08. (FCC/TRT 4/2022) Considere o seguinte texto:

Nowadays, firewalls are way more sophisticated than they used to be. They are now capable of inspecting the application-level protocols and are often not only deployed at the forefront of the infrastructure but also between each component, to monitor and limit unauthorized accesses.

For the same reason, some container orchestrator tools (such as ....I.... , but also the public cloud providers) offer the possibility to implement the so-called network policies, which are essentially ....II.... acting as a network firewall, hence not allowing (or dropping) unwanted network connections. Firewalls can be hardware appliances (with major vendors including Cisco and Check Point, among others), or even software distributions (such as PFSense and Zeroshell).

(Adaptado de: BONOCORE, Giuseppe. Hands-On Software Architecture with Java. Packt Publishing, 2022)

As lacunas **I** e **II** devem ser preenchidas, correta e respectivamente, com

a) *Kubernetes* e *Access Control Lists*
b) *Docker* e *Reverse Proxies*
c) *Rancher* e *Disaster Recovery Policies*
d) *Jenkins* e *Reverse Proxies*
e) Maven e *Access Control Lists*

Comentários:

Ótima questão para praticar o seu inglês rs

Para o item I, queremos a escolha de um orquestrador de contêineres. Dentre as alternativas apresentadas, só temos um orquestrador de contêineres: Kubernetes. O Docker você já conhece; Maven e Jenkins são ferramentas de automação; Rancher faz o gerenciamento de contêineres dentro de ambientes Kubernetes.

Sendo assim, a alternativa que aponta corretamente os itens I e II é a letra A.

Gabarito: Letra A

---

09. (IDECAN/UNIALB/2022) Compreender como o Kubernetes gerencia os recursos é fundamental para criar e executar seu cluster corretamente. Analise as afirmativas a seguir sobre Kubernets:

I. O Kubernetes aloca recursos de CPU e memória para contêineres com base em solicitações e limites.
II. As solicitações de um contêiner são as quantidades mínimas de recursos que ele precisa para executar. Seus limites especificam a quantidade máxima que pode ser usada.
III. Imagens de contêiner mínimas são mais rápidas de construir, enviar, implantar e iniciar. Quanto maior o contêiner, menores são as vulnerabilidades de segurança em potencial.

Assinale

a) se somente o item I estiver correto.
b) se somente o itens I e II estiverem corretos.
c) se somente o itens II e III estiverem corretos.
d) se todos os itens estiverem corretos.

Comentários:

Vamos ver cada um dos itens.

I. Certo. Definimos, dentro da diretiva *resources*, recursos mínimos solicitados (*request*) e limites máximos (*limit*).
II. Certo. Vide item I.
III. Errado. Pelo contrário, quanto maior o contêiner, maior as vulnerabilidades potenciais. Devemos tentar sempre manter o contêiner com o menor tamanho possível.

Sendo assim, corretos os itens I e II.

Gabarito: Letra B

---

10. (FCC/TRT 22/2022) No Kubernetes, os contêineres gerenciados pelo kubelet podem usar a estrutura de hook do ciclo de vida do contêiner para executar código acionado por eventos durante o gerenciamento de seu ciclo de vida. Há dois tipos de hooks:

I. Este hook é executado imediatamente após um contêiner ser criado. Mas não há garantia de que o hook será executado antes do ENTRYPOINT do contêiner. Nenhum parâmetro é passado para o handler.
II. Esse hook é chamado imediatamente antes de um contêiner ser terminated devido a uma solicitação de API ou um gerenciamento de evento como liveness/startup probe failure, preemption, resource contention e outros. Uma chamada a este hook falha se o contêiner já está em um estado terminated ou completed e o hook deve ser concluído antes que o sinal TERM seja enviado para parar o contêiner. Nenhum parâmetro é passado para o handler.

Os hooks I e II são, correta e respectivamente,

a) PostStart e PreStop.
b) HookStart e HookStop.
c) PodStart e PodStop.
d) KubeStart e KubeStop.
e) KubeletStart e KubeletStop.

Comentários:

Os hooks do ciclo de vida são executados de acordo com o ponto do ciclo. Com isso em mente, temos:

I. Um hook executado logo após o contêiner ser criado é um hook PostStart.
II. Um hook executado logo antes do contêiner ser encerrado é um hook PreStop.

Sendo assim, correta a letra A.

Gabarito: Letra A

---

11. (COCP/IF MT/2022) Considere as seguintes afirmações:

I - DevOps é uma cultura para organização e monitoramento do desenvolvimento de softwares para entrega contínua de resultados.
II - Kubernetes é uma técnica que utiliza contêineres para virtualização de serviços e recursos computacionais a nível de sistema operacional.

III - Docker é uma plataforma de orquestração e automatização de contêineres, conforme site oficial (https:// *www.docker.com/*).

Assinale a alternativa CORRETA.

a) Apenas a afirmação I está correta.
b) As afirmações I e II estão corretas.
c) Apenas a afirmação II está correta.
d) As afirmações II e III estão corretas.
e) As afirmações I, II e III estão corretas.

Comentários:

Ótima questão, que aborda tudo que vimos até agora no módulo.

I. Certo. DevOps é uma técnica que mistura desenvolvimento e operações, buscando entregas e integrações contínuas de aplicações.
II. Errado. Kubernetes é uma ferramenta, responsável pela orquestração de contêinres.
III. Errado. Docker é uma ferramenta para criação de contêineres, não orquestração.

Sendo assim, correto apenas o item I.

Gabarito: Letra A

---

12. (FUMARC/TRT 3/2022) Analise as afirmativas a seguir referentes a Kubernetes:

I – Kubernetes é uma plataforma de código aberto, portável e extensiva para o gerenciamento de cargas de trabalho e serviços distribuídos em contêineres.
II – Kubernetes é um sistema PaaS (plataforma como serviço) monolítico que opera no nível do hardware.
III – Um cluster Kubernetes consiste em um conjunto de servidores de processamento, chamados nós, que executam aplicações containerizadas.

Está CORRETO o que se afirma em:

a) I e II, apenas.
b) I e III, apenas.
c) II e III, apenas.
d) III, apenas.
e) I, II e III.

Comentários:

Vamos analisar cada item.

I. Certo. Descreve perfeitamente o Kubernetes.
II. Errado. O Kubernetes se classifica como um FaaS (veremos sobre isso no final do módulo), distribuído e opera em nível de software.

III. Certo. O cluster Kubernetes é um conjunto de nós, que executam os pods – compostos por aplicações containerizadas.

Corretos, portanto, os itens I e III.

Gabarito: Letra B

---

13. (CEBRASPE/TRT 8/2022) Entre as funcionalidades do Kubernetes, a que gerencia os contêineres quanto à verificação de integridade definida pelo usuário é

a) gerenciamento de configuração e de segredos.
b) orquestração de armazenamento.
c) lançamentos e reversões automatizadas.
d) empacotamento binário automático.
e) Autocorreção

Comentários:

No Kubernetes, a funcionalidade que reinicia os contêineres que falham, substitui os contêineres, elimina os contêineres que não respondem à verificação de integridade definida pelo usuário e não os anuncia aos clientes até que estejam prontos para servir, gerenciada pelo kubelet, é a **autocorreção**.

Gabarito: Letra E

---

14. (CEBRASPE/SERPRO/2021) A respeito do Kubernetes, julgue o item a seguir.

A camada de gerenciamento possui o componente etcd, cuja função é observar pods que foram criados sem nenhum node atribuído e selecionar um node para execução.

Comentários:

A camada de gerenciamento, ou controle, de fato conta com o etcd. Porém, o etcd é um repositório de dados do tipo chave-valor, armazenando as definições dos objetos do Kuberenetes. Ele não faz nenhuma observação nem nada do tipo.

Gabarito: Errado

---

15. (IADES/BRB/2021) Kubernetes é uma plataforma de código aberto, portável e extensiva para o gerenciamento de cargas de trabalho e serviços distribuídos em contêineres, que facilita tanto a configuração declarativa quanto a automação. Ele possui um ecossistema grande e de rápido crescimento. Serviços, suporte e ferramentas para Kubernetes estão amplamente disponíveis.
Disponível em: <https://kubernetes.io/pt-br/docs/concepts/overview/>. Acesso em: 25 jun. 2021, com adaptações.
Com base no texto apresentado e considerando o contexto do Kubernetes, assinale a alternativa que corresponde à ferramenta disponibilizada para realizar operações nos clusters Kubernetes, por meio de interface de linha de comando, pela qual é possível realizar a implantação de aplicações, inspecionar e gerenciar recursos do cluster e visualizar logs.

a) Kubeadm
b) Minikube
c) Kubectl
d) Kind
e) Kubelet

Comentários:

Galera, apesar do textão, a questão quer uma coisa bem simples: a ferramenta de interface de linha de comando (CLI) do Kubernetes. A essa altura já deve ser fácil pra você responder isso né?
A CLI do Kubernetes se chama `kubectl`.

Gabarito: Letra C

---

16. (CEBRASPE/SEFAZ CE/2021) Julgue o próximo item, relativo ao Apache Kafka e ao Kubernetes.

No Kubernetes, kubelet é uma pequena aplicação localizada em um nó que se comunica com o plano de controle, assegurando que os containers estejam em execução em um pod, que consiste no menor e mais simples objeto do Kubernetes.

Comentários:

Todas as definições estão certas. O Kubelet é um agente presente em cada nó do nosso Cluster K8s, e é reponsável por garantir a integridade dos *pods*, comunicando-se diretamente com o plano de controle.

Gabarito: Certo

---

17. (IUDS/IF RJ/2021) Analise as afirmações a seguir sobre Kubernetes:

I - Pod: um grupo de um ou mais containers implantados, em um único nó. Todos os containers em um pod compartilham o mesmo endereço IP, IPC, nome do host e outros recursos. Os pods separam a rede e o armazenamento do container subjacente. Isso facilita a movimentação dos containers pelo cluster.
II - Serviço: desacopla as definições de trabalho dos pods. Os proxies de serviço do Kubernetes, automaticamente, levam as solicitações de serviço para o pod correto, independentemente do local do pod no cluster ou se foi substituído.
III - kubectl: um serviço executado nos nós que lê os manifestos do container e garante que os containers definidos foram iniciados e estão em execução.

Estão corretas as afirmações:

a) I e II.
b) I e III.
c) II e III.
d) I, II e III.

Comentários:

Vamos identificar os itens corretos.

I. Certo. Perfeita a descrição dos *pods*.
II. Certo. Os serviços são formas de expor a aplicação à rede, levando as solicitações diretamente para o *pod* correto, fazendo uma abstração da rede interna.
III. Errado. O kubectl é o CLI do Kubernetes.

Corretos, portanto, os itens I e II.

Gabarito: Letra A

---

18. (FGV/TJ RO/2021) A equipe de desenvolvimento de sistemas de um tribunal de contas está guiando a implantação de um Webservice REST.

A implantação será dividida nos seguintes grupos distintos de containers Docker:
- Grupo A: responsável pela execução da aplicação do Webservice REST
- Grupo B: responsável pela execução do Sistema Gerenciador de Banco de Dados utilizado pelo Webservice REST

Os Grupos A e B terão seu próprio contexto de armazenamento e rede a serem orquestrados por um cluster de Kubernetes.

Para que a conexão de rede entre os containers dos Grupos A e B seja bem-sucedida pelo orquestrador, independentemente dos endereços IP a eles atribuídos, deverá ser configurado um novo:

a) pod;
b) replicaSet;
c) service;
d) ingress;
e) statefulSet.

Comentários:

Galera, para termos uma conexão de rede correta, precisamos abstrair de mudanças que ocorrem dentro da rede específica do nó e dos *pods*. Para isso, implementamos os *services*, que carregam as solicitações ao destino fazendo um mapeamento dinâmico dos endereços.

Gabarito: Letra C

---

19. (FGV/FUNSAÚDE CE/2021) Pod é uma unidade atômica de escalonamento, implantação e isolamento na execução de um grupo de contêineres no Kubernetes, analise as afirmativas a seguir.

I. Um Pod garante uma mesma localização para os seus contêineres, graças a isso eles têm diversas formas de interagir com bom desempenho, por exemplo, através de troca de arquivos, uso de interface de redes ou de mecanismos de comunicação entre processos.

II. Um Pod tem um endereço IP, um nome e uma faixa de portas compartilhadas por todos os contêineres pertencentes a ele. Isso significa que os contêineres de um mesmo Pod devem ser configurados cuidadosamente a fim de evitar conflitos de portas.
III. Um Pod é um elemento persistente no tempo, ele resiste às operações de redimensionamento, falhas de verificação de sanidade de contêineres e migrações entre nós.

Está correto o que se afirma em

a) I, somente.
b) II, somente.
c) III, somente.
d) I e II, somente.
e) I e III, somente.

Comentários:

Vamos ver cada um os itens.

I. Certo. Com a alocação de todos os contêineres para determinado serviço dentro de um mesmo *pod*, garantimos que eles compartilhem uma mesma localização.
II. Certo. Cada *pod* tem um endereço de IP e outras configurações de rede – que são aplicáveis ao contexto do seu nó.
III. Errado. Os *pods* são entidades efêmeras, não persistentes.

Corretas as afirmativas I e II.

Gabarito: Letra D

---

20. (CEBRASPE/BANRISUL/2022) Em relação a contêineres em aplicações, julgue o item a seguir.

O Kubernetes faz o escalonamento e a recuperação no caso de falha de uma aplicação.

Comentários:

Devido à efemeridade dos *pods*, o kubelet faz um monitoramento constante da saúde de cada um deles, removendo, substituindo ou adicionando novos ao *node* sempre que necessário. Isso, de fato, traz um escalonamento e uma robustez para a aplicação, fazendo uma recuperação ultra rápida.

Gabarito: Certo

---

21. (FUNDATEC/PROCERGS/2025)
O HEALTHCHECK do Docker substitui a livenessProbe/readinessProbe do Kubernetes, tornando desnecessária a configuração de probes no nível do cluster.

Comentários:

O Kubernetes ignora completamente a instrução HEALTHCHECK definida no Dockerfile da sua imagem original: o orquestrador tem os seus próprios mecanismos independentes para gerenciar o ciclo de vida e a saúde das aplicações. Enquanto o HEALTHCHECK do Docker serve para relatar o status do contêiner em ferramentas nativas da plataforma Docker, o Kubernetes exige que você configure explicitamente as sondas no seu próprio nível de orquestração.

Para que o kubelet saiba se o contêiner travou em um deadlock e precisa ser reiniciado, ou se já terminou de carregar e está pronto para receber requisições de rede, você precisa obrigatoriamente declarar a Liveness Probe e a Readiness Probe dentro do manifesto YAML do seu Pod ou do seu Deployment. Não existe nenhum tipo de substituição, herança ou conversão automática entre o Docker e o Kubernetes nesse sentido.

Gabarito: Errado

02648216103 - HENRIQUE AMORIM BARRETO

LISTA DE QUESTÕES

01. (FGV/SEF-MG/2023) Kubernetes é um sistema de orquestração de contêineres *open-source* que automatiza a implantação, o dimensionamento e a gestão de aplicações em contêineres.

Em relação ao conceito de Kubernetes, assinale a afirmativa **incorreta**.

a) Kube-proxy é um *proxy* de rede executado e mantém regras de rede em cada máquina do cluster.
b) Kubernetes utilizam controladores que rastreiam pelo menos um tipo de recurso Kubernetes.
c) Kubernetes utiliza *contexts* como mecanismo para isolar grupos de recursos dentro de um único *cluster*.
d) Kubelet é um agente que é executado em cada máquina do *cluster*; ele garante que os contêineres estejam sendo executados em um Pod.
e) Os objetos do Kubernetes são entidades persistentes no Kubernetes e utilizam estas entidades para representar o estado do *cluster*.

02. (FCC/MPE PB/2023) Considere que a equipe de TI do Ministério Público utiliza o Kubernetes como ferramenta de controle de clusters e neste ambiente também encontra-se o Kubectl, uma ferramenta de linha de comando para gerenciar objetos e clusters Kubernetes, instalados e funcionando em condições ideais. Um Analista foi solicitado a apresentar os comandos para as seguintes questões relativas ao Kubectl:

I. Verificar se o Kubectl pode autenticar e acessar o cluster Kubernetes de um usuário.
II. Visualizar como está a configuração do Kubectl.
III. Buscar a lista de clusters definidos no kubeconfig.

Os comandos para realizar as tarefas I, II e III são, correta e respectivamente,

a) $ kubectl kubernetes-info / $ kubectl config_view / $ kubectl get-clusters
b) $ kubectl cluster-info / $ kubectl view_config / $ kubectl config list-clusters
c) $ kubectl kubectl-info / $ kubectl config view / $ kubectl list-clusters
d) $ kubectl kubernetes-info / $ kubectl view_config / $ kubectl get-clusters
e) $ kubectl cluster-info / $ kubectl config view / $ kubectl config get-clusters

03. (FEPESE/PREF. B. CAMBORIÚ/2023) Assinale a alternativa que descreve corretamente o conceito de Serviço do Kubernetes.

a) A entrega de pods como instâncias de processos à aplicação.
b) A orquestração de containers ao longo de todo o cluster, com gerenciamento automatizado.
c) Uma forma de expor uma aplicação rodando sobre um conjunto de pods como um serviço de rede.
d) A distribuição de pods pelos nós do cluster, implementando um balanceamento de carga pró-ativo.
e) É o componente que monitora a existência de Pods recentemente criados sem nó designado, e lhes atribui um nó para que sejam executados.

04. (FEPESE/PREF. B. CAMBORIÚ/2023) Analise as afirmativas abaixo sobre Kubernetes.

1. O Controle Plane é responsável por gerenciar os principais componentes e garantir que o funcionamento esteja de acordo com o estado almejado da aplicação.
2. O etcd é um repositório de chave-valor consistente e de alta disponibilidade.
3. O servidor de APIs do Kubernetes é denominado kube-container.

Assinale a alternativa que indica todas as afirmativas corretas.

a) É correta apenas a afirmativa 1.
b) São corretas apenas as afirmativas 1 e 2.
c) São corretas apenas as afirmativas 1 e 3.
d) São corretas apenas as afirmativas 2 e 3.
e) São corretas as afirmativas 1, 2 e 3.

05. (AOCP/IFMA/2023) No Kubernetes, qual é a abordagem correta para implementar comunicação segura entre os componentes do cluster e garantir autenticação e autorização adequadas?

a) Desabilitar o controle de acesso baseado em função (RBAC) e permitir o acesso a todos os usuários e serviços.
b) Utilizar certificados autoassinados para todos os componentes do cluster e ignorar a validação de certificados.
c) Implementar políticas de segurança usando o etcd como armazenamento centralizado de credenciais e chaves de criptografia.
d) Configurar o Kubernetes para utilizar criptografia TLS, certificados válidos e controle de acesso baseado em função (RBAC).
e) Utilizar somente redes não criptografadas entre os componentes do cluster para melhorar o desempenho.

06. (VUNESP/TJM SP/2023) Em um arquivo YAML de definição de um objeto do Kubernetes, a descrição do estado desejado desse objeto, aplicável à grande parte dos tipos de objetos, é dada pelo campo

a) namespace.
b) metadata.
c) status.
d) desired.
e) spec.

07. (FGV/SEFAZ AM/2022) Suponha a entrega de um laboratório (SaaS) como microsserviços na nuvem, onde cada aluno terá um espaço dedicado de memória, processamento, e poderá abrir uma interface Web e rodar códigos em Python. O espaço individual de cada aluno será criado a partir de uma mesma imagem.

Assinale a opção que apresenta a plataforma de entrega na nuvem para atender a essa demanda.

a) Big Query.
b) Azure Cosmos DB.
c) Azure Kubernetes.
d) Amazon Simple Storage Service.
e) Amazon Quantum Solutions Lab.

08. (FCC/TRT 4/2022) Considere o seguinte texto:

Nowadays, firewalls are way more sophisticated than they used to be. They are now capable of inspecting the application-level protocols and are often not only deployed at the forefront of the infrastructure but also between each component, to monitor and limit unauthorized accesses.
For the same reason, some container orchestrator tools (such as ....I.... , but also the public cloud providers) offer the possibility to implement the so-called network policies, which are essentially ....II.... acting as a network firewall, hence not allowing (or dropping) unwanted network connections. Firewalls can be hardware appliances (with major vendors including Cisco and Check Point, among others), or even software distributions (such as PFSense and Zeroshell).

(Adaptado de: BONOCORE, Giuseppe. Hands-On Software Architecture with Java. Packt Publishing, 2022)

As lacunas **I** e **II** devem ser preenchidas, correta e respectivamente, com

a) *Kubernetes* e *Access Control Lists*
b) *Docker* e *Reverse Proxies*
c) *Rancher* e *Disaster Recovery Policies*
d) *Jenkins* e *Reverse Proxies*
e) Maven e *Access Control Lists*

09. (IDECAN/UNIALB/2022) Compreender como o Kubernetes gerencia os recursos é fundamental para criar e executar seu cluster corretamente. Analise as afirmativas a seguir sobre Kubernets:

I. O Kubernetes aloca recursos de CPU e memória para contêineres com base em solicitações e limites.
II. As solicitações de um contêiner são as quantidades mínimas de recursos que ele precisa para executar. Seus limites especificam a quantidade máxima que pode ser usada.
III. Imagens de contêiner mínimas são mais rápidas de construir, enviar, implantar e iniciar. Quanto maior o contêiner, menores são as vulnerabilidades de segurança em potencial.

Assinale

a) se somente o item I estiver correto.
b) se somente o itens I e II estiverem corretos.
c) se somente o itens II e III estiverem corretos.
d) se todos os itens estiverem corretos.

10. (FCC/TRT 22/2022) No Kubernetes, os contêineres gerenciados pelo kubelet podem usar a estrutura de hook do ciclo de vida do contêiner para executar código acionado por eventos durante o gerenciamento de seu ciclo de vida. Há dois tipos de hooks:

I. Este hook é executado imediatamente após um contêiner ser criado. Mas não há garantia de que o hook será executado antes do ENTRYPOINT do contêiner. Nenhum parâmetro é passado para o handler.
II. Esse hook é chamado imediatamente antes de um contêiner ser terminated devido a uma solicitação de API ou um gerenciamento de evento como liveness/startup probe failure, preemption, resource contention e outros. Uma chamada a este hook falha se o contêiner já está em um estado terminated ou completed e o

hook deve ser concluído antes que o sinal TERM seja enviado para parar o contêiner. Nenhum parâmetro é passado para o handler.

Os hooks I e II são, correta e respectivamente,

a) PostStart e PreStop.
b) HookStart e HookStop.
c) PodStart e PodStop.
d) KubeStart e KubeStop.
e) KubeletStart e KubeletStop.

11. (COCP/IF MT/2022) Considere as seguintes afirmações:

I - DevOps é uma cultura para organização e monitoramento do desenvolvimento de softwares para entrega contínua de resultados.
II - Kubernetes é uma técnica que utiliza contêineres para virtualização de serviços e recursos computacionais a nível de sistema operacional.
III - Docker é uma plataforma de orquestração e automatização de contêineres, conforme site oficial (https:// *www.docker.com/*).

Assinale a alternativa CORRETA.

a) Apenas a afirmação I está correta.
b) As afirmações I e II estão corretas.
c) Apenas a afirmação II está correta.
d) As afirmações II e III estão corretas.
e) As afirmações I, II e III estão corretas.

12. (FUMARC/TRT 3/2022) Analise as afirmativas a seguir referentes a Kubernetes:

I – Kubernetes é uma plataforma de código aberto, portável e extensiva para o gerenciamento de cargas de trabalho e serviços distribuídos em contêineres.
II – Kubernetes é um sistema PaaS (plataforma como serviço) monolítico que opera no nível do hardware.
III – Um cluster Kubernetes consiste em um conjunto de servidores de processamento, chamados nós, que executam aplicações containerizadas.

Está CORRETO o que se afirma em:

a) I e II, apenas.
b) I e III, apenas.
c) II e III, apenas.
d) III, apenas.
e) I, II e III.

13. (CEBRASPE/TRT 8/2022) Entre as funcionalidades do Kubernetes, a que gerencia os contêineres quanto à verificação de integridade definida pelo usuário é

a) gerenciamento de configuração e de segredos.
b) orquestração de armazenamento.
c) lançamentos e reversões automatizadas.
d) empacotamento binário automático.
e) Autocorreção

14. (CEBRASPE/SERPRO/2021) A respeito do Kubernetes, julgue o item a seguir.

A camada de gerenciamento possui o componente etcd, cuja função é observar pods que foram criados sem nenhum node atribuído e selecionar um node para execução.

15. (IADES/BRB/2021) Kubernetes é uma plataforma de código aberto, portável e extensiva para o gerenciamento de cargas de trabalho e serviços distribuídos em contêineres, que facilita tanto a configuração declarativa quanto a automação. Ele possui um ecossistema grande e de rápido crescimento. Serviços, suporte e ferramentas para Kubernetes estão amplamente disponíveis.
Disponível em: <https://kubernetes.io/pt-br/docs/concepts/overview/>. Acesso em: 25 jun. 2021, com adaptações.
Com base no texto apresentado e considerando o contexto do Kubernetes, assinale a alternativa que corresponde à ferramenta disponibilizada para realizar operações nos clusters Kubernetes, por meio de interface de linha de comando, pela qual é possível realizar a implantação de aplicações, inspecionar e gerenciar recursos do cluster e visualizar logs.

a) Kubeadm
b) Minikube
c) Kubectl
d) Kind
e) Kubelet

16. (CEBRASPE/SEFAZ CE/2021) Julgue o próximo item, relativo ao Apache Kafka e ao Kubernetes.

No Kubernetes, kubelet é uma pequena aplicação localizada em um nó que se comunica com o plano de controle, assegurando que os containers estejam em execução em um pod, que consiste no menor e mais simples objeto do Kubernetes.

17. (IUDS/IF RJ/2021) Analise as afirmações a seguir sobre Kubernetes:

I - Pod: um grupo de um ou mais containers implantados, em um único nó. Todos os containers em um pod compartilham o mesmo endereço IP, IPC, nome do host e outros recursos. Os pods separam a rede e o armazenamento do container subjacente. Isso facilita a movimentação dos containers pelo cluster.
II - Serviço: desacopla as definições de trabalho dos pods. Os proxies de serviço do Kubernetes, automaticamente, levam as solicitações de serviço para o pod correto, independentemente do local do pod no cluster ou se foi substituído.
III - kubectl: um serviço executado nos nós que lê os manifestos do container e garante que os containers definidos foram iniciados e estão em execução.

Estão corretas as afirmações:

a) I e II.
b) I e III.

c) II e III.
d) I, II e III.

18. (FGV/TJ RO/2021) A equipe de desenvolvimento de sistemas de um tribunal de contas está guiando a implantação de um Webservice REST.

A implantação será dividida nos seguintes grupos distintos de containers Docker:
- Grupo A: responsável pela execução da aplicação do Webservice REST
- Grupo B: responsável pela execução do Sistema Gerenciador de Banco de Dados utilizado pelo Webservice REST

Os Grupos A e B terão seu próprio contexto de armazenamento e rede a serem orquestrados por um cluster de Kubernetes.

Para que a conexão de rede entre os containers dos Grupos A e B seja bem-sucedida pelo orquestrador, independentemente dos endereços IP a eles atribuídos, deverá ser configurado um novo:

a) pod;
b) replicaSet;
c) service;
d) ingress;
e) statefulSet.

19. (FGV/FUNSAÚDE CE/2021) Pod é uma unidade atômica de escalonamento, implantação e isolamento na execução de um grupo de contêineres no Kubernetes, analise as afirmativas a seguir.

I. Um Pod garante uma mesma localização para os seus contêineres, graças a isso eles têm diversas formas de interagir com bom desempenho, por exemplo, através de troca de arquivos, uso de interface de redes ou de mecanismos de comunicação entre processos.
II. Um Pod tem um endereço IP, um nome e uma faixa de portas compartilhadas por todos os contêineres pertencentes a ele. Isso significa que os contêineres de um mesmo Pod devem ser configurados cuidadosamente a fim de evitar conflitos de portas.
III. Um Pod é um elemento persistente no tempo, ele resiste às operações de redimensionamento, falhas de verificação de sanidade de contêineres e migrações entre nós.

Está correto o que se afirma em

a) I, somente.
b) II, somente.
c) III, somente.
d) I e II, somente.
e) I e III, somente.

20. (CEBRASPE/BANRISUL/2022) Em relação a contêineres em aplicações, julgue o item a seguir.

O Kubernetes faz o escalonamento e a recuperação no caso de falha de uma aplicação.

21. (FUNDATEC/PROCERGS/2025)

O HEALTHCHECK do Docker substitui a livenessProbe/readinessProbe do Kubernetes, tornando desnecessária a configuração de probes no nível do cluster.

GABARITO

GABARITO



1. Letra C
2. Letra E
3. Letra C
4. Letra B
5. Letra D
6. Letra E
7. Letra C
8. Letra A
9. Letra B
10. Letra A
11. Letra A
12. Letra B
13. Letra E
14. Errado
15. Letra C
16. Certo
17. Letra A
18. Letra C
19. Letra D
20. Certo
21. Errado

PODMAN: CONTAINERS SEM DAEMON NO LINUX

Introdução ao Podman e Arquitetura Daemonless

Vamos iniciar nosso estudo sobre o **Podman**, a ferramenta de gerenciamento de containers que se tornou referência no ecossistema Linux nos últimos anos, especialmente em distribuições derivadas do Red Hat (RHEL, Fedora, CentOS Stream, Rocky Linux, AlmaLinux). O nome Podman vem de **Pod Manager**, já antecipando uma das suas características distintivas: a capacidade nativa de agrupar containers em pods, em uma sintaxe próxima ao Kubernetes.

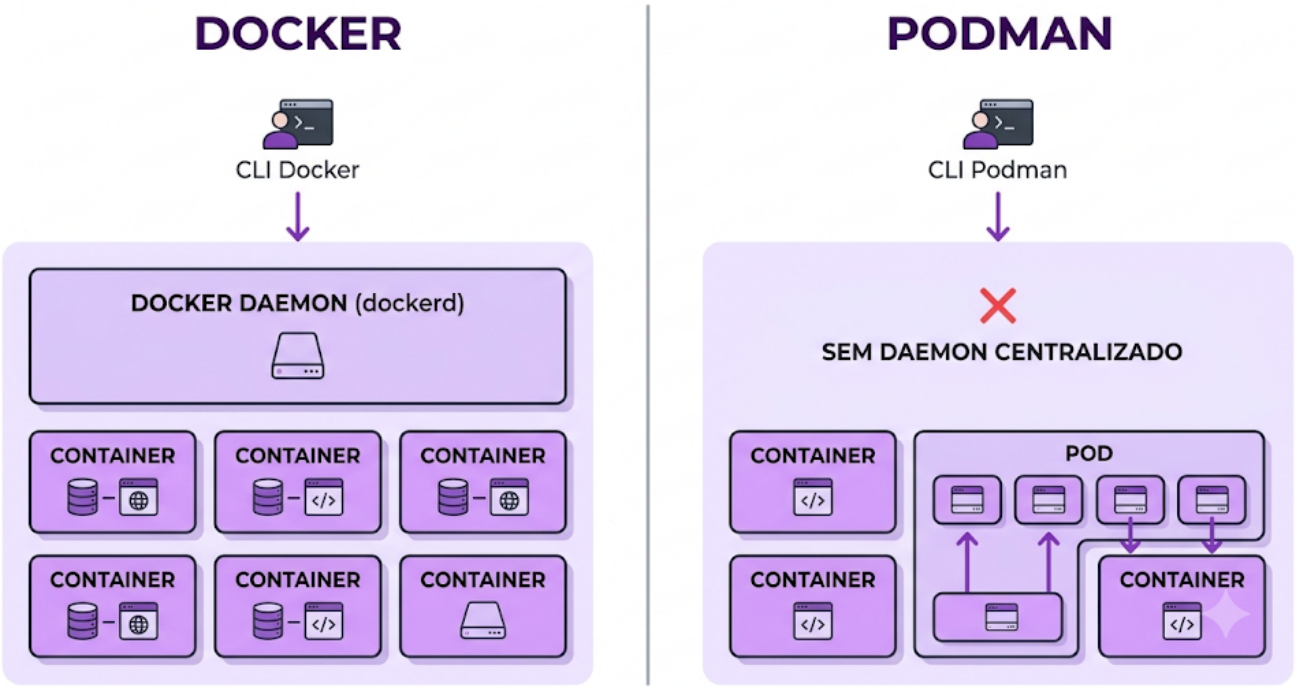
Antes de mergulharmos nos detalhes técnicos, é importante entender o contexto histórico. O Docker, lançado em 2013, popularizou a ideia de containers de aplicação como unidade de empacotamento e execução. O modelo arquitetural escolhido pela Docker Inc. foi cliente-servidor: o usuário invoca o binário docker, que por sua vez conversa com um daemon de longa duração, o dockerd, rodando como root no host. Esse daemon é o responsável por criar namespaces, montar sistemas de arquivos e iniciar processos.

O Podman propõe outra abordagem: nele, não existe daemon central. Cada execução do binário podman cria diretamente o container como processo filho do próprio cliente, no contexto do usuário que invocou o comando. Em termos de unix tradicional, é um modelo fork-exec direto, sem intermediários, em que o kernel cuida do isolamento por meio de namespaces e cgroups, como sempre foi a base técnica de qualquer container Linux.

Vantagens do modelo daemonless

A ausência de daemon traz três benefícios principais. Primeiro, não há um ponto único de falha: se um container travar, isso não derruba os outros, porque cada um está vinculado ao seu próprio processo de invocação. Segundo, a auditoria fica natural: o processo do container aparece como filho do usuário no ps e nas ferramentas de auditoria, o que facilita rastrear quem subiu o quê. Terceiro, a integração com o systemd, init padrão das distribuições modernas, fica direta, porque cada container pode virar uma unit gerenciada pelo init, sem competir com um daemon paralelo.

Há também uma consequência de segurança importante. No Docker tradicional, qualquer usuário com acesso ao socket do dockerd pode, na prática, virar root no host (basta montar / e editar arquivos sensíveis a partir de dentro do container). No Podman, como não há daemon privilegiado, esse vetor de ataque desaparece. Estudaremos isso em mais detalhe na seção sobre modo rootless.



Observe que no Podman a seta vai direto do binário ao container, sem passar por um processo intermediário rodando como root. Essa diferença arquitetural é o que sustenta o modelo rootless seguro.

Fonte: Prof. Vinícius Borges

Compatibilidade de CLI com o Docker

Apesar da diferença arquitetural, o Podman foi desenhado com compatibilidade de linha de comando com o Docker. A maior parte dos subcomandos (run, ps, images, build, pull, push, exec, stop, rm) tem sintaxe equivalente. Em distribuições derivadas do RHEL, é comum encontrar um pacote chamado podman-docker que instala um wrapper /usr/bin/docker apontando para o podman. Isso permite reaproveitar scripts, tutoriais e até pipelines de CI/CD com pouca ou nenhuma alteração.

Além da compatibilidade de CLI, o Podman expõe uma API REST compatível com a API do Docker Engine, via o comando podman system service. Com isso, ferramentas que conversam com a API do Docker (como o cliente Docker, o docker-compose ou o testcontainers) podem apontar para o socket do Podman e funcionar sem modificações.

> "O Podman fornece a mesma experiência de linha de comando do Docker, mas com uma arquitetura mais segura e flexível, baseada em fork-exec direto e em namespaces do kernel." Documentação oficial do projeto Podman.

Para a prova:

- O Podman é daemonless: não há serviço central; cada container é processo filho do binário podman.

- A CLI do Podman é compatível com a do Docker para a maioria dos subcomandos cotidianos.

- O Podman expõe uma API REST compatível com a do Docker Engine via podman system service.

- O modelo arquitetural daemonless é a base que sustenta o modo rootless seguro.



(PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman adota uma arquitetura sem daemon central, em que cada container é executado como um processo filho direto do cliente que o invocou, dispensando um serviço de longa duração em background como ocorre no modelo do Docker tradicional.

Comentários:

Conforme estudamos nesta seção, o Podman foi desenhado com arquitetura daemonless. Cada execução do binário podman dispara o container como processo filho direto, sob controle do usuário que executou o comando. Isso é exatamente o oposto do modelo do Docker clássico, em que o cliente conversa com o dockerd. A consequência prática é uma melhor integração com o systemd e a eliminação do daemon root como vetor de ataque.

Gabarito: Certo

Operação Cotidiana: Containers, Imagens, Volumes e Registries

Nesta seção, vamos colocar a mão na massa nos comandos do dia a dia. O ciclo básico de uso do Podman envolve quatro operações: puxar uma imagem de um registry, executar um container a partir dela, gerenciar o armazenamento persistente de dados e listar ou parar o que está em execução.

Subcomandos essenciais

O comando central é o podman run, que cria e inicia um container a partir de uma imagem em uma única operação. As flags mais usadas no cotidiano são: -d (detach, executa em background), -p HOST:CONTAINER (mapeamento de portas), -v ORIGEM:DESTINO (montagem de volumes ou bind mounts), --name (atribui um nome amigável ao container) e -e CHAVE=VALOR (define variáveis de ambiente).

Para listar containers em execução usa-se podman ps; para incluir os parados, podman ps -a. As imagens locais são listadas com podman images. Para parar um container usa-se podman stop NOME, e para removê-lo, podman rm NOME. Imagens são removidas com podman rmi IMAGEM, e a limpeza geral de containers e imagens órfãs é feita com podman system prune.

Operação	Comando	Equivalente Docker
Puxar imagem	podman pull IMG	docker pull IMG
Subir container	podman run -d -p ... IMG	docker run -d -p ... IMG
Listar em execução	podman ps	docker ps
Parar container	podman stop NOME	docker stop NOME
Ver logs	podman logs NOME	docker logs NOME
Entrar no container	podman exec -it NOME bash	docker exec -it NOME bash

Volumes e bind mounts

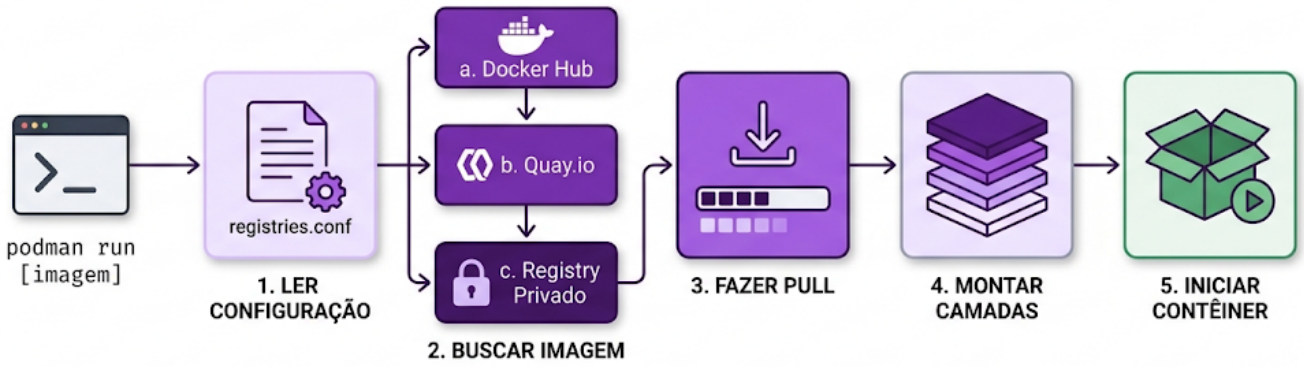
Containers são, por natureza, efêmeros: quando removidos, os dados gravados na camada de escrita são perdidos. Para preservar estado entre execuções, o Podman oferece dois mecanismos. Os bind mounts mapeiam um caminho do host diretamente para um caminho do container, com a flag -v /host/path:/container/path. São úteis em desenvolvimento, quando queremos expor código-fonte ou logs visíveis no editor do host.

Os volumes nomeados são gerenciados pelo Podman em uma área de storage dedicada, criada com podman volume create NOME e usada com -v NOME:/container/path. No modo rootless, essa área fica em ~/.local/share/containers/storage/volumes. Volumes nomeados sobrevivem à remoção do container e são portáveis entre containers, sendo a escolha recomendada para dados de aplicação como bancos de dados.

Configuração de registries

Quando você pede uma imagem sem prefixo, como podman pull nginx, o Podman precisa decidir de qual registry buscar. Essa lista ordenada é definida em /etc/containers/registries.conf, no formato TOML. Diferente do Docker, que assume implicitamente o Docker Hub, o Podman permite que o administrador configure múltiplos registries (Docker Hub, quay.io, registry.redhat.io, registries privados internos) e a ordem em que serão consultados.

Esse mesmo arquivo é compartilhado com o Buildah e o CRI-O, formando uma configuração unificada para o ecossistema OCI no Linux. Lá também se definem registries inseguros (sem TLS), espelhos para reduzir tráfego e bloqueios de registries específicos. Para autenticar em um registry privado, usa-se podman login REGISTRY, que grava as credenciais em ~/.config/containers/auth.json.



Observe que o cliente Podman é quem orquestra a busca por registries, ao contrário do Docker em que essa lógica fica no daemon. A ordem definida em registries.conf é determinante quando o nome da imagem vem sem prefixo.

Fonte: Prof. Vinícius Borges

Para a prova:

- podman run é o ponto de entrada para criar e iniciar containers; -d coloca em background, -p mapeia portas.

- Bind mounts expõem caminhos do host; volumes nomeados ficam em área gerenciada pelo Podman.

- A lista de registries fica em /etc/containers/registries.conf, formato TOML, compartilhada com Buildah e CRI-O.

- O Podman não fixa Docker Hub como única origem; o administrador define a ordem.



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a configuração de registries de imagens no Podman, assinale a alternativa correta.

a) O Podman aceita apenas o Docker Hub como registry de origem.

b) Não há configuração de registries: o usuário deve sempre informar a URL completa da imagem.

c) O Podman fixa o quay.io como único registry permitido.

d) A lista de registries é compilada estaticamente no binário.

e) A lista de registries é configurada em /etc/containers/registries.conf, podendo incluir Docker Hub, quay.io, registry.redhat.io ou registries privados.

Comentários:

Como vimos acima, /etc/containers/registries.conf é o arquivo TOML que centraliza a configuração de registries no Podman e em todo o ecossistema OCI no Linux (Buildah, CRI-O). É lá que se define a lista ordenada de registries para imagens sem prefixo, registries inseguros, espelhos e bloqueios.

Gabarito: Letra E

Modo Rootless: Namespaces de Usuário e Isolamento sem Root

Chegamos a um dos pontos centrais do Podman, o que mais o diferencia do Docker clássico: o modo rootless, em que containers são executados por usuários comuns, sem necessidade de privilégios de superusuário. Esse modelo é o padrão recomendado em estações de trabalho e em muitos cenários de servidor.

O mecanismo de user namespaces

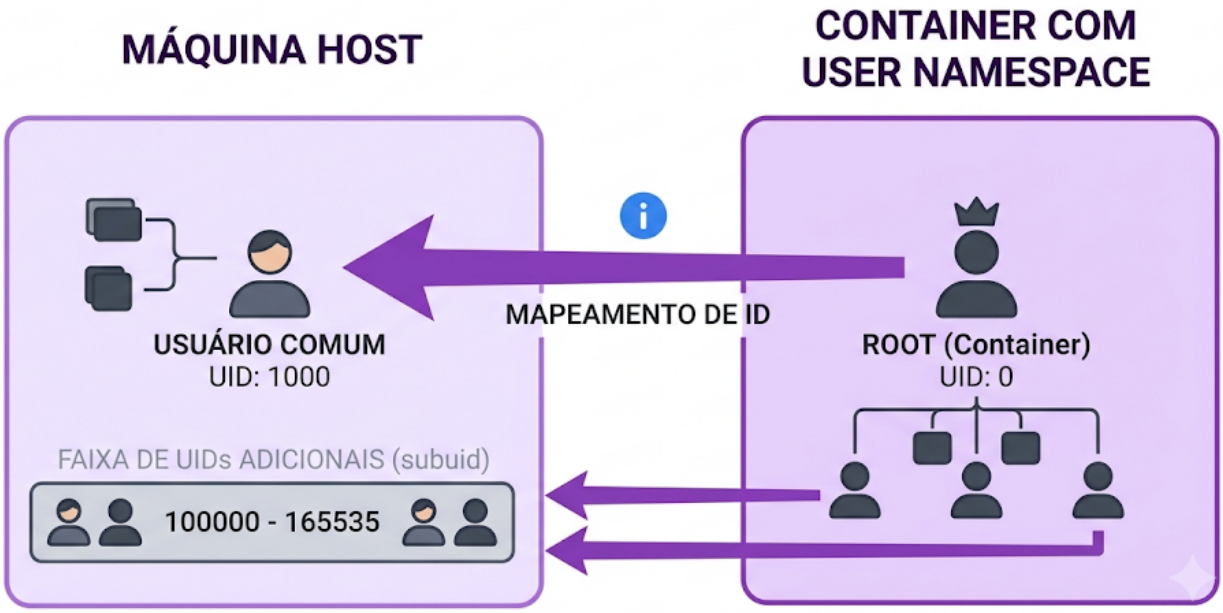
A base técnica do rootless é o user namespace, recurso do kernel Linux que permite remapear UIDs e GIDs entre o host e o namespace. Cada usuário recebe, em /etc/subuid e /etc/subgid, uma faixa de UIDs e GIDs adicionais que pode usar dentro de namespaces de usuário, tipicamente 65.536 IDs por usuário, começando em 100000. Esses arquivos são gerenciados pelo utilitário shadow-utils.

Quando um container rootless é criado, o Podman aloca um user namespace e configura o mapeamento: o UID 0 (root) dentro do container corresponde ao UID do usuário no host (ou a um UID da faixa subuid). Dentro do container, processos veem-se como root e podem executar operações administrativas no escopo daquele namespace; do lado de fora, no host, eles são apenas processos comuns do usuário, sem privilégios reais. Esse desacoplamento é a chave do isolamento sem root.

Limitações inerentes ao rootless

O modo rootless herda algumas restrições do kernel que não podem ser contornadas. A mais conhecida é a impossibilidade de fazer bind em portas privilegiadas (abaixo de 1024) sem CAP_NET_BIND_SERVICE no namespace correto. O sysctl net.ipv4.ip_unprivileged_port_start define o limite inferior das portas que usuários não privilegiados podem ocupar, e o padrão das distribuições geralmente é 1024. Para mapear porta 80 ou 443 em rootless, há três alternativas: ajustar esse sysctl, colocar um proxy reverso na porta privilegiada, ou rodar em modo rootful.

Outra restrição é o desempenho de rede. Como não é possível criar interfaces bridge ou veth sem CAP_NET_ADMIN, o tráfego de rede de containers rootless passa por um stack em espaço de usuário. A primeira implementação foi o slirp4netns, que cria uma interface tap dentro do namespace de rede e traduz pacotes para sockets normais no host. Em versões mais recentes, o pasta (parte do projeto passt) substitui o slirp4netns com melhor desempenho e menor overhead.



Observe que o root dentro do container nunca é o root real do host. Esse mapeamento é o que permite ao usuário comum executar containers com isolamento equivalente ao do modo rootful.

Fonte: Prof. Vinícius Borges

Comparativo rootful vs rootless

Aspecto	Rootful	Rootless
Execução	como root	como usuário comum
Portas privilegiadas	OK direto	exige sysctl ou proxy
Networking	bridge nativa	slirp4netns ou pasta
Storage padrão	/var/lib/containers	~/.local/share/containers
Segurança	menor isolamento	isolamento adicional via user namespace
Desempenho de rede	máximo	menor (proxy em userspace)

Para a prova:

- Rootless usa user namespaces; o UID 0 do container mapeia para o UID do usuário no host.

- /etc/subuid e /etc/subgid definem faixas adicionais de IDs por usuário (geralmente 65.536 IDs).

- Portas abaixo de 1024 exigem ajuste no sysctl ou proxy reverso; o Podman não contorna o kernel.

- A rede em rootless passa por slirp4netns ou pasta em espaço de usuário, com leve queda de desempenho.



(PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, o usuário que executa os containers acessa o UID 0 (root) do host por meio de um mecanismo de elevação temporária de privilégios, configurado nos arquivos /etc/sudoers.d, sem necessidade de mapeamentos de namespace.

Comentários:

Atenção: o modo rootless funciona exatamente para evitar elevação de privilégios. Como estudamos nesta seção, o mecanismo central é o user namespace combinado com /etc/subuid e /etc/subgid. O UID 0 dentro do container é mapeado para o UID do usuário no host, e o processo nunca tem privilégio real de root. Sudoers não tem papel algum nesse fluxo. É um detalhe sutil que merece atenção, pois confundir esses mecanismos é um erro comum.

Gabarito: Errado

Pods: Agrupando Containers em uma Unidade Lógica

O conceito de Pod vem do Kubernetes e foi incorporado ao Podman, dando origem ao próprio nome da ferramenta. Um Pod é um agrupamento lógico de um ou mais containers que precisam coexistir, compartilhando recursos e podendo ser gerenciados como unidade única.

O que um Pod compartilha

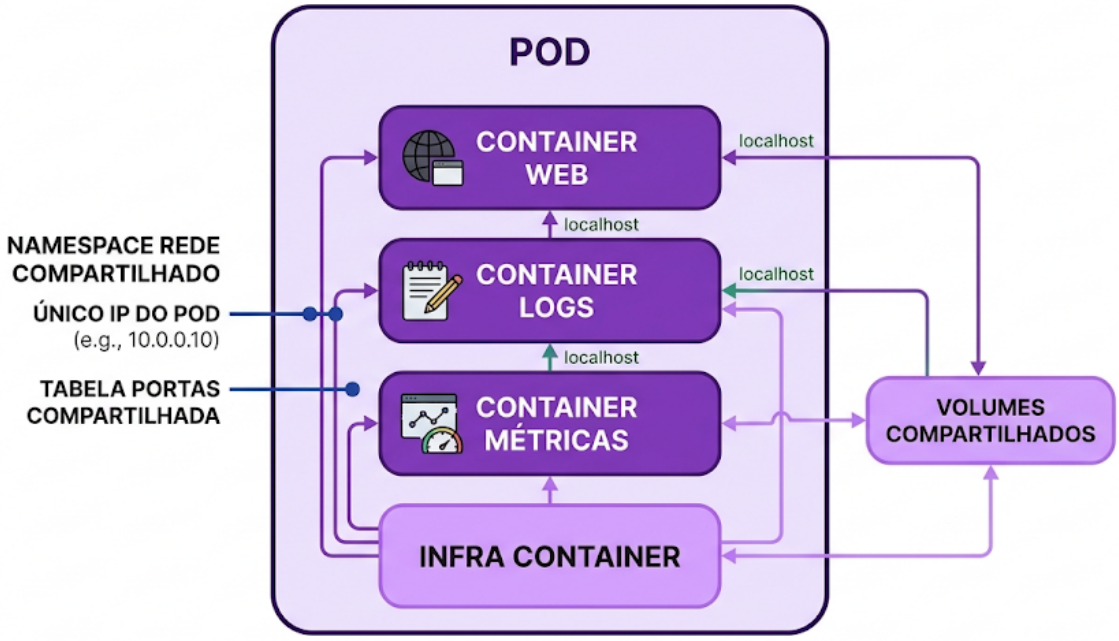
Containers dentro do mesmo pod compartilham o namespace de rede, o que significa: mesmo endereço IP, mesma tabela de portas e capacidade de se comunicar via localhost. Eles também podem compartilhar o namespace IPC, permitindo uso de memória compartilhada e semáforos entre processos. Já o namespace de sistema de arquivos permanece separado, mantendo o isolamento das raízes de cada container.

Esse modelo facilita executar localmente um conjunto de containers que dependem uns dos outros, como uma aplicação web e seu sidecar de logging, ou um banco e um exportador de métricas. Em produção, esses mesmos containers rodariam em um Pod do Kubernetes; em desenvolvimento e em pequenos servidores, o Podman oferece a mesma abstração sem precisar de cluster orquestrado.

Criando e gerenciando pods

A criação de um pod começa com podman pod create --name meupod -p 8080:80. Esse comando cria um container especial chamado infra container, que segura os namespaces compartilhados (rede, IPC) enquanto os demais containers ficam acoplados. Em seguida, containers são adicionados com podman run -d --pod meupod IMAGEM. Cada container herda o namespace de rede do pod, e o mapeamento de portas é declarado uma única vez, no pod.

Os comandos de gerenciamento operam no pod inteiro: podman pod start, podman pod stop, podman pod rm, podman pod ps. Isso permite tratar o conjunto como uma unidade, parando ou reiniciando tudo de uma vez. Há também o utilíssimo podman generate kube, que produz um manifesto YAML do pod no formato Kubernetes, permitindo migrar a aplicação para um cluster real sem reescrever a configuração.



Observe que os três containers compartilham o mesmo IP do pod e se comunicam via localhost. O infra container é uma peça invisível que mantém os namespaces vivos mesmo quando os containers de aplicação são reiniciados.

Fonte: Prof. Vinícius Borges

Para a prova:

- Pod é agrupamento lógico de containers que compartilham namespace de rede (e opcionalmente IPC).

- O nome Podman vem de Pod Manager.

- podman generate kube produz manifesto YAML compatível com Kubernetes.

- Containers de um pod compartilham o IP e o mapeamento de portas é declarado no pod, não em cada container.



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o conceito de Pod no Podman, assinale a alternativa correta.

a) Pod é uma forma de empacotar e distribuir imagens de container.

b) Pod é um agrupamento lógico de um ou mais containers que compartilham namespaces de rede, podendo ser gerenciado como unidade única.

c) Pod é o nome dado ao processo daemon do Podman.

d) Pod é um sistema de orquestração distribuída que substitui o Kubernetes.

e) Pod é uma camada de armazenamento persistente compartilhada entre containers.

Comentários:

Conforme estudamos nesta seção, o Pod é a abstração inspirada no Kubernetes que agrupa containers compartilhando rede e, opcionalmente, IPC. As alternativas A, C, D e E confundem o conceito com imagens, com daemon (que o Podman nem tem), com orquestrador e com volume. O recurso permite ainda gerar o manifesto YAML do pod para Kubernetes, fechando o ciclo entre desenvolvimento local e produção em cluster.

Gabarito: Letra B

Construção de Imagens com Buildah e Containerfile

Containers rodam a partir de imagens, e imagens precisam ser construídas. No ecossistema Podman, o motor de build é o Buildah, uma ferramenta companheira que se especializa em criar imagens OCI sem precisar de daemon. O subcomando podman build é, na prática, uma chamada ao Buildah, mas o usuário pode também invocar o buildah diretamente para fluxos mais avançados.

Containerfile e Dockerfile

O Buildah lê arquivos de build com a mesma sintaxe do Dockerfile: instruções FROM, RUN, COPY, ADD, CMD, ENTRYPOINT, ENV, ARG, EXPOSE, USER, WORKDIR e demais. A única diferença é a recomendação de nomenclatura: o projeto prefere chamar o arquivo de Containerfile, nome neutro sem amarra de marca, embora Dockerfile continue funcionando. Quando o Podman roda o build, ele procura primeiro por Containerfile e, se não encontrar, recai sobre Dockerfile, sem alterar interpretação.

Vejamos um exemplo prático de Containerfile para uma aplicação Python em produção. As instruções definem a imagem base, o diretório de trabalho, a cópia do código, a instalação de dependências e o comando padrão de execução. A sintaxe é idêntica ao Dockerfile que você já conhece.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Para construir a imagem, basta executar podman build -t minhaapp:1.0 . no diretório do Containerfile. Tags adicionais podem ser anexadas com flags -t repetidas, e o push para um registry é feito com podman push minhaapp:1.0 quay.io/usuario/minhaapp:1.0.

Vantagens do Buildah

Além de processar Containerfiles, o Buildah expõe uma API de build script onde cada passo pode ser executado individualmente, sem precisar de um arquivo declarativo. Isso é útil em pipelines de CI/CD que precisam montar imagens de forma programática, sem o overhead de processar um Dockerfile gigante a cada execução. Também é possível iniciar com uma imagem vazia (scratch), construir tudo de zero e gerar imagens mínimas, sem camadas desnecessárias.

"O Buildah é uma ferramenta que facilita a construção de imagens de container compatíveis com Open Container Initiative (OCI). Ela é compatível com Dockerfile, mas não exige um daemon e oferece controle granular sobre cada camada da imagem." Documentação do projeto Buildah.

Para a prova:

- O motor de build do Podman é o Buildah.

- O Buildah lê Dockerfile sem qualquer adaptação; o nome preferido pela comunidade é Containerfile.

- podman build é, na prática, uma chamada ao Buildah.

- O Buildah também oferece API de build programático e suporta imagens scratch para minimalismo.



(PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman possui um motor de build próprio, denominado Buildah, que utiliza sintaxe declarativa específica (arquivos com extensão .build), sendo incompatível com Dockerfiles legados originalmente escritos para o Docker.

Comentários:

Nada disso! Como vimos nesta seção, o Buildah é totalmente compatível com a sintaxe de Dockerfile. A comunidade prefere o nome Containerfile, neutro, mas o Podman procura primeiro por Containerfile e, se não encontrar, recai sobre Dockerfile. Não existe extensão .build no fluxo padrão. Portanto, Dockerfiles legados rodam sem alteração com podman build, o que torna a migração do Docker para o Podman bastante suave.

Gabarito: Errado

Integração com Systemd e o Recurso Quadlet

Em servidores Linux modernos, o systemd é o init responsável por gerenciar todos os serviços do sistema. Como o Podman não tem daemon próprio, faz todo sentido que cada container vire uma unit do systemd, gerenciada como qualquer outro serviço, com start, stop, restart, enable e disable. Vamos ver as duas abordagens disponíveis para isso: o legado podman generate systemd e o moderno Quadlet.

O comando podman generate systemd

Em versões mais antigas do Podman, o fluxo era manual. O usuário criava o container com podman run, depois executava podman generate systemd --new --files --name MEUCONTAINER, e o comando produzia um arquivo container-MEUCONTAINER.service no diretório atual. Esse arquivo era então copiado para /etc/systemd/system (rootful) ou ~/.config/systemd/user (rootless), e o systemd era recarregado com systemctl daemon-reload.

Depois disso, o container podia ser gerenciado com systemctl start container-MEUCONTAINER, e habilitado para subir no boot com systemctl enable. No modo rootless, ainda era preciso ativar a sessão lingering do usuário (loginctl enable-linger USUARIO) para que as units de usuário continuassem rodando mesmo sem login interativo. O fluxo funcionava, mas era verboso e exigia repetir o ciclo a cada mudança no container.

O Quadlet: configuração declarativa nativa

A partir do Podman 4.4, lançado em 2023, o Quadlet entrou como recurso de primeira classe e mudou o jogo. A ideia é simples: em vez de gerar arquivos .service complicados, o usuário escreve um arquivo declarativo com extensão .container (ou .pod, .volume, .network, .kube) descrevendo o que ele quer, e um gerador do systemd traduz isso em unit no momento da inicialização. A sintaxe é parecida com uma unit do systemd, mas com seções específicas do Podman.

Os arquivos Quadlet ficam em /etc/containers/systemd/ (rootful) ou ~/.config/containers/systemd/ (rootless). Quando o systemd inicializa, ele executa o gerador podman-system-generator, que lê esses arquivos e produz units .service correspondentes. O usuário nunca toca nos arquivos gerados: alterar o .container e reexecutar systemctl daemon-reload basta para aplicar mudanças.

```
[Unit]

Description=Nginx container



[Container]

Image=docker.io/library/nginx:latest

PublishPort=8080:80
```

```
Volume=mydata:/usr/share/nginx/html

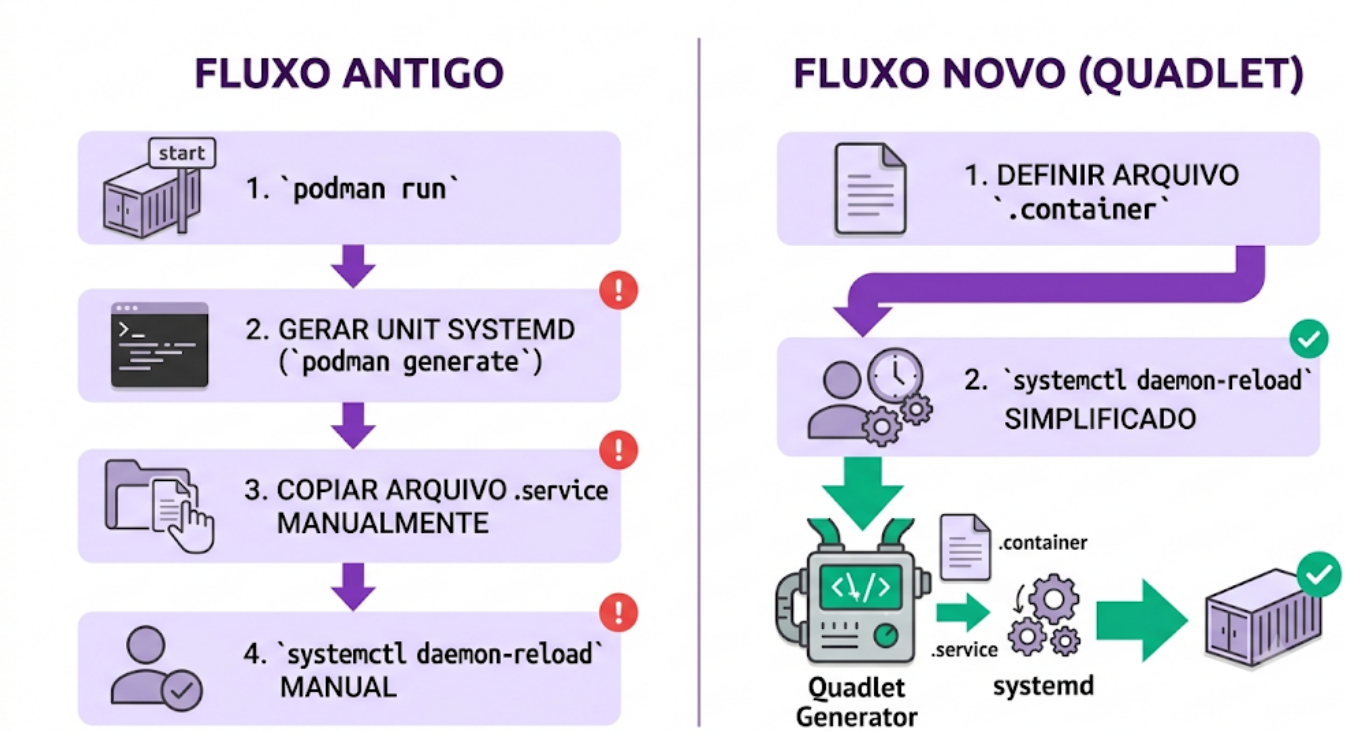

[Service]

Restart=always


[Install]

WantedBy=multi-user.target default.target
```

Esse arquivo, salvo como nginx.container em /etc/containers/systemd/, é traduzido pelo gerador em uma unit completa, com todas as flags do podman run preenchidas. É declarativo, conciso e versionável em Git. O Quadlet também suporta arquivos .pod para grupos de containers, .volume para volumes nomeados, .network para redes e .kube para manifestos Kubernetes completos.



Observe a redução de etapas. No Quadlet, o arquivo declarativo é a fonte da verdade, sem unit .service intermediária para manter atualizada manualmente.

Fonte: Prof. Vinícius Borges

Aspecto	podman generate systemd	Quadlet
Disponível desde	Podman 1.x	Podman 4.4 (2023)

Tipo de arquivo	.service (gerado)	.container, .pod, .volume, .network, .kube
Manutenção	manual; refazer a cada mudança	declarativo; daemon-reload basta
Sintaxe	verbosa, específica do Podman	concisa, próxima ao systemd
Local rootful	/etc/systemd/system	/etc/containers/systemd
Local rootless	~/.config/systemd/user	~/.config/containers/systemd

Para a prova:

- podman generate systemd cria arquivos .service a partir de containers existentes (modelo legado).

- Quadlet, do Podman 4.4 em diante, permite arquivos declarativos .container, .pod, .volume, .network, .kube.

- O gerador do systemd traduz arquivos Quadlet em units .service no boot, sem intervenção manual.

- Em rootless, é preciso ativar loginctl enable-linger para que units de usuário rodem fora da sessão.



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o Quadlet, recurso introduzido em versões recentes do Podman, assinale a alternativa correta.

a) O Quadlet substitui o systemd no gerenciamento de serviços, oferecendo um init próprio focado em containers.

b) O Quadlet é uma ferramenta gráfica para criação de Pods, sem relação com units systemd.

c) O Quadlet permite descrever containers em arquivos declarativos com extensões como .container e .pod, traduzidos em units systemd por um gerador na inicialização.

d) O Quadlet é apenas uma forma alternativa de chamar o subcomando podman play kube.

e) O Quadlet exige que o Podman rode em modo rootful, sendo incompatível com containers de usuário.

Comentários:

Conforme estudamos nesta seção, o Quadlet é a evolução do podman generate systemd. Os arquivos .container, .pod, .volume, .network, .kube são declarativos, ficam em /etc/containers/systemd (rootful) ou ~/.config/containers/systemd (rootless), e o gerador do systemd transforma cada um em uma unit na inicialização.

Gabarito: Letra C

QUESTÕES COMENTADAS



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman adota uma arquitetura sem daemon central, em que cada container é executado como um processo filho direto do cliente que o invocou, dispensando um serviço de longa duração em background como ocorre no modelo do Docker tradicional.

Comentários:

Realmente, o Podman foi desenhado com arquitetura daemonless. Cada execução do binário podman dispara o container como processo filho direto, sob o controle do usuário que executou o comando. Isso contrasta com o Docker clássico, em que o cliente conversa com o dockerd, um serviço central rodando como root. Vantagens dessa abordagem: não há um ponto único de falha em um daemon, os containers ficam vinculados ao processo do usuário (útil para auditoria) e o modelo se integra naturalmente com o systemd, já que cada container pode virar uma unidade gerenciada pelo init do sistema.

Gabarito: Certo

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere que um administrador deseja iniciar um container em modo destacado (background) a partir da imagem oficial do nginx, mapeando a porta 8080 do host para a porta 80 do container. Assinale a alternativa que apresenta o comando Podman correto para essa operação.

a) podman start -d --port 8080:80 nginx

b) podman create -bg -p 8080:80 nginx

c) podman run -d -p 8080:80 nginx

d) podman exec --detach 8080:80 nginx

e) podman launch --background --map 8080:80 nginx

Comentários:

O comando podman run é o ponto de entrada para criar e iniciar containers em uma única operação. A flag -d (de detach) coloca o container em background, e -p HOST:CONTAINER faz o mapeamento de portas. Por padrão, o Podman puxa a imagem do registry configurado se ela não estiver presente localmente.

Gabarito: Letra C

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, em que o container é executado sem privilégios de superusuário, é possível mapear portas privilegiadas do host (abaixo de 1024) diretamente para o container, pois o Podman aplica capabilities específicas que liberam essa restrição imposta pelo kernel.

Comentários:

Cuidado, pessoal! No modo rootless, o Podman não consegue contornar a restrição padrão do kernel Linux que impede usuários não privilegiados de fazer bind em portas privilegiadas (abaixo de 1024). Sem o CAP_NET_BIND_SERVICE no namespace correto, o sysctl net.ipv4.ip_unprivileged_port_start limita a faixa que pode ser usada (geralmente começa em 1024). Para mapear porta 80 ou 443 em modo rootless, existem alternativas: ajustar o sysctl para baixar o limite, colocar um proxy reverso na porta privilegiada, ou rodar em modo rootful.

Gabarito: Errado

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o conceito de Pod no Podman, assinale a alternativa correta.

a) Pod é uma forma de empacotar e distribuir imagens de container, equivalente ao formato OCI distribuído via registries.

b) Pod é um agrupamento lógico de um ou mais containers que compartilham namespaces de rede, podendo ser gerenciado como unidade única.

c) Pod é o nome dado ao processo daemon do Podman que coordena a execução dos containers no host.

d) Pod é um sistema de orquestração distribuída que substitui o Kubernetes em ambientes de borda.

e) Pod é uma camada de armazenamento persistente compartilhada entre containers, equivalente aos volumes nomeados.

Comentários:

O conceito de Pod no Podman foi inspirado no Kubernetes. Um Pod agrupa containers que precisam coexistir, fazendo-os compartilhar o mesmo namespace de rede (mesmo IP, mesmas portas) e, opcionalmente, IPC. Cria-se um pod com podman pod create e adicionam-se containers com podman run --pod nome_do_pod. Esse modelo facilita executar localmente um conjunto de containers que depois rodarão em Kubernetes; o Podman ainda gera o manifesto YAML do pod com podman generate kube.

Gabarito: Letra B

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman oferece compatibilidade de linha de comando com o Docker, de modo que a maior parte dos subcomandos (run, ps, images, build, pull, push) tem sintaxe equivalente, sendo possível criar um alias do tipo alias docker=podman para reaproveitar scripts e tutoriais sem grandes adaptações.

Comentários:

Isso mesmo! Um dos princípios de projeto do Podman foi a compatibilidade de comando com o Docker, justamente para reduzir o custo de migração. O conjunto principal de subcomandos espelha o do Docker, e o pacote oficial em distribuições derivadas do RHEL chega a instalar um wrapper docker que apenas redireciona para o podman. Há diferenças em comportamento e em comandos avançados (rede, swarm, compose com daemon nativo), mas para o uso cotidiano o alias funciona. Vale lembrar também que o Podman expõe uma API compatível com a API REST do Docker, via podman system service, o que permite que ferramentas como o docker-compose ou o cliente Docker conversem com o Podman.

Gabarito: Certo

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a integração entre Podman e systemd para gerenciamento de containers como serviços de sistema, assinale a alternativa correta.

a) O Podman exige um daemon próprio rodando sob systemd, semelhante ao dockerd, para que containers sejam tratados como serviços.

b) A única forma de gerenciar containers Podman via systemd é editar manualmente arquivos unit em /etc/systemd/system, sem suporte oficial de geração automática.

c) Containers Podman não podem ser iniciados automaticamente no boot do sistema, pois dependem sempre de invocação interativa pelo usuário.

d) O Podman fornece o subcomando podman generate systemd para criar arquivos unit a partir de containers existentes, e versões recentes incluem o Quadlet, que descreve containers em arquivos .container interpretados pelo systemd.

e) A integração com systemd só está disponível no modo rootful, sendo incompatível com containers executados em modo rootless.

Comentários:

A integração com systemd é um dos pontos fortes do Podman justamente por causa da arquitetura daemonless. Em versões mais antigas, o comando podman generate systemd produz um arquivo unit pronto para colocar em /etc/systemd/system (rootful) ou em ~/.config/systemd/user (rootless). A partir do Podman 4.4, o Quadlet entrou como recurso de primeira classe: o usuário escreve um arquivo .container declarativo e um gerador do systemd traduz isso em unit no boot. As demais alternativas estão erradas: o Podman não usa daemon (a), há suporte oficial de geração de units (b), containers podem rodar no boot (c) e o modo rootless trabalha com unidades de usuário do systemd (e).

Gabarito: Letra D

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman possui um motor de build próprio, denominado Buildah, que utiliza sintaxe declarativa específica (arquivos com extensão .build), sendo incompatível com Dockerfiles legados originalmente escritos para o Docker.

Comentários:

Nada disso! O Podman integra, sim, o Buildah como motor de build (o subcomando podman build é, na prática, uma chamada ao Buildah), mas o Buildah é totalmente compatível com a sintaxe de Dockerfile. A comunidade prefere chamar esse arquivo de Containerfile, nome neutro, sem amarra de marca, e o Podman procura primeiro por um arquivo chamado Containerfile e, se não encontrar, recai sobre Dockerfile. Não existe extensão .build no fluxo padrão do Buildah. Portanto, Dockerfiles legados rodam sem alteração com podman build.

Gabarito: Errado

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere que um administrador precisa preservar os dados de um banco PostgreSQL em container Podman entre reinicializações. Sobre as estratégias de armazenamento, assinale a alternativa correta.

a) O Podman suporta tanto bind mounts (que mapeiam diretamente um diretório do host para o container) quanto volumes nomeados (gerenciados pelo próprio Podman em uma área de storage interna).

b) O Podman aceita armazenamento persistente apenas via NFS, ficando vedado o uso de diretórios locais do host para volumes.

c) Volumes em Podman não persistem após o container ser removido, sendo automaticamente apagados junto com ele em qualquer cenário.

d) O subcomando podman volume não existe; o gerenciamento de volumes só pode ser feito pela API REST.

e) O Podman utiliza o sistema de arquivos do container em modo de escrita direto na imagem, sem necessidade de volumes para persistência.

Comentários:

O Podman oferece os dois mecanismos clássicos de armazenamento persistente. Os bind mounts são úteis quando se quer expor um diretório existente do host (logs, código-fonte em desenvolvimento), via flag -v /host/path:/container/path. Os volumes nomeados (podman volume create dbdata e -v dbdata:/var/lib/postgresql/data) são gerenciados pelo Podman em uma área dedicada (geralmente em ~/.local/share/containers para rootless), sobrevivem à remoção do container e são portáveis entre containers.

Gabarito: Letra A

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, a conectividade de rede do container com o mundo externo é provida, por padrão, por um stack em espaço de usuário (slirp4netns ou, em versões mais recentes, pasta), que cria uma interface tap em um namespace de rede sem exigir privilégios de superusuário.

Comentários:

Conforme estudamos, o modo rootless do Podman não pode criar interfaces de rede tradicionais (bridge, veth) sem CAP_NET_ADMIN. A solução é fazer o tráfego de rede passar por um proxy em espaço de usuário: o slirp4netns foi a primeira implementação dessa ideia, e o pasta (parte do projeto passt) é a alternativa mais recente, com melhor desempenho. Esses utilitários traduzem pacotes do namespace de rede do container para sockets normais no host. A consequência prática

é que o desempenho de rede em rootless é menor que em rootful, mas a portabilidade e a segurança compensam para a maioria dos cenários.

Gabarito: Certo

10. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a configuração de registries de imagens no Podman, assinale a alternativa correta.

a) O Podman aceita apenas o Docker Hub como registry de origem; outras fontes precisam ser configuradas no arquivo /etc/hosts.

b) Não há configuração de registries: o usuário deve sempre informar a URL completa da imagem no comando podman pull.

c) O Podman fixa o quay.io como único registry permitido, sem possibilidade de adicionar outros.

d) A lista de registries é definida em /etc/podman/registries.list e é compilada estaticamente no binário, não podendo ser alterada após a instalação.

e) A lista de registries usados quando o nome da imagem é informado sem prefixo é configurada em /etc/containers/registries.conf, podendo incluir Docker Hub, quay.io, registry.redhat.io ou registries privados.

Comentários:

A configuração de registries do Podman reside no arquivo /etc/containers/registries.conf (formato TOML). Esse arquivo é compartilhado com outras ferramentas do ecossistema OCI no Linux, como o Buildah e o CRI-O. Lá se define a lista ordenada de registries que serão consultados quando o usuário pedir uma imagem sem prefixo (ex.: podman pull nginx em vez de podman pull docker.io/library/nginx). Também se configuram registries inseguros, sem TLS, espelhos e bloqueios.

Gabarito: Letra E

11. (PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, o usuário que executa os containers acessa o UID 0 (root) do host por meio de um mecanismo de elevação temporária de privilégios, configurado nos arquivos /etc/sudoers.d, sem necessidade de mapeamentos de namespace.

Comentários:

Atenção: o modo rootless funciona exatamente para evitar elevação de privilégios. O mecanismo central é o user namespace do kernel Linux, combinado com os arquivos /etc/subuid e /etc/subgid. Esses arquivos definem faixas de UID e GID adicionais que cada usuário pode usar dentro de namespaces de usuário (tipicamente 65.536 IDs por usuário, começando em 100000). Dentro do container, o root é o UID 0 do namespace, mas no host esse UID é mapeado para o UID do usuário que invocou o Podman (ou para um UID da faixa subuid). O processo nunca tem privilégio real de root no host. O sudoers não tem papel algum nesse fluxo.

Gabarito: Errado

12. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o Quadlet, recurso introduzido em versões recentes do Podman, assinale a alternativa correta.

a) O Quadlet substitui o systemd no gerenciamento de serviços, oferecendo um init próprio focado em containers.

b) O Quadlet é uma ferramenta gráfica para criação de Pods, sem relação com units systemd.

c) O Quadlet permite descrever containers em arquivos declarativos com extensões como .container e .pod, que são traduzidos em units systemd por um gerador na inicialização, evitando a manutenção manual de arquivos service gerados por podman generate systemd.

d) O Quadlet é apenas uma forma alternativa de chamar o subcomando podman play kube, sem novidade arquitetural.

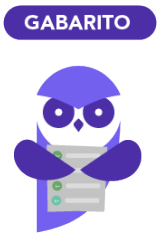
e) O Quadlet exige que o Podman rode em modo rootful, sendo incompatível com containers de usuário.

Comentários:

O Quadlet foi a evolução natural do podman generate systemd. No modelo antigo, o usuário criava o container com podman run, depois executava podman generate systemd --new para gerar um arquivo .service, copiava esse arquivo para o local apropriado e recarregava o systemd. Sempre que mudasse algo, esse ciclo precisava ser refeito, e o arquivo .service tinha sintaxe verbosa e detalhes específicos do Podman. Com o Quadlet, o usuário escreve um arquivo .container (ou .pod, .volume, .network, .kube) com sintaxe simples e declarativa, salva em /etc/containers/systemd/ (rootful) ou ~/.config/containers/systemd/ (rootless), e o gerador do systemd transforma isso em unit no momento da inicialização. As alternativas A, B e D confundem o conceito; a E erra ao afirmar incompatibilidade com rootless.

Gabarito: Letra C

GABARITO



1. Certo
2. Letra C
3. Errado
4. Letra B
5. Certo
6. Letra D
7. Errado
8. Letra A
9. Certo
10. Letra E
11. Errado
12. Letra C

LISTA DE QUESTÕES



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman adota uma arquitetura sem daemon central, em que cada container é executado como um processo filho direto do cliente que o invocou, dispensando um serviço de longa duração em background como ocorre no modelo do Docker tradicional.

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere que um administrador deseja iniciar um container em modo destacado (background) a partir da imagem oficial do nginx, mapeando a porta 8080 do host para a porta 80 do container. Assinale a alternativa que apresenta o comando Podman correto para essa operação.

a) podman start -d --port 8080:80 nginx

b) podman create -bg -p 8080:80 nginx

c) podman run -d -p 8080:80 nginx

d) podman exec --detach 8080:80 nginx

e) podman launch --background --map 8080:80 nginx

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, em que o container é executado sem privilégios de superusuário, é possível mapear portas privilegiadas do host (abaixo de 1024) diretamente para o container, pois o Podman aplica capabilities específicas que liberam essa restrição imposta pelo kernel.

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o conceito de Pod no Podman, assinale a alternativa correta.

a) Pod é uma forma de empacotar e distribuir imagens de container, equivalente ao formato OCI distribuído via registries.

b) Pod é um agrupamento lógico de um ou mais containers que compartilham namespaces de rede, podendo ser gerenciado como unidade única.

c) Pod é o nome dado ao processo daemon do Podman que coordena a execução dos containers no host.

d) Pod é um sistema de orquestração distribuída que substitui o Kubernetes em ambientes de borda.

e) Pod é uma camada de armazenamento persistente compartilhada entre containers, equivalente aos volumes nomeados.

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman oferece compatibilidade de linha de comando com o Docker, de modo que a maior parte dos subcomandos (run, ps, images, build, pull, push) tem sintaxe equivalente, sendo possível criar um alias do tipo alias docker=podman para reaproveitar scripts e tutoriais sem grandes adaptações.

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a integração entre Podman e systemd para gerenciamento de containers como serviços de sistema, assinale a alternativa correta.

a) O Podman exige um daemon próprio rodando sob systemd, semelhante ao dockerd, para que containers sejam tratados como serviços.

b) A única forma de gerenciar containers Podman via systemd é editar manualmente arquivos unit em /etc/systemd/system, sem suporte oficial de geração automática.

c) Containers Podman não podem ser iniciados automaticamente no boot do sistema, pois dependem sempre de invocação interativa pelo usuário.

d) O Podman fornece o subcomando podman generate systemd para criar arquivos unit a partir de containers existentes, e versões recentes incluem o Quadlet, que descreve containers em arquivos .container interpretados pelo systemd.

e) A integração com systemd só está disponível no modo rootful, sendo incompatível com containers executados em modo rootless.

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O Podman possui um motor de build próprio, denominado Buildah, que utiliza sintaxe declarativa específica (arquivos com extensão .build), sendo incompatível com Dockerfiles legados originalmente escritos para o Docker.

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere que um administrador precisa preservar os dados de um banco PostgreSQL em container Podman entre reinicializações. Sobre as estratégias de armazenamento, assinale a alternativa correta.

a) O Podman suporta tanto bind mounts (que mapeiam diretamente um diretório do host para o container) quanto volumes nomeados (gerenciados pelo próprio Podman em uma área de storage interna).

b) O Podman aceita armazenamento persistente apenas via NFS, ficando vedado o uso de diretórios locais do host para volumes.

c) Volumes em Podman não persistem após o container ser removido, sendo automaticamente apagados junto com ele em qualquer cenário.

d) O subcomando podman volume não existe; o gerenciamento de volumes só pode ser feito pela API REST.

e) O Podman utiliza o sistema de arquivos do container em modo de escrita direto na imagem, sem necessidade de volumes para persistência.

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, a conectividade de rede do container com o mundo externo é provida, por padrão, por um stack em espaço de usuário (slirp4netns ou, em versões mais recentes, pasta), que cria uma interface tap em um namespace de rede sem exigir privilégios de superusuário.

10. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a configuração de registries de imagens no Podman, assinale a alternativa correta.

a) O Podman aceita apenas o Docker Hub como registry de origem; outras fontes precisam ser configuradas no arquivo /etc/hosts.

b) Não há configuração de registries: o usuário deve sempre informar a URL completa da imagem no comando podman pull.

c) O Podman fixa o quay.io como único registry permitido, sem possibilidade de adicionar outros.

d) A lista de registries é definida em /etc/podman/registries.list e é compilada estaticamente no binário, não podendo ser alterada após a instalação.

e) A lista de registries usados quando o nome da imagem é informado sem prefixo é configurada em /etc/containers/registries.conf, podendo incluir Docker Hub, quay.io, registry.redhat.io ou registries privados.

11. (PROF. VINICIUS BORGES / INÉDITA - 2026)

No modo rootless do Podman, o usuário que executa os containers acessa o UID 0 (root) do host por meio de um mecanismo de elevação temporária de privilégios, configurado nos arquivos /etc/sudoers.d, sem necessidade de mapeamentos de namespace.

12. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o Quadlet, recurso introduzido em versões recentes do Podman, assinale a alternativa correta.

a) O Quadlet substitui o systemd no gerenciamento de serviços, oferecendo um init próprio focado em containers.

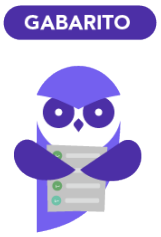
b) O Quadlet é uma ferramenta gráfica para criação de Pods, sem relação com units systemd.

c) O Quadlet permite descrever containers em arquivos declarativos com extensões como .container e .pod, que são traduzidos em units systemd por um gerador na inicialização, evitando a manutenção manual de arquivos service gerados por podman generate systemd.

d) O Quadlet é apenas uma forma alternativa de chamar o subcomando podman play kube, sem novidade arquitetural.

e) O Quadlet exige que o Podman rode em modo rootful, sendo incompatível com containers de usuário.

GABARITO



1. Certo
2. Letra C
3. Errado
4. Letra B
5. Certo
6. Letra D
7. Errado
8. Letra A
9. Certo
10. Letra E
11. Errado
12. Letra C

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.